

CNN-MalGen06

Code Walkthrough, Reconstruction Guide, and Smoke-Test Diagnosis

Brightfield histology tissue segmentation | Technical audit of the uploaded source bundle

Prepared 28 July 2026

Purpose

This document is designed so you can understand the Codex-generated project well enough to reimplement the same scientific model and inputs in your own code. It first explains the end-to-end design and the scientific role of each component, then gives a file-by-file walkthrough and a source-line appendix. It also diagnoses the `AssertionError` from `smoke_test.py` and separates that numerical issue from other project inconsistencies discovered during review.

Scope and exclusions

The walkthrough covers the authored source files `data.py`, `model.py`, `train.py`, `predict.py`, `quantify.py`, `smoke_test.py`, the two SLURM scripts, and `requirements.txt`. `README.md` is summarized rather than annotated line-by-line because it is prose documentation. IDE metadata (`.idea`), compiled Python cache files (`__pycache__`), empty dataset/project folders, and the binary checkpoint are not source code; the checkpoint is instead inspected structurally in the audit section.

Key findings at a glance

- The current model is a 70,971,963-parameter dual-scale semantic segmentation network: ResNet-50 local encoder + ResNet-18 context encoder + nine handcrafted luminance derivatives + a residual bilinear decoder.
- The six final tissue probabilities are hierarchical: the coarse network predicts five parent classes, then a conditional sigmoid splits the shared Fibrocartilage/Muscle parent into two final classes.
- Your smoke-test `AssertionError` is caused by applying a $1e-4$ probability-sum tolerance to probabilities constructed under CUDA float16 autocast. The formula is correct; reduced-precision rounding can produce a worst-case error around $9.77e-4$.
- The strongest code fix is to convert `coarse_logits` and `muscle_logits` to float32 before softmax/sigmoid and before the parent split, rather than casting the already-built probabilities afterward.
- PyTorch 2.12.1 officially pairs with torchvision 0.27.1 and provides a CUDA 13.2 (cu132) wheel. A locally installed CUDA 13.3 toolkit is not, by itself, evidence of a mismatch; a newer NVIDIA driver is backward-compatible with applications built against an older CUDA runtime.
- The included `best_model.pt` is not a checkpoint for the current AccurateTissueNet. It lacks `model_config` and contains old `enc1/up1/classifier`-style weights; current `predict.py` will reject it as legacy.

1. Project map and execution flow

File	Role	Main dependency flow
data.py	Dataset discovery, crop sampling, augmentation, boundary target generation.	train.py -> SegmentationDataset
model.py	Network architecture and probability composition.	train.py / predict.py / smoke_test.py -> AccurateTissueNet
train.py	Losses, metrics, optimization loop, validation, checkpointing.	data.py + model.py
predict.py	Overlapping tiled inference and disk-backed stitching.	model.py + trained checkpoint
quantify.py	Converts hard/probability predictions to calibrated mm ² areas.	prediction outputs + model.CLASS_NAMES
smoke_test.py	Minimal forward/loss/backward integration test.	model.py + train.complete_loss
slurm_*.sbatch	Cluster launch wrappers for training and prediction.	Shell -> Python entry points
requirements.txt	Minimum package versions.	Environment setup

End-to-end data path

1. Input: RGB brightfield image plus a single-channel/indexed PNG mask containing IDs 0-5.
2. Sampling: training chooses Bone/Fibrocartilage boundary-centered, class-centered, or random native-resolution crops. Validation uses deterministic grid locations.
3. Dual scale: a local crop is kept at native resolution; a context field context_scale times wider is extracted around the same center and downsampled to the same tensor size.
4. Feature extraction: local RGB goes through a ResNet-50 pathway augmented by nine fixed derivative/texture maps; context RGB goes through ResNet-18.
5. Fusion: global context modulates the local bottleneck using FiLM-like gamma/beta parameters, and center-aligned context features are concatenated with the local bottleneck.
6. Decoding: bilinear upsampling plus residual convolution blocks restore native output resolution using local encoder skip connections.
7. Heads: five-class coarse logits, conditional Muscle-vs-Fibrocartilage logit, Bone/Fibro boundary logit, and a regional Muscle-presence logit.
8. Training objective: weighted combination of coarse cross entropy, conditional Muscle BCE, final six-class Dice, boundary BCE+Dice, and Muscle-presence BCE.
9. Inference: overlapping tiles are center-weighted and accumulated in memory-mapped arrays, then divided by accumulated blend weights.
10. Quantification: hard-mask pixel counts and probability sums are multiplied by calibrated pixel area to obtain mm².

2. data.py - how training examples are built

The data loader deliberately avoids resizing a whole specimen. Instead it samples native-resolution local patches and derives a wider, spatially aligned context view for the second encoder. The mask is cropped only at local scale, because final segmentation supervision is per native-resolution local pixel.

Sample discovery

discover_samples expects root/images and root/masks. Every accepted image extension must have a same-stem PNG mask. This gives a strict one-to-one mapping and fails early on missing annotations.

Boundary target

bone_fibro_boundary identifies pixels of class 0 next to class 1 and vice versa using eight-neighbour adjacency, then expands that contact band to the requested width. This creates dedicated supervision for the interface used in area measurements.

Sampling mixture

During augmentation, 45% of draws attempt a Bone/Fibro boundary center, the next 35% attempt a randomly selected foreground-class pixel, and the remaining 20% are unbiased random centers. If a requested focus set is empty, code falls through to later strategies.

Context construction

The local crop is crop_size x crop_size. Context is crop_size*context_scale at native scale (default 2048 when crop=512 and scale=4), then Lanczos-resized back to crop_size x crop_size. Thus both encoders receive tensors with identical H,W but very different physical field of view.

Matched augmentation

Horizontal/vertical flips and 90-degree rotations are identical for local, context, and local mask. The same brightness/contrast/saturation/gamma factors are also applied to local and context, preserving cross-scale correspondence.

3. model.py - network architecture

The architecture is not a conventional single U-Net. It is a multi-input hierarchical network that mixes learned RGB features, fixed image-derivative features, local detail, and larger-field context.

Nine handcrafted channels

- Laplacian: second-derivative edge/curvature response
- Gaussian-weighted local deviation: local texture/contrast
- Gradient magnitude: edge strength
- Structure tensor maximum eigenvalue: dominant local orientation energy
- Structure tensor minimum eigenvalue: secondary orientation energy
- Structure coherence: anisotropy/orientation consistency
- Hessian determinant: local second-order blob/saddle structure
- Hessian maximum eigenvalue: stronger principal curvature
- Hessian minimum eigenvalue: weaker principal curvature

These fixed maps are scaled, transformed with asinh , clamped, then passed through BatchNorm and learned convolutions. The design gives the CNN access to explicit morphology-sensitive derivatives without making them hand-engineered final decisions.

Local and context encoders

Local RGB uses a pretrained ResNet-50. Its initial 64-channel RGB stem is concatenated with a 64-channel learned stem derived from the nine handcrafted maps, then fused back to 64 channels before entering ResNet-50 layer1. Wider context uses a pretrained ResNet-18 independently. The local branch therefore prioritizes detail/capacity while the context branch is cheaper.

Context fusion

The context branch contributes in two ways. First, global average pooled context creates FiLM-like gamma and beta vectors that slightly rescale/shift each local bottleneck channel. Second, the central region of the context feature map corresponding to the local field is cropped, resized, projected, and concatenated directly with the local bottleneck. This combines global regional information with spatially centered context evidence.

Decoder and output heads

Four DecoderBlock stages use bilinear interpolation rather than transposed convolutions. Each stage concatenates a local encoder skip feature, then a residual ConvBlock refines the result. After full-resolution interpolation and a final ConvBlock, four heads are produced: five coarse tissue logits, one conditional Muscle logit, one boundary logit, and one regional Muscle-presence logit.

Hierarchical probability equation

Let the five-way coarse softmax be $[p_{\text{Bone}}, p_{\text{parent}}, p_{\text{Cartilage}}, p_{\text{Marrow}}, p_{\text{Background}}]$, where parent means Fibrocartilage-or-Muscle. Let $m = \text{sigmoid}(\text{muscle_logit}) = P(\text{Muscle} \mid \text{parent})$. The final six probabilities are:

```
Bone = p_Bone
Fibrocartilage = p_parent * (1 - m)
Cartilage = p_Cartilage
Muscle = p_parent * m
Marrow = p_Marrow
Background = p_Background
```

In exact arithmetic, Fibrocartilage + Muscle = p_{parent} , so the six final probabilities sum to exactly the five-way softmax sum of 1. This identity is central to the smoke-test diagnosis later.

4. train.py - objective, optimization, and validation

The training code separates the hierarchy into complementary losses rather than asking a single six-way head to learn everything.

Component	Weight	Target	Purpose
Coarse CE	0.40	5-class map; Muscle merged with Fibrocartilage	Primary tissue partition
Conditional Muscle BCE	0.20	Only pixels whose true class is Fibrocartilage or Muscle	Learns the child split inside the parent

Six-class Dice	0.25	Final hierarchical six-class probabilities	Overlap balance, including smaller classes
Boundary BCE + Dice	0.10	Bone/Fibro contact band	Sharper biologically important interface
Muscle presence BCE	0.05	Any Muscle exists in local crop	Regional prior that nudges the pixelwise child logit

Optimization details

- AdamW uses a lower default learning rate for encoder parameters (3e-5) than newly initialized decoder/heads (3e-4).
- CUDA training enables autocast float16 and GradScaler unless --no-amp is passed.
- Gradient accumulation divides each batch loss before backward; optimizer step occurs every accumulation_steps batches or on the last batch.
- Gradients are unscaled before clipping to a norm of 1.0 by default.
- ReduceLROnPlateau monitors validation mean IoU and halves learning rates after six non-improving epochs.
- Best and last checkpoints include model weights, model_config, optimizer/scheduler/scaler state, metrics, epoch, and parsed arguments.

Validation caveat

If --val-data is omitted, the code makes an 85/15 split at the image-file level. The README correctly warns that final experiments should split by patient/specimen to avoid biologically related sections leaking across train and validation.

5. predict.py - native-resolution tiled inference

Prediction is designed for very large histology images without loading a six-channel float32 probability volume entirely into RAM. It combines overlapping local tiles with matched large-context crops and disk-backed accumulators.

Tile coverage

tile_starts advances by tile_size-overlap and forces a final start at length-tile_size, guaranteeing right/bottom edge coverage.

Center weighting

blend_window uses a 2D Hann window so tile centers contribute more strongly than tile borders. A 0.05 floor prevents zero total weights at the image outer boundary.

Batch staging

Pinned host tensors are preallocated when using CUDA. Each uint8 tile/context is copied into float32 host buffers and divided by 255 before asynchronous device transfer.

Disk-backed stitching

Six class sums, one boundary sum, and one weight sum are np.memmap/open_memmap arrays. Each tile contributes probability*window. After all tiles, each pixel is divided by its accumulated weight.

Output encoding

Hard labels are argmax IDs; color_mask.png uses a six-color palette; each class probability and the boundary probability are saved as 8-bit grayscale PNGs. probabilities.npy retains float32 probabilities for quantitative use.

6. quantify.py - area measurement

Hard area uses the number of argmax pixels per class. Expected/probability-weighted area sums each class probability over all pixels, which preserves uncertainty rather than forcing every pixel to a single class. Pixel area is width_um * height_um / 1,000,000 to convert square micrometers to square millimeters. The final CSV reports class pixels, mm², and percentages among non-background tissue.

7. Why smoke_test.py raises AssertionError on CUDA

The failing assertion is line 66:

```
maximum_sum_error = probabilities.sum(1).sub(1).abs().max().detach().item()
assert maximum_sum_error < 1e-4
```

This tests a correct mathematical invariant, but the tolerance assumes near-float32 arithmetic. On CUDA, smoke_test.py unconditionally sets amp=True and computes both the model outputs and complete_loss inside torch.autocast(... dtype=torch.float16).

What happens numerically

The five-way softmax is split into six classes using two floating-point products: parent*(1-m) and parent*m. In exact arithmetic those add to parent. In float16, 1-m, the products, and their later addition are rounded. Float16 spacing around values near 1 is approximately $2^{-10} = 0.0009765625$, already larger than the smoke test threshold of 0.0001.

Reproduction performed during this audit

Calculation	Observed maximum sum error	Interpretation
Actual smoke_test forward on CPU/float32 in audit environment	2.38e-07	Passes 1e-4 easily
Same hierarchical probability formula with synthetic float16 logits	9.765625e-04	Fails 1e-4 due to half-precision rounding
Same synthetic logits converted to float32 before softmax/sigmoid	2.38e-07	Restores float32-level normalization

The audit environment used PyTorch 2.10.0 CPU for the full smoke forward and independent synthetic tensors for the half-precision arithmetic reproduction; therefore this is a reproduction of the numerical mechanism, not a claim that your exact GPU produced exactly the same value. The magnitude matches what float16 representation predicts.

Why line 114 in train.py does not prevent it

```
probabilities = model.proBABILITIES_from_outputs(outputs).float()
```

The `.float()` is applied after probabilities_from_outputs has already executed the softmax/sigmoid and parent split under autocast. Converting a rounded float16 result to float32 preserves the rounded value; it cannot reconstruct bits that were already lost.

Recommended fix

Make probability composition explicitly float32 at its source. This keeps the invariant accurate in training, smoke testing, and prediction:

```
@staticmethod
def probabilities_from_outputs(outputs: dict[str, torch.Tensor]) -> torch.Tensor:
    coarse = outputs["coarse_logits"].float().softmax(dim=1)
    muscle_given_parent = outputs["muscle_logits"].float().sigmoid()
    parent = coarse[:, 1:2]
    return torch.cat(
        [
            coarse[:, 0:1],
            parent * (1 - muscle_given_parent),
            coarse[:, 2:3],
            parent * muscle_given_parent,
            coarse[:, 3:4],
            coarse[:, 4:5],
        ],
        dim=1,
    )
```

This is preferable to merely changing the smoke-test threshold, because the source comment in train.py explicitly intends the final Dice probability computation to be float32, and prediction also benefits from the more accurate normalized probabilities.

Diagnostic confirmation before editing

A simple confirmation is to temporarily set `amp = False` at smoke_test.py line 27 and rerun on CUDA. If the assertion disappears, that directly isolates the issue to autocast precision rather than model algebra. You can also run the smoke test on CPU, where this script leaves AMP disabled.

8. Your PyTorch 2.12.1+cu132 and CUDA 13.3 environment

As of 28 July 2026, PyTorch's official previous-versions page lists torch 2.12.1 with torchvision 0.27.1 for Linux/Windows CUDA 13.2 wheels (`cu132`). PyTorch 2.12 introduced CUDA 13.2 as a supported/experimental binary option. NVIDIA documents CUDA 13.x driver compatibility and states that newer drivers are backward-compatible with applications compiled against earlier CUDA toolkits/runtimes.

Therefore, having CUDA Toolkit 13.3 installed locally while using `torch==2.12.1+cu132` is not the cause of this assertion. The fact that your program reached line 66 after a complete forward/loss calculation also shows that CUDA tensor execution was already working. What matters most is that your PyTorch and torchvision builds are paired and the NVIDIA driver is sufficiently new.

Recommended environment check:

```
python -c "import torch, torchvision; print('torch', torch.__version__); print('torch CUDA runtime', torch.version.cuda);
print('torchvision', torchvision.__version__); print('CUDA available', torch.cuda.is_available()); print('GPU',
torch.cuda.get_device_name(0) if torch.cuda.is_available() else 'none')"
```

For the exact torch 2.12.1 pairing, torchvision should be 0.27.1. `torch.version.cuda` should report the runtime against which that PyTorch wheel was built (13.2 for a cu132 wheel), even if the separately installed CUDA Toolkit is 13.3.

9. Additional code-audit findings before you rewrite

Legacy checkpoint mismatch

The uploaded `best_model.pt` contains keys ``model``, ``classes``, and ``val_miou`` but no ``model_config``; its state dict has only about 118,805 tensor elements and old names such as ``enc1``, ``enc2``, ``up1``, ``up2``, and ``classifier``. Current `AccurateTissueNet` has about 70.97 million parameters and `predict.py` lines 371-375 deliberately reject a checkpoint without `model_config`. You will need a checkpoint produced by the current `train.py` before current `predict.py` can be used.

Requirements are too broad for reproducibility

`requirements.txt` says only `torch>=2.2` and `torchvision>=0.17`. For a paper, freeze the final environment (at minimum exact `torch`, `torchvision`, `numpy`, `Pillow`, `Python`, `CUDA wheel/runtime`, `driver`, `OS`, and `GPU model`) in an `environment/lock` file or `methods` supplement.

Smoke-test gradient check is weak

The backward check uses ``any(...)``, so it passes if just one parameter has a finite gradient. A stronger integration test should require every trainable parameter that is expected to receive a gradient to have a finite gradient, or explicitly list intentionally disconnected parameters.

Probability precision should be fixed in one central method

Both training and prediction call `probabilities_from_outputs`. Fixing `dtype` there avoids separate `train/predict` behavior and makes the mathematical normalization contract explicit.

Validation sampling is patch-based

Validation uses a fixed grid of ``val_patches_per_image``, not necessarily every pixel of each validation image. That is efficient for training feedback but should not be confused with full-specimen validation. For a paper, use full-slide/tiled inference for final specimen-level performance and area analysis.

Muscle-presence target is local-crop presence

The target is ``np.any(local_mask == 3)``. The head consumes pooled local+context features, but supervision means “Muscle appears in the local output crop,” not “Muscle appears anywhere in the larger context field.” Keep that definition explicit in your rewrite.

10. A clean-room-style rewrite plan for you

The safest way to make the implementation genuinely yours is not to rename variables line-for-line. Rebuild from the scientific specification and verify behavior with small tests. Preserve the model concept and inputs, but choose your own module boundaries, names, abstractions, and tests.

1. Write the data contract first

Define six class IDs/names, required image-mask matching, native pixel scale, crop size, context scale, and mask validity. Write tests for missing masks, invalid IDs, crop borders, and indexed PNG preservation.

2. Reimplement crop sampling independently

Implement a centered crop helper, Bone/Fibro boundary detector, then sampling policies. Verify crop/context centers with synthetic coordinate grids before using histology.

3. Implement handcrafted features as one tested module

Write each derivative kernel/formula from its mathematical definition. Test output shape, finiteness, and responses on simple ramps/edges/blobs.

4. Build local and context encoders

Instantiate ResNet-50/18, decide explicitly where handcrafted features enter, and print tensor shapes at each stage. Do not proceed until shape tests pass for 64, 512, and one larger multiple of 32.

5. Implement context fusion from the equations

Write one module for global FiLM modulation and one for center-aligned spatial context. Unit-test crop correspondence using synthetic feature maps.

6. Build your decoder

Use bilinear resize + skip concatenation + residual convolution blocks. Choose your own class/API names while matching channel dimensions and outputs.

7. Implement the hierarchy in float32 by contract

Document that `probabilities_from_logits` returns float32 six-class probabilities summing to one within a small tolerance, regardless of AMP state. Unit-test random fp16 and fp32 logits.

8. Write losses independently

Implement coarse mapping, conditional child BCE, Dice, boundary loss, and presence loss. Test each loss on perfect and deliberately wrong toy predictions.

9. Write the epoch runner

Add device transfer, channels-last optionally, autocast around the expensive model, but compute numerically sensitive probability/loss reductions in float32. Add gradient checks and reproducible seeds.

10. Write tiled inference after training is stable

Test tile coverage and blend normalization on a synthetic image where a mock model returns known probabilities. Only then connect the real network.

11. Write quantification last

Verify mm^2 conversion on hand-calculated toy masks and confirm probability-weighted areas sum appropriately.

12. Record provenance for the paper

Keep a dated design notebook/commit history showing your own implementation decisions and validation. Follow your journal/institution rules on disclosure of AI-assisted code/design.

11. Model specification you can code from

Use this as a functional specification rather than copying the original syntax:

- Input A: $N \times 3 \times H \times W$ native-resolution RGB local patch; H, W multiples of 32.
- Input B: $N \times 3 \times H \times W$ RGB context image representing a `context_scale`-times wider native field centered on Input A and downsampled to $H \times W$.

- Local backbone: ResNet-50 initialized from ImageNet weights; preserve layer1-layer4 skip features.
- Handcrafted branch: nine luminance-derived channels -> BatchNorm -> learned 32/64-channel stem; fuse with the 64-channel RGB stem before ResNet-50 layer1.
- Context backbone: ResNet-18 initialized from ImageNet weights to a 512-channel deepest feature map.
- Global context modulation: pooled 512-d context -> MLP -> 512 gamma + 512 beta; bounded gamma ($\sim 0.1 * \tanh$) and scaled beta applied to the 512-channel projected local bottleneck.
- Spatial context fusion: crop center $1/\text{context_scale}$ fraction of the deepest context map, resize to local bottleneck spatial shape, project 512->256, concatenate with local 512, fuse 768->512.
- Decoder: 512->512 with local layer3 skip (1024); then 512->256 with local layer2 skip (512); 256->128 with local layer1 skip (256); 128->64 with fused stem skip (64); final full-resolution 64-channel refinement.
- Heads: 5-channel coarse 1x1 conv; 1-channel conditional Muscle conv with additive regional-presence prior; 1-channel Bone/Fibro boundary conv; pooled local+context MLP for regional Muscle presence.
- Final probabilities: split the coarse Fibrocartilage-or-Muscle parent using a sigmoid child probability, performing this composition in float32.

12. External version/compatibility references

[1] PyTorch, “Previous PyTorch Versions,” v2.12.1 section: torch==2.12.1 paired with torchvision==0.27.1; CUDA 13.2 install index cu132. <https://pytorch.org/get-started/previous-versions/>

[2] PyTorch Foundation, “PyTorch 2.12 Release Blog,” 13 May 2026: CUDA 13.2 added as a PyTorch 2.12 binary option. <https://pytorch.org/blog/pytorch-2-12-release-blog/>

[3] NVIDIA, “CUDA Toolkit 13.3 Update 1 - Release Notes,” compatibility tables and driver notes. <https://docs.nvidia.com/cuda/cuda-toolkit-release-notes/>

[4] NVIDIA, “CUDA Compatibility,” backward-compatibility guidance: newer drivers can run applications built against older CUDA runtimes/toolkits. <https://docs.nvidia.com/deploy/cuda-compatibility/>

Appendix A. Source-line walkthrough

Every nonblank source line in the authored scripts is reproduced with its original line number and a concise explanation. Blank lines are intentionally omitted from the appendix because they carry only visual spacing; original line numbers are preserved, so you can map every explanation back to the uploaded files. For multi-line expressions, some lines necessarily describe how they continue or close the statement started immediately above; the function-level chapters provide the deeper scientific interpretation.

data.py

Major definitions: `discover_samples`, `_neighbor_any`, `bone_fibro_boundary`, `_extract_centered`, `_color_jitter`, `SegmentationDataset`, `__init__`, `__len__`, `_select_center`, `__getitem__`

L001 `from __future__ import annotations`

Explanation: Postpones evaluation of type annotations, reducing runtime coupling and allowing forward references.

L003 `import math`

Explanation: Imports dependencies used by this module: mathematical helpers

L004 `import random`

Explanation: Imports dependencies used by this module: Python pseudo-random sampling/augmentation

L005 `from pathlib import Path`

Explanation: Imports dependencies used by this module: filesystem paths

L007 `import numpy as np`

Explanation: Imports dependencies used by this module: array and numerical operations

L008 `import torch`

Explanation: Imports dependencies used by this module: tensor computation and automatic differentiation

L009 `from PIL import Image`

Explanation: Imports dependencies used by this module: image I/O and resizing

L010 `from torch.utils.data import Dataset`

Explanation: Imports dependencies used by this module: tensor computation and automatic differentiation; PyTorch dataset protocol

L013 `Image.MAX_IMAGE_PIXELS = None`

Explanation: Assigns the result of this expression to `Image.MAX_IMAGE_PIXELS` for use later in `data.py`.

L014 `IMAGE_SUFFIXES = {".png", ".jpg", ".jpeg", ".tif", ".tiff", ".bmp"}`

Explanation: Assigns `IMAGE_SUFFIXES` as the accepted source-image filename extensions.

L015 `BACKGROUND_ID = 5`

Explanation: Assigns `BACKGROUND_ID` as the numeric class ID used for background padding.

L018 `def discover_samples(root: str | Path) -> list[tuple[Path, Path]]:`

Explanation: Finds source images and their same-stem PNG masks, validating that both directories and every mask exist.

L019 `root = Path(root)`

Explanation: Normalizes/stores filesystem path `root`.

L020 `image_dir, mask_dir = root / "images", root / "masks"`

Explanation: Continues the implementation of `discover_samples`; this line is part of the surrounding multi-line expression or statement.

L021 `if not image_dir.is_dir() or not mask_dir.is_dir():`

Explanation: Conditional branch: evaluates this condition and executes the indented block only when it is true.

L022 `raise FileNotFoundError(f"Expected {image_dir} and {mask_dir}")`

Explanation: Stops execution with an explicit error because an input/state invariant is violated.

L023 `samples: list[tuple[Path, Path]] = []`

Explanation: Assigns the result of this expression to `samples` for use later in `discover_samples`.

L024 `for image_path in sorted(image_dir.iterdir()):`

Explanation: Loop: iterates over the specified sequence/range and executes the following block for each item.

L025 `if image_path.suffix.lower() not in IMAGE_SUFFIXES:`

Explanation: Conditional branch: evaluates this condition and executes the indented block only when it is true.

L026 `continue`

Explanation: Skips the rest of the current loop iteration and proceeds to the next item.

L027 `mask_path = mask_dir / f"{image_path.stem}.png"`

Explanation: Assigns the result of this expression to `mask_path` for use later in `discover_samples`.

L028 `if not mask_path.exists():`

Explanation: Conditional branch: evaluates this condition and executes the indented block only when it is true.

L029 `raise FileNotFoundError(`

Explanation: Stops execution with an explicit error because an input/state invariant is violated.

L030 `f"Missing mask for {image_path.name}: {mask_path}"`

Explanation: Continues the implementation of `discover_samples`; this line is part of the surrounding multi-line expression or statement.

L031 `)`

Explanation: Closes the multi-line call/list/tuple/array started above.

L032 samples.append((image_path, mask_path))

Explanation: Continues the implementation of `discover\_samples`; this line is part of the surrounding multi-line expression or statement.

L033 if not samples:

Explanation: Conditional branch: evaluates this condition and executes the indented block only when it is true.

L034 raise RuntimeError(f"No image/mask pairs found under {root}")

Explanation: Stops execution with an explicit error because an input/state invariant is violated.

L035 return samples

Explanation: Returns the computed value(s) to the caller.

L038 def _neighbor_any(mask: np.ndarray) -> np.ndarray:

Explanation: Builds an 8-neighbour dilation-like Boolean map: each pixel becomes true if any surrounding pixel is true.

L039 padded = np.pad(mask, 1, mode="constant", constant_values=False)

Explanation: Computes NumPy quantity `padded` used by the surrounding `\_neighbor\_any` logic.

L040 result = np.zeros_like(mask, dtype=bool)

Explanation: Computes NumPy quantity `result` used by the surrounding `\_neighbor\_any` logic.

L041 height, width = mask.shape

Explanation: Continues the implementation of `\_neighbor\_any`; this line is part of the surrounding multi-line expression or statement.

L042 for dy in range(3):

Explanation: Loop: iterates over the specified sequence/range and executes the following block for each item.

L043 for dx in range(3):

Explanation: Loop: iterates over the specified sequence/range and executes the following block for each item.

L044 if dy == 1 and dx == 1:

Explanation: Conditional branch: evaluates this condition and executes the indented block only when it is true.

L045 continue

Explanation: Skips the rest of the current loop iteration and proceeds to the next item.

L046 result |= padded[dy : dy + height, dx : dx + width]

Explanation: Continues the implementation of `\_neighbor\_any`; this line is part of the surrounding multi-line expression or statement.

L047 return result

Explanation: Returns the computed value(s) to the caller.

L050 def bone_fibro_boundary(mask: np.ndarray, width: int = 2) -> np.ndarray:

Explanation: Creates a binary supervision map around direct Bone/Fibrocartilage contacts and optionally widens it.

L051 bone = mask == 0

Explanation: Assigns the result of this expression to `bone` for use later in `bone\_fibro\_boundary`.

L052 fibro = mask == 1

Explanation: Assigns the result of this expression to `fibro` for use later in `bone\_fibro\_boundary`.

L053 boundary = (bone & _neighbor_any(fibro)) | (fibro & _neighbor_any(bone))

Explanation: Assigns the result of this expression to `boundary` for use later in `bone\_fibro\_boundary`.

L054 for _ in range(max(0, width - 1)):

Explanation: Loop: iterates over the specified sequence/range and executes the following block for each item.

L055 boundary |= _neighbor_any(boundary)

Explanation: Continues the implementation of `bone\_fibro\_boundary`; this line is part of the surrounding multi-line expression or statement.

L056 return boundary

Explanation: Returns the computed value(s) to the caller.

L059 def _extract_centered(

Explanation: Extracts a square crop centered at a requested coordinate and pads beyond image borders.

L060 array: np.ndarray,

Explanation: Continues the surrounding multi-line call/container with this argument or value.

L061 center_y: int,

Explanation: Continues the surrounding multi-line call/container with this argument or value.

L062 center_x: int,

Explanation: Continues the surrounding multi-line call/container with this argument or value.

L063 size: int,

Explanation: Continues the surrounding multi-line call/container with this argument or value.

L064 constant_value: int | None = None,

Explanation: Assigns the result of this expression to `constant\_value` for use later in `\_extract\_centered`.

L065)-> np.ndarray:

Explanation: Closes or continues the surrounding multi-line Python expression.

L066 height, width = array.shape[:2]

Explanation: Continues the implementation of `\_extract\_centered`; this line is part of the surrounding multi-line expression or statement.

L067 top = center_y - size // 2

Explanation: Assigns the result of this expression to `top` for use later in `\_extract\_centered`.

L068 left = center_x - size // 2

Explanation: Assigns the result of this expression to `left` for use later in `\_extract\_centered`.

L069 bottom, right = top + size, left + size

Explanation: Continues the implementation of `\_extract\_centered`; this line is part of the surrounding multi-line expression or statement.

L070 pad_top, pad_left = max(0, -top), max(0, -left)

Explanation: Continues the implementation of `\_extract\_centered`; this line is part of the surrounding multi-line expression or statement.

L071 pad_bottom, pad_right = max(0, bottom - height), max(0, right - width)

Explanation: Continues the implementation of `\_extract\_centered`; this line is part of the surrounding multi-line expression or statement.

L072 top, left = max(0, top), max(0, left)

Explanation: Continues the implementation of `\_extract\_centered`; this line is part of the surrounding multi-line expression or statement.

L073 bottom, right = min(height, bottom), min(width, right)

Explanation: Continues the implementation of `\_extract\_centered`; this line is part of the surrounding multi-line expression or statement.

L074 crop = array[top:bottom, left:right]

Explanation: Assigns the result of this expression to `crop` for use later in `\_extract\_centered`.

L075 if pad_top or pad_bottom or pad_left or pad_right:

Explanation: Conditional branch: evaluates this condition and executes the indented block only when it is true.

L076 padding = [(pad_top, pad_bottom), (pad_left, pad_right)]

Explanation: Assigns the result of this expression to `padding` for use later in `\_extract\_centered`.

L077 if array.ndim == 3:

Explanation: Conditional branch: evaluates this condition and executes the indented block only when it is true.

L078 padding.append((0, 0))

Explanation: Continues the implementation of `\_extract\_centered`; this line is part of the surrounding multi-line expression or statement.

L079 if constant_value is None:

Explanation: Conditional branch: evaluates this condition and executes the indented block only when it is true.

L080 crop = np.pad(crop, padding, mode="edge")

Explanation: Computes NumPy quantity `crop` used by the surrounding `\_extract\_centered` logic.

L081 else:

Explanation: Fallback branch used when the preceding condition is false.

L082 crop = np.pad(

Explanation: Computes NumPy quantity `crop` used by the surrounding `\_extract\_centered` logic.

L083 crop,

Explanation: Continues the surrounding multi-line argument, tuple, list, or unpacking expression.

L084 padding,

Explanation: Continues the surrounding multi-line argument, tuple, list, or unpacking expression.

L085 mode="constant",

Explanation: Assigns the result of this expression to `mode` for use later in `\_extract\_centered`.

L086 constant_values=constant_value,

Explanation: Assigns the result of this expression to `constant\_values` for use later in `\_extract\_centered`.

L087)

Explanation: Closes the multi-line call/list/tuple/array started above.

L088 return np.ascontiguousarray(crop)

Explanation: Returns the computed value(s) to the caller.

L091 def _color_jitter(

Explanation: Applies matched brightness, contrast, saturation, and gamma augmentation in normalized RGB space.

L092 `image: np.ndarray,`
Explanation: Continues the surrounding multi-line call/container with this argument or value.

L093 `brightness: float,`
Explanation: Continues the surrounding multi-line call/container with this argument or value.

L094 `contrast: float,`
Explanation: Continues the surrounding multi-line call/container with this argument or value.

L095 `saturation: float,`
Explanation: Continues the surrounding multi-line call/container with this argument or value.

L096 `gamma: float,`
Explanation: Continues the surrounding multi-line call/container with this argument or value.

L097 `) -> np.ndarray:`
Explanation: Closes or continues the surrounding multi-line Python expression.

L098 `image = image.astype(np.float32) / 255.0`
Explanation: Computes NumPy quantity `image` used by the surrounding `\_color\_jitter` logic.

L099 `image *= brightness`
Explanation: Continues the implementation of `\_color\_jitter`; this line is part of the surrounding multi-line expression or statement.

L100 `channel_mean = image.mean(axis=(0, 1), keepdims=True)`
Explanation: Assigns the result of this expression to `channel\_mean` for use later in `\_color\_jitter`.

L101 `image = (image - channel_mean) * contrast + channel_mean`
Explanation: Assigns the result of this expression to `image` for use later in `\_color\_jitter`.

L102 `gray = (`
Explanation: Assigns the result of this expression to `gray` for use later in `\_color\_jitter`.

L103 `0.2126 * image[..., 0:1]`
Explanation: Continues the implementation of `\_color\_jitter`; this line is part of the surrounding multi-line expression or statement.

L104 `+ 0.7152 * image[..., 1:2]`
Explanation: Continues the implementation of `\_color\_jitter`; this line is part of the surrounding multi-line expression or statement.

L105 `+ 0.0722 * image[..., 2:3]`
Explanation: Continues the implementation of `\_color\_jitter`; this line is part of the surrounding multi-line expression or statement.

L106 `)`
Explanation: Closes the multi-line call/list/tuple/array started above.

L107 `image = gray + saturation * (image - gray)`
Explanation: Assigns the result of this expression to `image` for use later in `\_color\_jitter`.

L108 `image = np.power(np.clip(image, 0, 1), gamma)`
Explanation: Computes NumPy quantity `image` used by the surrounding `\_color\_jitter` logic.

L109 `return np.clip(image, 0, 1)`
Explanation: Returns the computed value(s) to the caller.

L112 `class SegmentationDataset(Dataset):`
Explanation: PyTorch Dataset that samples aligned local/context crops, mask labels, boundary targets, and muscle-presence targets.

L113 `"""`
Explanation: Starts/continues the docstring documenting the surrounding class or function.

L114 `Returns aligned native-resolution local crops, 1:context_scale context crops`
Explanation: Continues the implementation of `SegmentationDataset`; this line is part of the surrounding multi-line expression or statement.

L115 `resized to the local dimensions, class masks, and Bone/Fibro boundaries.`
Explanation: Continues the implementation of `SegmentationDataset`; this line is part of the surrounding multi-line expression or statement.

L116 `"""`
Explanation: Starts/continues the docstring documenting the surrounding class or function.

L118 `def __init__(`
Explanation: Validates dataset settings and stores the discovered sample list.

L119 `self,`
Explanation: Continues the surrounding multi-line argument, tuple, list, or unpacking expression.

L120 `root: str | Path,`

Explanation: Continues the surrounding multi-line call/container with this argument or value.

L121 augment: bool = False,

Explanation: Assigns the result of this expression to `augment` for use later in `\_\_init\_\_`.

L122 crop_size: int = 512,

Explanation: Assigns the result of this expression to `crop\_size` for use later in `\_\_init\_\_`.

L123 context_scale: int = 4,

Explanation: Assigns the result of this expression to `context\_scale` for use later in `\_\_init\_\_`.

L124 boundary_focus_probability: float = 0.45,

Explanation: Assigns the result of this expression to `boundary\_focus\_probability` for use later in `\_\_init\_\_`.

L125 class_focus_probability: float = 0.35,

Explanation: Assigns the result of this expression to `class\_focus\_probability` for use later in `\_\_init\_\_`.

L126 patches_per_image: int = 2,

Explanation: Assigns the result of this expression to `patches\_per\_image` for use later in `\_\_init\_\_`.

L127 sample_indices: list[int] | None = None,

Explanation: Assigns the result of this expression to `sample\_indices` for use later in `\_\_init\_\_`.

L128) -> None:

Explanation: Closes or continues the surrounding multi-line Python expression.

L129 if crop_size <= 0 or crop_size % 32:

Explanation: Conditional branch: evaluates this condition and executes the indented block only when it is true.

L130 raise ValueError("crop_size must be a positive multiple of 32")

Explanation: Stops execution with an explicit error because an input/state invariant is violated.

L131 if context_scale < 1:

Explanation: Conditional branch: evaluates this condition and executes the indented block only when it is true.

L132 raise ValueError("context_scale must be at least 1")

Explanation: Stops execution with an explicit error because an input/state invariant is violated.

L133 self.root = Path(root)

Explanation: Stores `root` on the module/dataset instance for reuse by later methods.

L134 self.augment = augment

Explanation: Stores `augment` on the module/dataset instance for reuse by later methods.

L135 self.crop_size = crop_size

Explanation: Stores `crop\_size` on the module/dataset instance for reuse by later methods.

L136 self.context_scale = context_scale

Explanation: Stores `context\_scale` on the module/dataset instance for reuse by later methods.

L137 self.boundary_focus_probability = boundary_focus_probability

Explanation: Stores `boundary\_focus\_probability` on the module/dataset instance for reuse by later methods.

L138 self.class_focus_probability = class_focus_probability

Explanation: Stores `class\_focus\_probability` on the module/dataset instance for reuse by later methods.

L139 self.patches_per_image = max(1, patches_per_image)

Explanation: Stores `patches\_per\_image` on the module/dataset instance for reuse by later methods.

L140 all_samples = discover_samples(root)

Explanation: Assigns the result of this expression to `all\_samples` for use later in `\_\_init\_\_`.

L141 self.samples = (

Explanation: Stores `samples` on the module/dataset instance for reuse by later methods.

L142 [all_samples[index] for index in sample_indices]

Explanation: Continues the implementation of `\_\_init\_\_`; this line is part of the surrounding multi-line expression or statement.

L143 if sample_indices is not None

Explanation: Conditional branch: evaluates this condition and executes the indented block only when it is true.

L144 else all_samples

Explanation: Continues the implementation of `\_\_init\_\_`; this line is part of the surrounding multi-line expression or statement.

L145)

Explanation: Closes the multi-line call/list/tuple/array started above.

L147 def __len__(self) -> int:

Explanation: Reports the virtual number of sampled patches, not just the number of whole slides/images.

L148 return len(self.samples) * self.patches_per_image

Explanation: Returns the computed value(s) to the caller.

L150 def _select_center(

Explanation: Chooses deterministic validation centers or biased/random training centers.

L151 self, mask: np.ndarray, patch_number: int

Explanation: Continues the implementation of `\_select\_center`; this line is part of the surrounding multi-line expression or statement.

L152)-> tuple[int, int]:

Explanation: Closes or continues the surrounding multi-line Python expression.

L153 height, width = mask.shape

Explanation: Continues the implementation of `\_select\_center`; this line is part of the surrounding multi-line expression or statement.

L154 if not self.augment:

Explanation: Conditional branch: evaluates this condition and executes the indented block only when it is true.

L155 grid = math.ceil(math.sqrt(self.patches_per_image))

Explanation: Assigns the result of this expression to `grid` for use later in `\_select\_center`.

L156 row, column = divmod(patch_number, grid)

Explanation: Continues the implementation of `\_select\_center`; this line is part of the surrounding multi-line expression or statement.

L157 center_y = round((row + 0.5) * height / grid)

Explanation: Assigns the result of this expression to `center\_y` for use later in `\_select\_center`.

L158 center_x = round((column + 0.5) * width / grid)

Explanation: Assigns the result of this expression to `center\_x` for use later in `\_select\_center`.

L159 return min(height - 1, center_y), min(width - 1, center_x)

Explanation: Returns the computed value(s) to the caller.

L161 choice = random.random()

Explanation: Assigns the result of this expression to `choice` for use later in `\_select\_center`.

L162 if choice < self.boundary_focus_probability:

Explanation: Conditional branch: evaluates this condition and executes the indented block only when it is true.

L163 boundary_y, boundary_x = np.nonzero(

Explanation: Continues the implementation of `\_select\_center`; this line is part of the surrounding multi-line expression or statement.

L164 bone_fibro_boundary(mask, width=1)

Explanation: Continues the implementation of `\_select\_center`; this line is part of the surrounding multi-line expression or statement.

L165)

Explanation: Closes the multi-line call/list/tuple/array started above.

L166 if boundary_y.size:

Explanation: Conditional branch: evaluates this condition and executes the indented block only when it is true.

L167 selected = random.randrange(boundary_y.size)

Explanation: Assigns the result of this expression to `selected` for use later in `\_select\_center`.

L168 jitter = self.crop_size // 4

Explanation: Assigns the result of this expression to `jitter` for use later in `\_select\_center`.

L169 center_y = int(boundary_y[selected]) + random.randint(

Explanation: Assigns the result of this expression to `center\_y` for use later in `\_select\_center`.

L170 -jitter, jitter

Explanation: Continues the implementation of `\_select\_center`; this line is part of the surrounding multi-line expression or statement.

L171)

Explanation: Closes the multi-line call/list/tuple/array started above.

L172 center_x = int(boundary_x[selected]) + random.randint(

Explanation: Assigns the result of this expression to `center\_x` for use later in `\_select\_center`.

L173 -jitter, jitter

Explanation: Continues the implementation of `\_select\_center`; this line is part of the surrounding multi-line expression or statement.

L174)

Explanation: Closes the multi-line call/list/tuple/array started above.

L175 return (

Explanation: Returns the computed value(s) to the caller.

L176 min(height - 1, max(0, center_y)),

Explanation: Continues the surrounding multi-line call/container with this argument or value.

L177 min(width - 1, max(0, center_x)),

Explanation: Continues the surrounding multi-line call/container with this argument or value.

L178)

Explanation: Closes the multi-line call/list/tuple/array started above.

L180 if choice < (

Explanation: Conditional branch: evaluates this condition and executes the indented block only when it is true.

L181 self.boundary_focus_probability + self.class_focus_probability

Explanation: Continues the implementation of `\_select\_center`; this line is part of the surrounding multi-line expression or statement.

L182):

Explanation: Closes or continues the surrounding multi-line Python expression.

L183 present = np.unique(mask)

Explanation: Computes NumPy quantity `present` used by the surrounding `\_select\_center` logic.

L184 foreground = present[present != BACKGROUND_ID]

Explanation: Assigns the result of this expression to `foreground` for use later in `\_select\_center`.

L185 selected_class = int(

Explanation: Assigns the result of this expression to `selected\_class` for use later in `\_select\_center`.

L186 random.choice(foreground.tolist() or present.tolist())

Explanation: Continues the implementation of `\_select\_center`; this line is part of the surrounding multi-line expression or statement.

L187)

Explanation: Closes the multi-line call/list/tuple/array started above.

L188 class_y, class_x = np.nonzero(mask == selected_class)

Explanation: Continues the implementation of `\_select\_center`; this line is part of the surrounding multi-line expression or statement.

L189 if class_y.size:

Explanation: Conditional branch: evaluates this condition and executes the indented block only when it is true.

L190 selected = random.randrange(class_y.size)

Explanation: Assigns the result of this expression to `selected` for use later in `\_select\_center`.

L191 return int(class_y[selected]), int(class_x[selected])

Explanation: Returns the computed value(s) to the caller.

L193 return random.randrange(height), random.randrange(width)

Explanation: Returns the computed value(s) to the caller.

L195 def __getitem__(self, index: int) -> dict[str, torch.Tensor | str]:

Explanation: Loads one image/mask pair, samples a local crop and wider context crop, augments them identically, and returns tensors.

L196 sample_index, patch_number = divmod(index, self.patches_per_image)

Explanation: Continues the implementation of `\_\_getitem\_\_`; this line is part of the surrounding multi-line expression or statement.

L197 image_path, mask_path = self.samples[sample_index]

Explanation: Continues the implementation of `\_\_getitem\_\_`; this line is part of the surrounding multi-line expression or statement.

L198 with Image.open(image_path) as image_file:

Explanation: Context manager ensuring the opened resource/state is cleaned up correctly after the block.

L199 image = np.array(image_file.convert("RGB"), dtype=np.uint8, copy=True)

Explanation: Converts image/data into an independent NumPy array stored as `image`.

L200 with Image.open(mask_path) as mask_file:

Explanation: Context manager ensuring the opened resource/state is cleaned up correctly after the block.

L201 # Preserve indexed-PNG class IDs. Converting a palette mask to

Explanation: Developer comment: Preserve indexed-PNG class IDs. Converting a palette mask to

L202 # luminance would silently replace indices with palette brightness.

Explanation: Developer comment: luminance would silently replace indices with palette brightness.

L203 mask = np.array(mask_file, dtype=np.int64, copy=True)

Explanation: Converts image/data into an independent NumPy array stored as `mask`.

L204 if mask.ndim != 2:

Explanation: Conditional branch: evaluates this condition and executes the indented block only when it is true.

L205 raise ValueError(

Explanation: Stops execution with an explicit error because an input/state invariant is violated.

L206 f"{mask_path} must be a single-channel or indexed PNG, "

Explanation: Continues the implementation of `\_\_getitem\_\_`; this line is part of the surrounding multi-line expression or statement.

L207 f"received shape {mask.shape}"

Explanation: Continues the implementation of `\_\_getitem\_\_`; this line is part of the surrounding multi-line expression or statement.

L208)

Explanation: Closes the multi-line call/list/tuple/array started above.

L209 if image.shape[:2] != mask.shape:

Explanation: Conditional branch: evaluates this condition and executes the indented block only when it is true.

L210 raise ValueError(

Explanation: Stops execution with an explicit error because an input/state invariant is violated.

L211 f"Image and mask dimensions differ for {image_path.name}: "

Explanation: Continues the implementation of `\_\_getitem\_\_`; this line is part of the surrounding multi-line expression or statement.

L212 f"{image.shape[:2]} versus {mask.shape}"

Explanation: Continues the implementation of `\_\_getitem\_\_`; this line is part of the surrounding multi-line expression or statement.

L213)

Explanation: Closes the multi-line call/list/tuple/array started above.

L214 if mask.min() < 0 or mask.max() > 5:

Explanation: Conditional branch: evaluates this condition and executes the indented block only when it is true.

L215 raise ValueError(f"{mask_path} contains class IDs outside 0..5")

Explanation: Stops execution with an explicit error because an input/state invariant is violated.

L217 center_y, center_x = self._select_center(mask, patch_number)

Explanation: Continues the implementation of `\_\_getitem\_\_`; this line is part of the surrounding multi-line expression or statement.

L218 local = _extract_centered(

Explanation: Assigns the result of this expression to `local` for use later in `\_\_getitem\_\_`.

L219 image, center_y, center_x, self.crop_size

Explanation: Continues the implementation of `\_\_getitem\_\_`; this line is part of the surrounding multi-line expression or statement.

L220)

Explanation: Closes the multi-line call/list/tuple/array started above.

L221 local_mask = _extract_centered(

Explanation: Assigns the result of this expression to `local\_mask` for use later in `\_\_getitem\_\_`.

L222 mask,

Explanation: Continues the surrounding multi-line argument, tuple, list, or unpacking expression.

L223 center_y,

Explanation: Continues the surrounding multi-line argument, tuple, list, or unpacking expression.

L224 center_x,

Explanation: Continues the surrounding multi-line argument, tuple, list, or unpacking expression.

L225 self.crop_size,

Explanation: Continues the surrounding multi-line call/container with this argument or value.

L226 constant_value=BACKGROUND_ID,

Explanation: Assigns the result of this expression to `constant\_value` for use later in `\_\_getitem\_\_`.

L227)

Explanation: Closes the multi-line call/list/tuple/array started above.

L228 context_native = _extract_centered(

Explanation: Assigns the result of this expression to `context\_native` for use later in `\_\_getitem\_\_`.

L229 image,

Explanation: Continues the surrounding multi-line argument, tuple, list, or unpacking expression.

L230 center_y,

Explanation: Continues the surrounding multi-line argument, tuple, list, or unpacking expression.

L231 center_x,

Explanation: Continues the surrounding multi-line argument, tuple, list, or unpacking expression.

L232 self.crop_size * self.context_scale,

Explanation: Continues the surrounding multi-line call/container with this argument or value.

L233)

Explanation: Closes the multi-line call/list/tuple/array started above.

L234 context_image = Image.fromarray(context_native)

Explanation: Assigns the result of this expression to `context\_image` for use later in `\_\_getitem\_\_`.

L235 context = np.array(

Explanation: Converts image/data into an independent NumPy array stored as `context`.

L236 context_image.resize(

Explanation: Continues the implementation of `\_\_getitem\_\_`; this line is part of the surrounding multi-line expression or statement.

L237 (self.crop_size, self.crop_size),

Explanation: Continues the surrounding multi-line call/container with this argument or value.

L238 Image.Resampling.LANCZOS,

Explanation: Continues the surrounding multi-line call/container with this argument or value.

L239 reducing_gap=2.0,

Explanation: Assigns the result of this expression to `reducing\_gap` for use later in `\_\_getitem\_\_`.

L240),

Explanation: Closes or continues the surrounding multi-line Python expression.

L241 dtype=np.uint8,

Explanation: Computes NumPy quantity `dtype` used by the surrounding `\_\_getitem\_\_` logic.

L242 copy=True,

Explanation: Assigns the result of this expression to `copy` for use later in `\_\_getitem\_\_`.

L243)

Explanation: Closes the multi-line call/list/tuple/array started above.

L244 context_image.close()

Explanation: Continues the implementation of `\_\_getitem\_\_`; this line is part of the surrounding multi-line expression or statement.

L246 if self.augment:

Explanation: Conditional branch: evaluates this condition and executes the indented block only when it is true.

L247 if random.random() < 0.5:

Explanation: Conditional branch: evaluates this condition and executes the indented block only when it is true.

L248 local = local[:, ::-1].copy()

Explanation: Assigns the result of this expression to `local` for use later in `\_\_getitem\_\_`.

L249 context = context[:, ::-1].copy()

Explanation: Assigns the result of this expression to `context` for use later in `\_\_getitem\_\_`.

L250 local_mask = local_mask[:, ::-1].copy()

Explanation: Assigns the result of this expression to `local\_mask` for use later in `\_\_getitem\_\_`.

L251 if random.random() < 0.5:

Explanation: Conditional branch: evaluates this condition and executes the indented block only when it is true.

L252 local = local[:, :1, :].copy()

Explanation: Assigns the result of this expression to `local` for use later in `\_\_getitem\_\_`.

L253 context = context[:, :1, :].copy()

Explanation: Assigns the result of this expression to `context` for use later in `\_\_getitem\_\_`.

L254 local_mask = local_mask[:, :1, :].copy()

Explanation: Assigns the result of this expression to `local\_mask` for use later in `\_\_getitem\_\_`.

L255 rotation = random.randrange(4)

Explanation: Assigns the result of this expression to `rotation` for use later in `\_\_getitem\_\_`.

L256 local = np.rot90(local, rotation).copy()

Explanation: Computes NumPy quantity `local` used by the surrounding `\_\_getitem\_\_` logic.

L257 context = np.rot90(context, rotation).copy()

Explanation: Computes NumPy quantity `context` used by the surrounding `\_\_getitem\_\_` logic.

L258 local_mask = np.rot90(local_mask, rotation).copy()

Explanation: Computes NumPy quantity `local\_mask` used by the surrounding `\_\_getitem\_\_` logic.

L260 brightness = random.uniform(0.85, 1.15)

Explanation: Assigns the result of this expression to `brightness` for use later in `\_\_getitem\_\_`.

L261 contrast = random.uniform(0.85, 1.15)

Explanation: Assigns the result of this expression to `contrast` for use later in `\_\_getitem\_\_`.

L262 saturation = random.uniform(0.75, 1.25)

Explanation: Assigns the result of this expression to `saturation` for use later in `\_\_getitem\_\_`.

L263 gamma = random.uniform(0.85, 1.15)

Explanation: Assigns the result of this expression to `gamma` for use later in `\_\_getitem\_\_`.

L264 local = _color_jitter(

Explanation: Assigns the result of this expression to `local` for use later in `\_\_getitem\_\_`.

L265 local, brightness, contrast, saturation, gamma

Explanation: Continues the implementation of `\_\_getitem\_\_`; this line is part of the surrounding multi-line expression or statement.

L266)

Explanation: Closes the multi-line call/list/tuple/array started above.

L267 context = _color_jitter(

Explanation: Assigns the result of this expression to `context` for use later in `\_\_getitem\_\_`.

L268 context, brightness, contrast, saturation, gamma

Explanation: Continues the implementation of `\_\_getitem\_\_`; this line is part of the surrounding multi-line expression or statement.

L269)

Explanation: Closes the multi-line call/list/tuple/array started above.

L270 else:

Explanation: Fallback branch used when the preceding condition is false.

L271 local = local.astype(np.float32) / 255.0

Explanation: Computes NumPy quantity `local` used by the surrounding `\_\_getitem\_\_` logic.

L272 context = context.astype(np.float32) / 255.0

Explanation: Computes NumPy quantity `context` used by the surrounding `\_\_getitem\_\_` logic.

L274 boundary = bone_fibro_boundary(local_mask, width=3).astype(np.float32)

Explanation: Computes NumPy quantity `boundary` used by the surrounding `\_\_getitem\_\_` logic.

L275 muscle_presence = np.float32(np.any(local_mask == 3))

Explanation: Computes NumPy quantity `muscle\_presence` used by the surrounding `\_\_getitem\_\_` logic.

L276 return {

Explanation: Returns the computed value(s) to the caller.

L277 "local": torch.from_numpy(local).permute(2, 0, 1),

Explanation: Adds this string literal to the surrounding tuple/list/message or multi-line string.

L278 "context": torch.from_numpy(context).permute(2, 0, 1),

Explanation: Adds this string literal to the surrounding tuple/list/message or multi-line string.

L279 "mask": torch.from_numpy(local_mask),

Explanation: Adds this string literal to the surrounding tuple/list/message or multi-line string.

L280 "boundary": torch.from_numpy(boundary),

Explanation: Adds this string literal to the surrounding tuple/list/message or multi-line string.

L281 "muscle_presence": torch.tensor(muscle_presence),

Explanation: Adds this string literal to the surrounding tuple/list/message or multi-line string.

L282 "name": image_path.stem,

Explanation: Adds this string literal to the surrounding tuple/list/message or multi-line string.

L283 }

Explanation: Closes the multi-line container started above.

model.py

Major definitions: ModelConfig, _kernel, HandcraftedFeatures, _group_count, ConvBlock, DecoderBlock, AccurateTissueNet, __init__, _conv, forward, model_config, _normalize_rgb, _context_center_features, probabilities_from_outputs, probabilities

L001 from __future__ import annotations

Explanation: Postpones evaluation of type annotations, reducing runtime coupling and allowing forward references.

L003 from dataclasses import asdict, dataclass

Explanation: Imports dependencies used by this module: dataclass-to-dictionary conversion; compact immutable configuration object

L005 import torch

Explanation: Imports dependencies used by this module: tensor computation and automatic differentiation

L006 from torch import nn

Explanation: Imports dependencies used by this module: tensor computation and automatic differentiation; PyTorch neural-network layers

L007 import torch.nn.functional as F

Explanation: Imports dependencies used by this module: tensor computation and automatic differentiation; PyTorch functional neural-network operations; PyTorch neural-network layers

L008 from torchvision.models import (

Explanation: Imports dependencies used by this module.

L009 ResNet18_Weights,

Explanation: Continues the surrounding multi-line argument, tuple, list, or unpacking expression.

L010 ResNet50_Weights,

Explanation: Continues the surrounding multi-line argument, tuple, list, or unpacking expression.

L011 resnet18,

Explanation: Continues the surrounding multi-line argument, tuple, list, or unpacking expression.

L012 resnet50,

Explanation: Continues the surrounding multi-line argument, tuple, list, or unpacking expression.

L013)

Explanation: Closes the multi-line call/list/tuple/array started above.

L016 CLASS_NAMES = (

Explanation: Assigns `CLASS\_NAMES` as the ordered six-class output label names.

L017 "Bone",

Explanation: Adds this string literal to the surrounding tuple/list/message or multi-line string.

L018 "Fibrocartilage",

Explanation: Adds this string literal to the surrounding tuple/list/message or multi-line string.

L019 "Cartilage",

Explanation: Adds this string literal to the surrounding tuple/list/message or multi-line string.

L020 "Muscle",

Explanation: Adds this string literal to the surrounding tuple/list/message or multi-line string.

L021 "Marrow",

Explanation: Adds this string literal to the surrounding tuple/list/message or multi-line string.

L022 "Background",

Explanation: Adds this string literal to the surrounding tuple/list/message or multi-line string.

L023)

Explanation: Closes the multi-line call/list/tuple/array started above.

L025 COARSE_CLASS_NAMES = (

Explanation: Assigns `COARSE\_CLASS\_NAMES` as the ordered five-class coarse label names where Fibrocartilage and Muscle share a parent.

L026 "Bone",

Explanation: Adds this string literal to the surrounding tuple/list/message or multi-line string.

L027 "Fibrocartilage_or_Muscle",

Explanation: Adds this string literal to the surrounding tuple/list/message or multi-line string.

L028 "Cartilage",

Explanation: Adds this string literal to the surrounding tuple/list/message or multi-line string.

L029 "Marrow",

Explanation: Adds this string literal to the surrounding tuple/list/message or multi-line string.

L030 "Background",

Explanation: Adds this string literal to the surrounding tuple/list/message or multi-line string.

L031)

Explanation: Closes the multi-line call/list/tuple/array started above.

L034 @dataclass(frozen=True)

Explanation: Decorates the following class as an immutable dataclass used for explicit model configuration.

L035 class ModelConfig:

Explanation: Defines class `ModelConfig`.

L036 context_scale: int = 4

Explanation: Assigns the result of this expression to `context\_scale` for use later in `ModelConfig`.

L037 pretrained: bool = True

Explanation: Assigns the result of this expression to `pretrained` for use later in `ModelConfig`.

L038 decoder_channels: int = 64

Explanation: Assigns the result of this expression to `decoder\_channels` for use later in `ModelConfig`.

L041 def _kernel(values: list[list[float]], divisor: float = 1.0) -> torch.Tensor:

Explanation: Converts a 3x3 numeric stencil into a float32 convolution kernel tensor.

L042 return torch.tensor(values, dtype=torch.float32).view(1, 1, 3, 3) / divisor

Explanation: Returns the computed value(s) to the caller.

L045 class HandcraftedFeatures(nn.Module):

Explanation: Computes nine fixed derivative/texture channels from luminance before a learned feature stem.

L046 """Nine fixed feature maps calculated from native-resolution luminance."""

Explanation: Starts/continues the docstring documenting the surrounding class or function.

L048 def __init__(self, eps: float = 1e-6) -> None:

Explanation: Constructs fixed convolution kernels, scaling constants, and/or neural layers for the current class.

L049 super().__init__()

Explanation: Continues the implementation of `\_\_init\_\_`; this line is part of the surrounding multi-line expression or statement.

L050 self.eps = eps

Explanation: Stores `eps` on the module/dataset instance for reuse by later methods.

L051 self.register_buffer(

Explanation: Continues the implementation of `\_\_init\_\_`; this line is part of the surrounding multi-line expression or statement.

L052 "sobel_x", _kernel([[[-1, 0, 1], [-2, 0, 2], [-1, 0, 1]], 8)

Explanation: Adds this string literal to the surrounding tuple/list/message or multi-line string.

L053)

Explanation: Closes the multi-line call/list/tuple/array started above.

L054 self.register_buffer(

Explanation: Continues the implementation of `\_\_init\_\_`; this line is part of the surrounding multi-line expression or statement.

L055 "sobel_y", _kernel([[[-1, -2, -1], [0, 0, 0], [1, 2, 1]], 8)

Explanation: Adds this string literal to the surrounding tuple/list/message or multi-line string.

L056)

Explanation: Closes the multi-line call/list/tuple/array started above.

L057 self.register_buffer(

Explanation: Continues the implementation of `\_\_init\_\_`; this line is part of the surrounding multi-line expression or statement.

L058 "laplace", _kernel([[0, 1, 0], [1, -4, 1], [0, 1, 0]])

Explanation: Adds this string literal to the surrounding tuple/list/message or multi-line string.

L059)

Explanation: Closes the multi-line call/list/tuple/array started above.

L060 self.register_buffer(

Explanation: Continues the implementation of `\_\_init\_\_`; this line is part of the surrounding multi-line expression or statement.

L061 "gaussian", _kernel([[1, 2, 1], [2, 4, 2], [1, 2, 1]], 16)

Explanation: Adds this string literal to the surrounding tuple/list/message or multi-line string.

L062)
Explanation: Closes the multi-line call/list/tuple/array started above.

L063 self.register_buffer(
Explanation: Continues the implementation of `\_\_init\_\_`; this line is part of the surrounding multi-line expression or statement.

L064 "dxx", _kernel([[1, -2, 1], [2, -4, 2], [1, -2, 1]], 4)
Explanation: Adds this string literal to the surrounding tuple/list/message or multi-line string.

L065)
Explanation: Closes the multi-line call/list/tuple/array started above.

L066 self.register_buffer(
Explanation: Continues the implementation of `\_\_init\_\_`; this line is part of the surrounding multi-line expression or statement.

L067 "dyy", _kernel([[1, 2, 1], [-2, -4, -2], [1, 2, 1]], 4)
Explanation: Adds this string literal to the surrounding tuple/list/message or multi-line string.

L068)
Explanation: Closes the multi-line call/list/tuple/array started above.

L069 self.register_buffer(
Explanation: Continues the implementation of `\_\_init\_\_`; this line is part of the surrounding multi-line expression or statement.

L070 "dxy", _kernel([[1, 0, -1], [0, 0, 0], [-1, 0, 1]], 4)
Explanation: Adds this string literal to the surrounding tuple/list/message or multi-line string.

L071)
Explanation: Closes the multi-line call/list/tuple/array started above.

L072 # Fixed scaling avoids the tile-dependent mean/std normalization that
Explanation: Developer comment: Fixed scaling avoids the tile-dependent mean/std normalization that

L073 # created visible seams. BatchNorm in the feature stem learns global
Explanation: Developer comment: created visible seams. BatchNorm in the feature stem learns global

L074 # training-set statistics and freezes them for inference.
Explanation: Developer comment: training-set statistics and freezes them for inference.

L075 self.register_buffer(
Explanation: Continues the implementation of `\_\_init\_\_`; this line is part of the surrounding multi-line expression or statement.

L076 "scales",
Explanation: Adds this string literal to the surrounding tuple/list/message or multi-line string.

L077 torch.tensor(
Explanation: Executes this numerical/layer operation as part of the surrounding expression.

L078 [0.25, 0.10, 0.10, 0.05, 0.05, 1.0, 0.05, 0.25, 0.25],
Explanation: Continues the surrounding multi-line call/container with this argument or value.

L079 dtype=torch.float32,
Explanation: Computes tensor quantity `dtype` using PyTorch operations in the surrounding `\_\_init\_\_` logic.

L080).view(1, 9, 1, 1),
Explanation: Closes or continues the surrounding multi-line Python expression.

L081)
Explanation: Closes the multi-line call/list/tuple/array started above.

L083 @staticmethod
Explanation: Marks the following method as independent of a class instance; it receives no `self` argument.

L084 def _conv(x: torch.Tensor, kernel: torch.Tensor) -> torch.Tensor:
Explanation: Pads by edge replication then applies a 3x3 convolution.

L085 return F.conv2d(F.pad(x, (1, 1, 1, 1), mode="replicate"), kernel)
Explanation: Returns the computed value(s) to the caller.

L087 def forward(self, rgb: torch.Tensor) -> torch.Tensor:
Explanation: Runs the current module forward and returns its output tensors.

L088 if rgb.ndim != 4 or rgb.shape[1] != 3:
Explanation: Conditional branch: evaluates this condition and executes the indented block only when it is true.

L089 raise ValueError(f"Expected [N, 3, H, W], received {tuple(rgb.shape)}")
Explanation: Stops execution with an explicit error because an input/state invariant is violated.

L091 gray = (

Explanation: Assigns the result of this expression to `gray` for use later in `forward`.

L092 0.2126 * rgb[:, 0:1]

Explanation: Continues the implementation of `forward`; this line is part of the surrounding multi-line expression or statement.

L093 + 0.7152 * rgb[:, 1:2]

Explanation: Continues the implementation of `forward`; this line is part of the surrounding multi-line expression or statement.

L094 + 0.0722 * rgb[:, 2:3]

Explanation: Continues the implementation of `forward`; this line is part of the surrounding multi-line expression or statement.

L095)

Explanation: Closes the multi-line call/list/tuple/array started above.

L096 gx = self._conv(gray, self.sobel_x)

Explanation: Assigns the result of this expression to `gx` for use later in `forward`.

L097 gy = self._conv(gray, self.sobel_y)

Explanation: Assigns the result of this expression to `gy` for use later in `forward`.

L098 laplace = self._conv(gray, self.laplace)

Explanation: Assigns the result of this expression to `laplace` for use later in `forward`.

L099 gradient_magnitude = torch.sqrt(gx.square() + gy.square() + self.eps)

Explanation: Computes tensor quantity `gradient\_magnitude` using PyTorch operations in the surrounding forward logic.

L101 local_mean = self._conv(gray, self.gaussian)

Explanation: Assigns the result of this expression to `local\_mean` for use later in `forward`.

L102 local_mean_sq = self._conv(gray.square(), self.gaussian)

Explanation: Assigns the result of this expression to `local\_mean\_sq` for use later in `forward`.

L103 weighted_deviation = torch.sqrt(

Explanation: Computes tensor quantity `weighted\_deviation` using PyTorch operations in the surrounding forward logic.

L104 torch.clamp(local_mean_sq - local_mean.square(), min=0) + self.eps

Explanation: Executes this numerical/layer operation as part of the surrounding expression.

L105)

Explanation: Closes the multi-line call/list/tuple/array started above.

L107 jxx = self._conv(gx.square(), self.gaussian)

Explanation: Assigns the result of this expression to `jxx` for use later in `forward`.

L108 jyy = self._conv(gy.square(), self.gaussian)

Explanation: Assigns the result of this expression to `jyy` for use later in `forward`.

L109 jxy = self._conv(gx * gy, self.gaussian)

Explanation: Assigns the result of this expression to `jxy` for use later in `forward`.

L110 j_trace = jxx + jyy

Explanation: Assigns the result of this expression to `j\_trace` for use later in `forward`.

L111 j_root = torch.sqrt((jxx - jyy).square() + 4 * jxy.square() + self.eps)

Explanation: Computes tensor quantity `j\_root` using PyTorch operations in the surrounding forward logic.

L112 structure_max = 0.5 * (j_trace + j_root)

Explanation: Assigns the result of this expression to `structure\_max` for use later in `forward`.

L113 structure_min = 0.5 * (j_trace - j_root)

Explanation: Assigns the result of this expression to `structure\_min` for use later in `forward`.

L114 coherence = (structure_max - structure_min) / (

Explanation: Assigns the result of this expression to `coherence` for use later in `forward`.

L115 structure_max + structure_min + self.eps

Explanation: Continues the implementation of `forward`; this line is part of the surrounding multi-line expression or statement.

L116)

Explanation: Closes the multi-line call/list/tuple/array started above.

L118 hxx = self._conv(gray, self.dxx)

Explanation: Assigns the result of this expression to `hxx` for use later in `forward`.

L119 hyy = self._conv(gray, self.dyy)

Explanation: Assigns the result of this expression to `hyy` for use later in `forward`.

L120 hxy = self._conv(gray, self.dxy)

Explanation: Assigns the result of this expression to `hxy` for use later in `forward`.

L121 hessian_det = hxx * hyy - hxy.square()

Explanation: Assigns the result of this expression to `hessian\_det` for use later in `forward`.

L122 h_trace = hxx + hyy

Explanation: Assigns the result of this expression to `h\_trace` for use later in `forward`.

L123 h_root = torch.sqrt((hxx - hyy).square() + 4 * hxy.square() + self.eps)

Explanation: Computes tensor quantity `h\_root` using PyTorch operations in the surrounding forward logic.

L124 hessian_max = 0.5 * (h_trace + h_root)

Explanation: Assigns the result of this expression to `hessian\_max` for use later in `forward`.

L125 hessian_min = 0.5 * (h_trace - h_root)

Explanation: Assigns the result of this expression to `hessian\_min` for use later in `forward`.

L127 raw = torch.cat([

Explanation: Computes tensor quantity `raw` using PyTorch operations in the surrounding forward logic.

L128 laplace,

Explanation: Continues the implementation of `forward`; this line is part of the surrounding multi-line expression or statement.

L129 weighted_deviation,

Explanation: Continues the surrounding multi-line argument, tuple, list, or unpacking expression.

L130 gradient_magnitude,

Explanation: Continues the surrounding multi-line argument, tuple, list, or unpacking expression.

L131 structure_max,

Explanation: Continues the surrounding multi-line argument, tuple, list, or unpacking expression.

L132 structure_min,

Explanation: Continues the surrounding multi-line argument, tuple, list, or unpacking expression.

L133 coherence,

Explanation: Continues the surrounding multi-line argument, tuple, list, or unpacking expression.

L134 hessian_det,

Explanation: Continues the surrounding multi-line argument, tuple, list, or unpacking expression.

L135 hessian_max,

Explanation: Continues the surrounding multi-line argument, tuple, list, or unpacking expression.

L136 hessian_min,

Explanation: Continues the surrounding multi-line argument, tuple, list, or unpacking expression.

L137],

Explanation: Continues the surrounding multi-line argument, tuple, list, or unpacking expression.

L138 dim=1,

Explanation: Closes or continues the surrounding multi-line Python expression.

L139)

Explanation: Assigns the result of this expression to `dim` for use later in `forward`.

L140)

Explanation: Closes the multi-line call/list/tuple/array started above.

L141 return torch.asinh(raw / self.scales).clamp(-8, 8)

Explanation: Returns the computed value(s) to the caller.

L144 def _group_count(channels: int) -> int:

Explanation: Selects the largest convenient GroupNorm group count that divides the channel count.

L145 for groups in (16, 8, 4, 2, 1):

Explanation: Loop: iterates over the specified sequence/range and executes the following block for each item.

L146 if channels % groups == 0:

Explanation: Conditional branch: evaluates this condition and executes the indented block only when it is true.

L147 return groups

Explanation: Returns the computed value(s) to the caller.

L148 return 1

Explanation: Returns the computed value(s) to the caller.

L151 class ConvBlock(nn.Module):

Explanation: Two 3x3 conv + GroupNorm + GELU layers with a residual shortcut.

L152 def __init__(self, in_channels: int, out_channels: int) -> None:

Explanation: Constructs fixed convolution kernels, scaling constants, and/or neural layers for the current class.

L153 `super().__init__()`

Explanation: Continues the implementation of `\_\_init\_\_`; this line is part of the surrounding multi-line expression or statement.

L154 `groups = _group_count(out_channels)`

Explanation: Assigns the result of this expression to `groups` for use later in `\_\_init\_\_`.

L155 `self.block = nn.Sequential(`

Explanation: Stores `block` on the module/dataset instance for reuse by later methods.

L156 `nn.Conv2d(in_channels, out_channels, 3, padding=1, bias=False),`

Explanation: Executes this numerical/layer operation as part of the surrounding expression.

L157 `nn.GroupNorm(groups, out_channels),`

Explanation: Executes this numerical/layer operation as part of the surrounding expression.

L158 `nn.GELU(),`

Explanation: Executes this numerical/layer operation as part of the surrounding expression.

L159 `nn.Conv2d(out_channels, out_channels, 3, padding=1, bias=False),`

Explanation: Executes this numerical/layer operation as part of the surrounding expression.

L160 `nn.GroupNorm(groups, out_channels),`

Explanation: Executes this numerical/layer operation as part of the surrounding expression.

L161 `nn.GELU(),`

Explanation: Executes this numerical/layer operation as part of the surrounding expression.

L162 `)`

Explanation: Closes the multi-line call/list/tuple/array started above.

L163 `self.residual = (`

Explanation: Stores `residual` on the module/dataset instance for reuse by later methods.

L164 `nn.Conv2d(in_channels, out_channels, 1, bias=False)`

Explanation: Executes this numerical/layer operation as part of the surrounding expression.

L165 `if in_channels != out_channels`

Explanation: Conditional branch: evaluates this condition and executes the indented block only when it is true.

L166 `else nn.Identity()`

Explanation: Continues the implementation of `\_\_init\_\_`; this line is part of the surrounding multi-line expression or statement.

L167 `)`

Explanation: Closes the multi-line call/list/tuple/array started above.

L169 `def forward(self, x: torch.Tensor) -> torch.Tensor:`

Explanation: Runs the current module forward and returns its output tensors.

L170 `return self.block(x) + self.residual(x)`

Explanation: Returns the computed value(s) to the caller.

L173 `class DecoderBlock(nn.Module):`

Explanation: Upsamples a decoder tensor, projects channels, concatenates a skip connection, and refines it.

L174 `def __init__(`

Explanation: Constructs fixed convolution kernels, scaling constants, and/or neural layers for the current class.

L175 `self, in_channels: int, skip_channels: int, out_channels: int`

Explanation: Continues the implementation of `\_\_init\_\_`; this line is part of the surrounding multi-line expression or statement.

L176 `) -> None:`

Explanation: Closes or continues the surrounding multi-line Python expression.

L177 `super().__init__()`

Explanation: Continues the implementation of `\_\_init\_\_`; this line is part of the surrounding multi-line expression or statement.

L178 `self.project = nn.Conv2d(in_channels, out_channels, 1, bias=False)`

Explanation: Stores `project` on the module/dataset instance for reuse by later methods.

L179 `self.refine = ConvBlock(out_channels + skip_channels, out_channels)`

Explanation: Stores `refine` on the module/dataset instance for reuse by later methods.

L181 `def forward(self, x: torch.Tensor, skip: torch.Tensor) -> torch.Tensor:`

Explanation: Runs the current module forward and returns its output tensors.

L182 `x = F.interpolate(`

Explanation: Computes tensor quantity `x` using PyTorch operations in the surrounding forward logic.

L183 x, size=skip.shape[-2:], mode="bilinear", align_corners=False

Explanation: Continues the implementation of `forward`; this line is part of the surrounding multi-line expression or statement.

L184)

Explanation: Closes the multi-line call/list/tuple/array started above.

L185 x = self.project(x)

Explanation: Assigns the result of this expression to `x` for use later in `forward`.

L186 return self.refine(torch.cat([x, skip], dim=1))

Explanation: Returns the computed value(s) to the caller.

L189 class AccurateTissueNet(nn.Module):

Explanation: Dual-scale segmentation network using ResNet-50 local features, ResNet-18 context features, handcrafted channels, hierarchical class heads, and a boundary head.

L190 """

Explanation: Starts/continues the docstring documenting the surrounding class or function.

L191 Accuracy-first dual-scale segmentation model.

Explanation: Continues the implementation of `AccurateTissueNet`; this line is part of the surrounding multi-line expression or statement.

L193 local_rgb contains a native-resolution patch. context_rgb contains the

Explanation: Continues the implementation of `AccurateTissueNet`; this line is part of the surrounding multi-line expression or statement.

L194 corresponding context_scale-times-wider field of view resized to the same

Explanation: Continues the implementation of `AccurateTissueNet`; this line is part of the surrounding multi-line expression or statement.

L195 tensor dimensions. Muscle is modeled conditionally inside the combined

Explanation: Continues the implementation of `AccurateTissueNet`; this line is part of the surrounding multi-line expression or statement.

L196 Fibrocartilage-or-Muscle parent class.

Explanation: Continues the implementation of `AccurateTissueNet`; this line is part of the surrounding multi-line expression or statement.

L197 """

Explanation: Starts/continues the docstring documenting the surrounding class or function.

L199 def __init__(

Explanation: Constructs fixed convolution kernels, scaling constants, and/or neural layers for the current class.

L200 self,

Explanation: Continues the surrounding multi-line argument, tuple, list, or unpacking expression.

L201 context_scale: int = 4,

Explanation: Assigns the result of this expression to `context\_scale` for use later in `\_\_init\_\_`.

L202 pretrained: bool = True,

Explanation: Assigns the result of this expression to `pretrained` for use later in `\_\_init\_\_`.

L203 decoder_channels: int = 64,

Explanation: Assigns the result of this expression to `decoder\_channels` for use later in `\_\_init\_\_`.

L204) -> None:

Explanation: Closes or continues the surrounding multi-line Python expression.

L205 super().__init__()

Explanation: Continues the implementation of `\_\_init\_\_`; this line is part of the surrounding multi-line expression or statement.

L206 if context_scale < 1:

Explanation: Conditional branch: evaluates this condition and executes the indented block only when it is true.

L207 raise ValueError("context_scale must be at least 1")

Explanation: Stops execution with an explicit error because an input/state invariant is violated.

L208 self.config = ModelConfig(

Explanation: Stores `config` on the module/dataset instance for reuse by later methods.

L209 context_scale=context_scale,

Explanation: Assigns the result of this expression to `context\_scale` for use later in `\_\_init\_\_`.

L210 pretrained=pretrained,

Explanation: Assigns the result of this expression to `pretrained` for use later in `\_\_init\_\_`.

L211 decoder_channels=decoder_channels,

Explanation: Assigns the result of this expression to `decoder\_channels` for use later in `\_\_init\_\_`.

L212)

Explanation: Closes the multi-line call/list/tuple/array started above.

L213 self.context_scale = context_scale

Explanation: Stores `context\_scale` on the module/dataset instance for reuse by later methods.

L215 local_weights = ResNet50_Weights.DEFAULT if pretrained else None

Explanation: Assigns `local\_weights` as the optional pretrained ResNet-50 weights.

L216 context_weights = ResNet18_Weights.DEFAULT if pretrained else None

Explanation: Assigns `context\_weights` as the optional pretrained ResNet-18 weights.

L217 local_encoder = resnet50(weights=local_weights)

Explanation: Assigns `local\_encoder` as the ResNet-50 backbone used for local/native-scale features.

L218 context_encoder = resnet18(weights=context_weights)

Explanation: Assigns `context\_encoder` as the ResNet-18 backbone used for wider-context features.

L220 self.rgb_conv1 = local_encoder.conv1

Explanation: Stores `rgb\_conv1` on the module/dataset instance for reuse by later methods.

L221 self.rgb_bn1 = local_encoder.bn1

Explanation: Stores `rgb\_bn1` on the module/dataset instance for reuse by later methods.

L222 self.rgb_relu = local_encoder.relu

Explanation: Stores `rgb\_relu` on the module/dataset instance for reuse by later methods.

L223 self.local_maxpool = local_encoder.maxpool

Explanation: Stores `local\_maxpool` on the module/dataset instance for reuse by later methods.

L224 self.local_layer1 = local_encoder.layer1

Explanation: Stores `local\_layer1` on the module/dataset instance for reuse by later methods.

L225 self.local_layer2 = local_encoder.layer2

Explanation: Stores `local\_layer2` on the module/dataset instance for reuse by later methods.

L226 self.local_layer3 = local_encoder.layer3

Explanation: Stores `local\_layer3` on the module/dataset instance for reuse by later methods.

L227 self.local_layer4 = local_encoder.layer4

Explanation: Stores `local\_layer4` on the module/dataset instance for reuse by later methods.

L229 self.context_conv1 = context_encoder.conv1

Explanation: Stores `context\_conv1` on the module/dataset instance for reuse by later methods.

L230 self.context_bn1 = context_encoder.bn1

Explanation: Stores `context\_bn1` on the module/dataset instance for reuse by later methods.

L231 self.context_relu = context_encoder.relu

Explanation: Stores `context\_relu` on the module/dataset instance for reuse by later methods.

L232 self.context_maxpool = context_encoder.maxpool

Explanation: Stores `context\_maxpool` on the module/dataset instance for reuse by later methods.

L233 self.context_layer1 = context_encoder.layer1

Explanation: Stores `context\_layer1` on the module/dataset instance for reuse by later methods.

L234 self.context_layer2 = context_encoder.layer2

Explanation: Stores `context\_layer2` on the module/dataset instance for reuse by later methods.

L235 self.context_layer3 = context_encoder.layer3

Explanation: Stores `context\_layer3` on the module/dataset instance for reuse by later methods.

L236 self.context_layer4 = context_encoder.layer4

Explanation: Stores `context\_layer4` on the module/dataset instance for reuse by later methods.

L238 self.handcrafted = HandcraftedFeatures()

Explanation: Stores `handcrafted` on the module/dataset instance for reuse by later methods.

L239 self.feature_stem = nn.Sequential(

Explanation: Stores `feature\_stem` on the module/dataset instance for reuse by later methods.

L240 nn.BatchNorm2d(9),

Explanation: Executes this numerical/layer operation as part of the surrounding expression.

L241 nn.Conv2d(9, 32, 5, stride=2, padding=2, bias=False),

Explanation: Executes this numerical/layer operation as part of the surrounding expression.

L242 nn.GroupNorm(8, 32),

Explanation: Executes this numerical/layer operation as part of the surrounding expression.

L243 nn.GELU(),

Explanation: Executes this numerical/layer operation as part of the surrounding expression.

L244 nn.Conv2d(32, 64, 3, padding=1, bias=False),

Explanation: Executes this numerical/layer operation as part of the surrounding expression.

L245 nn.GroupNorm(8, 64),

Explanation: Executes this numerical/layer operation as part of the surrounding expression.

L246 nn.GELU(),

Explanation: Executes this numerical/layer operation as part of the surrounding expression.

L247)

Explanation: Closes the multi-line call/list/tuple/array started above.

L248 self.stem_fusion = ConvBlock(128, 64)

Explanation: Stores `stem\_fusion` on the module/dataset instance for reuse by later methods.

L250 self.local_projection = ConvBlock(2048, 512)

Explanation: Stores `local\_projection` on the module/dataset instance for reuse by later methods.

L251 self.context_projection = ConvBlock(512, 256)

Explanation: Stores `context\_projection` on the module/dataset instance for reuse by later methods.

L252 self.context_film = nn.Sequential(

Explanation: Stores `context\_film` on the module/dataset instance for reuse by later methods.

L253 nn.Linear(512, 512),

Explanation: Executes this numerical/layer operation as part of the surrounding expression.

L254 nn.GELU(),

Explanation: Executes this numerical/layer operation as part of the surrounding expression.

L255 nn.Linear(512, 1024),

Explanation: Executes this numerical/layer operation as part of the surrounding expression.

L256)

Explanation: Closes the multi-line call/list/tuple/array started above.

L257 self.bottleneck_fusion = ConvBlock(768, 512)

Explanation: Stores `bottleneck\_fusion` on the module/dataset instance for reuse by later methods.

L259 d = decoder_channels

Explanation: Assigns the result of this expression to `d` for use later in `\_\_init\_\_`.

L260 self.decode4 = DecoderBlock(512, 1024, d * 8)

Explanation: Stores `decode4` on the module/dataset instance for reuse by later methods.

L261 self.decode3 = DecoderBlock(d * 8, 512, d * 4)

Explanation: Stores `decode3` on the module/dataset instance for reuse by later methods.

L262 self.decode2 = DecoderBlock(d * 4, 256, d * 2)

Explanation: Stores `decode2` on the module/dataset instance for reuse by later methods.

L263 self.decode1 = DecoderBlock(d * 2, 64, d)

Explanation: Stores `decode1` on the module/dataset instance for reuse by later methods.

L264 self.final_refine = ConvBlock(d, d)

Explanation: Stores `final\_refine` on the module/dataset instance for reuse by later methods.

L266 self.coarse_classifier = nn.Conv2d(d, 5, 1)

Explanation: Stores `coarse\_classifier` on the module/dataset instance for reuse by later methods.

L267 self.muscle_classifier = nn.Conv2d(d, 1, 1)

Explanation: Stores `muscle\_classifier` on the module/dataset instance for reuse by later methods.

L268 self.boundary_classifier = nn.Conv2d(d, 1, 1)

Explanation: Stores `boundary\_classifier` on the module/dataset instance for reuse by later methods.

L270 self.muscle_presence = nn.Sequential(

Explanation: Stores `muscle\_presence` on the module/dataset instance for reuse by later methods.

L271 nn.Linear(1024, 256),

Explanation: Executes this numerical/layer operation as part of the surrounding expression.

L272 nn.GELU(),

Explanation: Executes this numerical/layer operation as part of the surrounding expression.

L273 nn.Dropout(0.25),

Explanation: Executes this numerical/layer operation as part of the surrounding expression.

L274 nn.Linear(256, 1),

Explanation: Executes this numerical/layer operation as part of the surrounding expression.

L275)

Explanation: Closes the multi-line call/list/tuple/array started above.

L276 self.presence_scale = nn.Parameter(torch.tensor(0.25))

Explanation: Stores `presence\_scale` on the module/dataset instance for reuse by later methods.

L278 self.register_buffer(

Explanation: Continues the implementation of `\_\_init\_\_`; this line is part of the surrounding multi-line expression or statement.

L279 "rgb_mean",

Explanation: Adds this string literal to the surrounding tuple/list/message or multi-line string.

L280 torch.tensor([0.485, 0.456, 0.406]).view(1, 3, 1, 1),

Explanation: Executes this numerical/layer operation as part of the surrounding expression.

L281)

Explanation: Closes the multi-line call/list/tuple/array started above.

L282 self.register_buffer(

Explanation: Continues the implementation of `\_\_init\_\_`; this line is part of the surrounding multi-line expression or statement.

L283 "rgb_std",

Explanation: Adds this string literal to the surrounding tuple/list/message or multi-line string.

L284 torch.tensor([0.229, 0.224, 0.225]).view(1, 3, 1, 1),

Explanation: Executes this numerical/layer operation as part of the surrounding expression.

L285)

Explanation: Closes the multi-line call/list/tuple/array started above.

L287 def model_config(self) -> dict:

Explanation: Serializes only the architecture settings required to reconstruct a checkpoint-compatible model.

L288 config = asdict(self.config)

Explanation: Assigns the result of this expression to `config` for use later in `model\_config`.

L289 # A checkpoint load never needs to download pretrained weights.

Explanation: Developer comment: A checkpoint load never needs to download pretrained weights.

L290 config["pretrained"] = False

Explanation: Continues the implementation of `model\_config`; this line is part of the surrounding multi-line expression or statement.

L291 return config

Explanation: Returns the computed value(s) to the caller.

L293 def _normalize_rgb(self, rgb: torch.Tensor) -> torch.Tensor:

Explanation: Applies ImageNet RGB mean/std normalization expected by pretrained ResNet encoders.

L294 return (rgb - self.rgb_mean) / self.rgb_std

Explanation: Returns the computed value(s) to the caller.

L296 def _context_center_features(

Explanation: Crops the central region of the low-resolution context feature map corresponding to the local field of view and resizes it to the bottleneck grid.

L297 self, context_features: torch.Tensor, output_size: tuple[int, int]

Explanation: Continues the implementation of `\_context\_center\_features`; this line is part of the surrounding multi-line expression or statement.

L298) -> torch.Tensor:

Explanation: Closes or continues the surrounding multi-line Python expression.

L299 height, width = context_features.shape[-2:]

Explanation: Continues the implementation of `\_context\_center\_features`; this line is part of the surrounding multi-line expression or statement.

L300 crop_height = max(1, round(height / self.context_scale))

Explanation: Assigns the result of this expression to `crop\_height` for use later in `\_context\_center\_features`.

L301 crop_width = max(1, round(width / self.context_scale))

Explanation: Assigns the result of this expression to `crop\_width` for use later in `\_context\_center\_features`.

L302 top = max(0, (height - crop_height) // 2)

Explanation: Assigns the result of this expression to `top` for use later in `\_context\_center\_features`.

L303 left = max(0, (width - crop_width) // 2)

Explanation: Assigns the result of this expression to `left` for use later in `\_context\_center\_features`.

L304 center = context_features[

Explanation: Assigns the result of this expression to `center` for use later in `\_context\_center\_features`.

L305 , :, top : top + crop_height, left : left + crop_width

Explanation: Continues the implementation of `\_context\_center\_features`; this line is part of the surrounding multi-line expression or statement.

L306]

Explanation: Closes the multi-line container started above.

L307 return F.interpolate(

Explanation: Returns the computed value(s) to the caller.

L308 center, size=output_size, mode="bilinear", align_corners=False

Explanation: Continues the implementation of `\_context\_center\_features`; this line is part of the surrounding multi-line expression or statement.

L309)

Explanation: Closes the multi-line call/list/tuple/array started above.

L311 def forward(

Explanation: Runs the current module forward and returns its output tensors.

L312 self, local_rgb: torch.Tensor, context_rgb: torch.Tensor

Explanation: Continues the implementation of `forward`; this line is part of the surrounding multi-line expression or statement.

L313) -> dict[str, torch.Tensor]:

Explanation: Closes or continues the surrounding multi-line Python expression.

L314 if local_rgb.shape != context_rgb.shape:

Explanation: Conditional branch: evaluates this condition and executes the indented block only when it is true.

L315 raise ValueError(

Explanation: Stops execution with an explicit error because an input/state invariant is violated.

L316 "Local and resized context tensors must have identical shapes; "

Explanation: Adds this string literal to the surrounding tuple/list/message or multi-line string.

L317 f"received {tuple(local_rgb.shape)} and {tuple(context_rgb.shape)}")

Explanation: Continues the implementation of `forward`; this line is part of the surrounding multi-line expression or statement.

L318)

Explanation: Closes the multi-line call/list/tuple/array started above.

L320 local_normalized = self._normalize_rgb(local_rgb)

Explanation: Assigns the result of this expression to `local\_normalized` for use later in `forward`.

L321 rgb_stem = self.rgb_relu(self.rgb_bn1(self.rgb_conv1(local_normalized)))

Explanation: Assigns the result of this expression to `rgb\_stem` for use later in `forward`.

L322 feature_stem = self.feature_stem(self.handcrafted(local_rgb))

Explanation: Assigns the result of this expression to `feature\_stem` for use later in `forward`.

L323 stem = self.stem_fusion(torch.cat([rgb_stem, feature_stem], dim=1))

Explanation: Computes tensor quantity `stem` using PyTorch operations in the surrounding forward logic.

L325 local1 = self.local_layer1(self.local_maxpool(stem))

Explanation: Assigns the result of this expression to `local1` for use later in `forward`.

L326 local2 = self.local_layer2(local1)

Explanation: Assigns the result of this expression to `local2` for use later in `forward`.

L327 local3 = self.local_layer3(local2)

Explanation: Assigns the result of this expression to `local3` for use later in `forward`.

L328 local4 = self.local_layer4(local3)

Explanation: Assigns the result of this expression to `local4` for use later in `forward`.

L329 local_bottleneck = self.local_projection(local4)

Explanation: Assigns the result of this expression to `local\_bottleneck` for use later in `forward`.

L331 context_normalized = self._normalize_rgb(context_rgb)

Explanation: Assigns the result of this expression to `context\_normalized` for use later in `forward`.

L332 context = self.context_relu(

Explanation: Assigns the result of this expression to `context` for use later in `forward`.

L333 self.context_bn1(self.context_conv1(context_normalized))

Explanation: Continues the implementation of `forward`; this line is part of the surrounding multi-line expression or statement.

L334)

Explanation: Closes the multi-line call/list/tuple/array started above.

L335 context = self.context_layer1(self.context_maxpool(context))

Explanation: Assigns the result of this expression to `context` for use later in `forward`.

L336 context = self.context_layer2(context)

Explanation: Assigns the result of this expression to `context` for use later in `forward`.

L337 context = self.context_layer3(context)

Explanation: Assigns the result of this expression to `context` for use later in `forward`.

L338 context = self.context_layer4(context)

Explanation: Assigns the result of this expression to `context` for use later in `forward`.

L340 context_global = F.adaptive_avg_pool2d(context, 1).flatten(1)

Explanation: Computes tensor quantity `context\_global` using PyTorch operations in the surrounding forward logic.

L341 gamma, beta = self.context_film(context_global).chunk(2, dim=1)

Explanation: Continues the implementation of `forward`; this line is part of the surrounding multi-line expression or statement.

L342 gamma = 0.1 * torch.tanh(gamma).unsqueeze(-1).unsqueeze(-1)

Explanation: Computes tensor quantity `gamma` using PyTorch operations in the surrounding forward logic.

L343 beta = 0.1 * beta.unsqueeze(-1).unsqueeze(-1)

Explanation: Assigns the result of this expression to `beta` for use later in `forward`.

L344 local_bottleneck = local_bottleneck * (1 + gamma) + beta

Explanation: Assigns the result of this expression to `local\_bottleneck` for use later in `forward`.

L346 context_center = self._context_center_features(

Explanation: Assigns the result of this expression to `context\_center` for use later in `forward`.

L347 context, local_bottleneck.shape[-2:])

Explanation: Continues the implementation of `forward`; this line is part of the surrounding multi-line expression or statement.

L348)

Explanation: Closes the multi-line call/list/tuple/array started above.

L349 context_center = self.context_projection(context_center)

Explanation: Assigns the result of this expression to `context\_center` for use later in `forward`.

L350 bottleneck = self.bottleneck_fusion(

Explanation: Assigns the result of this expression to `bottleneck` for use later in `forward`.

L351 torch.cat([local_bottleneck, context_center], dim=1)

Explanation: Executes this numerical/layer operation as part of the surrounding expression.

L352)

Explanation: Closes the multi-line call/list/tuple/array started above.

L354 decoded = self.decode4(bottleneck, local3)

Explanation: Assigns the result of this expression to `decoded` for use later in `forward`.

L355 decoded = self.decode3(decoded, local2)

Explanation: Assigns the result of this expression to `decoded` for use later in `forward`.

L356 decoded = self.decode2(decoded, local1)

Explanation: Assigns the result of this expression to `decoded` for use later in `forward`.

L357 decoded = self.decode1(decoded, stem)

Explanation: Assigns the result of this expression to `decoded` for use later in `forward`.

L358 decoded = F.interpolate(

Explanation: Computes tensor quantity `decoded` using PyTorch operations in the surrounding forward logic.

L359 decoded,

Explanation: Continues the surrounding multi-line argument, tuple, list, or unpacking expression.

L360 size=local_rgb.shape[-2:],
Explanation: Assigns the result of this expression to `size` for use later in `forward`.

L361 mode="bilinear",
Explanation: Assigns the result of this expression to `mode` for use later in `forward`.

L362 align_corners=False,
Explanation: Assigns the result of this expression to `align\_corners` for use later in `forward`.

L363)
Explanation: Closes the multi-line call/list/tuple/array started above.

L364 decoded = self.final_refine(decoded)
Explanation: Assigns the result of this expression to `decoded` for use later in `forward`.

L366 pooled_local = F.adaptive_avg_pool2d(local_bottleneck, 1).flatten(1)
Explanation: Computes tensor quantity `pooled\_local` using PyTorch operations in the surrounding forward logic.

L367 presence_logit = self.muscle_presence(
Explanation: Assigns the result of this expression to `presence\_logit` for use later in `forward`.

L368 torch.cat([pooled_local, context_global], dim=1)
Explanation: Executes this numerical/layer operation as part of the surrounding expression.

L369)
Explanation: Closes the multi-line call/list/tuple/array started above.

L370 muscle_logit = self.muscle_classifier(decoded)
Explanation: Assigns the result of this expression to `muscle\_logit` for use later in `forward`.

L371 muscle_logit = muscle_logit + (
Explanation: Assigns the result of this expression to `muscle\_logit` for use later in `forward`.

L372 self.presence_scale * presence_logit[:, :, None, None]
Explanation: Continues the implementation of `forward`; this line is part of the surrounding multi-line expression or statement.

L373)
Explanation: Closes the multi-line call/list/tuple/array started above.

L375 return {
Explanation: Returns the computed value(s) to the caller.

L376 "coarse_logits": self.coarse_classifier(decoded),
Explanation: Adds this string literal to the surrounding tuple/list/message or multi-line string.

L377 "muscle_logits": muscle_logit,
Explanation: Adds this string literal to the surrounding tuple/list/message or multi-line string.

L378 "boundary_logits": self.boundary_classifier(decoded),
Explanation: Adds this string literal to the surrounding tuple/list/message or multi-line string.

L379 "muscle_presence_logits": presence_logit,
Explanation: Adds this string literal to the surrounding tuple/list/message or multi-line string.

L380 }
Explanation: Closes the multi-line container started above.

L382 @staticmethod
Explanation: Marks the following method as independent of a class instance; it receives no `self` argument.

L383 def probabilities_from_outputs(
Explanation: Converts five coarse logits plus a conditional muscle logit into six mutually exclusive final class probabilities.

L384 outputs: dict[str, torch.Tensor],
Explanation: Continues the surrounding multi-line call/container with this argument or value.

L385) -> torch.Tensor:
Explanation: Closes or continues the surrounding multi-line Python expression.

L386 coarse = outputs["coarse_logits"].softmax(dim=1)
Explanation: Applies softmax over the five coarse channels. Under CUDA autograd the logits may be float16, so the resulting coarse probabilities can also carry half-precision rounding.

L387 muscle_given_parent = outputs["muscle_logits"].sigmoid()
Explanation: Applies sigmoid to the conditional Muscle logit, producing $P(\text{Muscle} \mid \text{Fibrocartilage-or-Muscle parent})$.

L388 parent = coarse[:, 1:2]
Explanation: Extracts the parent probability $P(\text{Fibrocartilage or Muscle})$ from coarse channel 1.

L389 return torch.cat(

Explanation: Returns the computed value(s) to the caller.

L390 [

Explanation: Continues the implementation of `probabilities\_from\_outputs`; this line is part of the surrounding multi-line expression or statement.

L391 coarse[:, 0:1],

Explanation: Continues the surrounding multi-line call/container with this argument or value.

L392 parent * (1 - muscle_given_parent),

Explanation: Defines final Fibrocartilage probability as $\text{parent} \times (1 - \text{conditional Muscle probability})$.

L393 coarse[:, 2:3],

Explanation: Continues the surrounding multi-line call/container with this argument or value.

L394 parent * muscle_given_parent,

Explanation: Defines final Muscle probability as $\text{parent} \times \text{conditional Muscle probability}$. In exact arithmetic lines 392+394 recover the parent exactly; float16 rounding can break that exact equality.

L395 coarse[:, 3:4],

Explanation: Continues the surrounding multi-line call/container with this argument or value.

L396 coarse[:, 4:5],

Explanation: Continues the surrounding multi-line call/container with this argument or value.

L397],

Explanation: Closes or continues the surrounding multi-line Python expression.

L398 dim=1,

Explanation: Assigns the result of this expression to `dim` for use later in `probabilities\_from\_outputs`.

L399)

Explanation: Closes the multi-line call/list/tuple/array started above.

L401 def probabilities(

Explanation: Convenience wrapper that performs a forward pass and probability conversion.

L402 self, local_rgb: torch.Tensor, context_rgb: torch.Tensor

Explanation: Continues the implementation of `probabilities`; this line is part of the surrounding multi-line expression or statement.

L403) -> torch.Tensor:

Explanation: Closes or continues the surrounding multi-line Python expression.

L404 return self.probabilities_from_outputs(self(local_rgb, context_rgb))

Explanation: Returns the computed value(s) to the caller.

L407 if __name__ == "__main__":

Explanation: Python entry-point guard: the indented call runs only when this file is executed as a script.

L408 model = AccurateTissueNet(pretrained=False)

Explanation: Assigns the result of this expression to `model` for use later in `model.py`.

L409 local = torch.rand(1, 3, 256, 256)

Explanation: Creates random tensor data for `local` used by the smoke test.

L410 context = torch.rand(1, 3, 256, 256)

Explanation: Creates random tensor data for `context` used by the smoke test.

L411 with torch.inference_mode():

Explanation: Runs the following block without gradient tracking.

L412 probabilities = model.probabilities(local, context)

Explanation: Assigns `probabilities` as the six-class per-pixel probability tensor.

L413 parameters = sum(parameter.numel() for parameter in model.parameters())

Explanation: Assigns the result of this expression to `parameters` for use later in `model.py`.

L414 print(

Explanation: Prints runtime progress/diagnostic information to standard output.

L415 f"input={tuple(local.shape)} probabilities={tuple(probabilities.shape)} "

Explanation: Continues the implementation of `model.py`; this line is part of the surrounding multi-line expression or statement.

L416 f"parameters={parameters:,} sum={probabilities.sum(1).mean().item():.6f}"

Explanation: Continues the implementation of `model.py`; this line is part of the surrounding multi-line expression or statement.

L417)

Explanation: Closes the multi-line call/list/tuple/array started above.

train.py

Major definitions: coarse_targets, multiclass_dice_loss, binary_dice_loss, masked_muscle_loss, boundary_loss, complete_loss, Metrics, make_loader, run_epoch, parse_args, main, __init__, update, results

L001 from __future__ import annotations

Explanation: Postpones evaluation of type annotations, reducing runtime coupling and allowing forward references.

L003 import argparse

Explanation: Imports dependencies used by this module: command-line argument parsing

L004 from collections import defaultdict

Explanation: Imports dependencies used by this module: default-valued dictionaries

L005 import os

Explanation: Imports dependencies used by this module: environment-variable access

L006 from pathlib import Path

Explanation: Imports dependencies used by this module: filesystem paths

L007 import random

Explanation: Imports dependencies used by this module: Python pseudo-random sampling/augmentation

L009 import numpy as np

Explanation: Imports dependencies used by this module: array and numerical operations

L010 import torch

Explanation: Imports dependencies used by this module: tensor computation and automatic differentiation

L011 import torch.nn.functional as F

Explanation: Imports dependencies used by this module: tensor computation and automatic differentiation; PyTorch functional neural-network operations; PyTorch neural-network layers

L012 from torch.utils.data import DataLoader

Explanation: Imports dependencies used by this module: tensor computation and automatic differentiation; batch loading

L014 from data import SegmentationDataset, discover_samples

Explanation: Imports dependencies used by this module.

L015 from model import AccurateTissueNet, CLASS_NAMES

Explanation: Imports dependencies used by this module.

L018 COARSE_MAPPING = (0, 1, 2, 1, 3, 4)

Explanation: Assigns `COARSE\_MAPPING` as the mapping from six final IDs to five coarse IDs.

L021 def coarse_targets(mask: torch.Tensor) -> torch.Tensor:

Explanation: Maps the six final labels to five coarse labels, merging Fibrocartilage and Muscle into one parent class.

L022 mapping = torch.tensor(COARSE_MAPPING, device=mask.device)

Explanation: Computes tensor quantity `mapping` using PyTorch operations in the surrounding coarse_targets logic.

L023 return mapping[mask]

Explanation: Returns the computed value(s) to the caller.

L026 def multiclass_dice_loss(

Explanation: Computes soft multiclass Dice loss over all six final classes, optionally weighted.

L027 probabilities: torch.Tensor,

Explanation: Continues the surrounding multi-line call/container with this argument or value.

L028 target: torch.Tensor,

Explanation: Continues the surrounding multi-line call/container with this argument or value.

L029 class_weights: torch.Tensor | None = None,

Explanation: Assigns the result of this expression to `class\_weights` for use later in `multiclass\_dice\_loss`.

L030 eps: float = 1e-6,

Explanation: Assigns the result of this expression to `eps` for use later in `multiclass\_dice\_loss`.

L031) -> torch.Tensor:

Explanation: Closes or continues the surrounding multi-line Python expression.

L032 one_hot = F.one_hot(target, num_classes=len(CLASS_NAMES)).permute(0, 3, 1, 2)

Explanation: Computes tensor quantity `one\_hot` using PyTorch operations in the surrounding multiclass_dice_loss logic.

L033 one_hot = one_hot.to(probabilities.dtype)

Explanation: Assigns the result of this expression to `one\_hot` for use later in `multiclass\_dice\_loss`.

L034 dimensions = (0, 2, 3)

Explanation: Assigns the result of this expression to `dimensions` for use later in `multiclass\_dice\_loss`.

L035 intersection = (probabilities * one_hot).sum(dim=dimensions)

Explanation: Assigns the result of this expression to `intersection` for use later in `multiclass\_dice\_loss`.

L036 denominator = probabilities.sum(dim=dimensions) + one_hot.sum(

Explanation: Assigns the result of this expression to `denominator` for use later in `multiclass\_dice\_loss`.

L037 dim=dimensions

Explanation: Assigns the result of this expression to `dim` for use later in `multiclass\_dice\_loss`.

L038)

Explanation: Closes the multi-line call/list/tuple/array started above.

L039 loss_per_class = 1 - (2 * intersection + eps) / (denominator + eps)

Explanation: Assigns the result of this expression to `loss\_per\_class` for use later in `multiclass\_dice\_loss`.

L040 if class_weights is None:

Explanation: Conditional branch: evaluates this condition and executes the indented block only when it is true.

L041 return loss_per_class.mean()

Explanation: Returns the computed value(s) to the caller.

L042 weights = class_weights / class_weights.sum()

Explanation: Assigns the result of this expression to `weights` for use later in `multiclass\_dice\_loss`.

L043 return (loss_per_class * weights).sum()

Explanation: Returns the computed value(s) to the caller.

L046 def binary_dice_loss(

Explanation: Computes a soft Dice loss for a single binary target.

L047 probabilities: torch.Tensor, target: torch.Tensor, eps: float = 1e-6

Explanation: Assigns `probabilities` as the six-class per-pixel probability tensor.

L048) -> torch.Tensor:

Explanation: Closes or continues the surrounding multi-line Python expression.

L049 intersection = (probabilities * target).sum()

Explanation: Assigns the result of this expression to `intersection` for use later in `binary\_dice\_loss`.

L050 denominator = probabilities.sum() + target.sum()

Explanation: Assigns the result of this expression to `denominator` for use later in `binary\_dice\_loss`.

L051 return 1 - (2 * intersection + eps) / (denominator + eps)

Explanation: Returns the computed value(s) to the caller.

L054 def masked_muscle_loss(

Explanation: Trains the child classifier only on pixels belonging to the Fibrocartilage/Muscle parent region.

L055 logits: torch.Tensor, mask: torch.Tensor

Explanation: Continues the implementation of `masked\_muscle\_loss`; this line is part of the surrounding multi-line expression or statement.

L056) -> torch.Tensor:

Explanation: Closes or continues the surrounding multi-line Python expression.

L057 valid = (mask == 1) | (mask == 3)

Explanation: Assigns the result of this expression to `valid` for use later in `masked\_muscle\_loss`.

L058 if not torch.any(valid):

Explanation: Conditional branch: evaluates this condition and executes the indented block only when it is true.

L059 return logits.sum() * 0

Explanation: Returns the computed value(s) to the caller.

L060 target = (mask == 3).float()

Explanation: Assigns the result of this expression to `target` for use later in `masked\_muscle\_loss`.

L061 selected_logits = logits[:, 0][valid].float()

Explanation: Assigns the result of this expression to `selected\_logits` for use later in `masked\_muscle\_loss`.

L062 selected_target = target[valid]

Explanation: Assigns the result of this expression to `selected\_target` for use later in `masked\_muscle\_loss`.

L063 positives = selected_target.sum()

Explanation: Assigns the result of this expression to `positives` for use later in `masked\_muscle\_loss`.

L064 negatives = selected_target.numel() - positives

Explanation: Assigns the result of this expression to `negatives` for use later in `masked\_muscle\_loss`.

L065 positive_weight = (negatives / positives.clamp_min(1)).clamp(0.25, 20)

Explanation: Assigns the result of this expression to `positive\_weight` for use later in `masked\_muscle\_loss`.

L066 return F.binary_cross_entropy_with_logits(

Explanation: Returns the computed value(s) to the caller.

L067 selected_logits,

Explanation: Continues the surrounding multi-line argument, tuple, list, or unpacking expression.

L068 selected_target,

Explanation: Continues the surrounding multi-line argument, tuple, list, or unpacking expression.

L069 pos_weight=positive_weight,

Explanation: Assigns the result of this expression to `pos\_weight` for use later in `masked\_muscle\_loss`.

L070)

Explanation: Closes the multi-line call/list/tuple/array started above.

L073 def boundary_loss(

Explanation: Combines class-balanced binary cross-entropy and Dice for the Bone/Fibrocartilage boundary head.

L074 logits: torch.Tensor, target: torch.Tensor

Explanation: Continues the implementation of `boundary\_loss`; this line is part of the surrounding multi-line expression or statement.

L075) -> torch.Tensor:

Explanation: Closes or continues the surrounding multi-line Python expression.

L076 target = target[:, None].float()

Explanation: Assigns the result of this expression to `target` for use later in `boundary\_loss`.

L077 positives = target.sum()

Explanation: Assigns the result of this expression to `positives` for use later in `boundary\_loss`.

L078 negatives = target.numel() - positives

Explanation: Assigns the result of this expression to `negatives` for use later in `boundary\_loss`.

L079 positive_weight = (negatives / positives.clamp_min(1)).clamp(1, 50)

Explanation: Assigns the result of this expression to `positive\_weight` for use later in `boundary\_loss`.

L080 bce = F.binary_cross_entropy_with_logits(

Explanation: Computes tensor quantity `bce` using PyTorch operations in the surrounding `boundary\_loss` logic.

L081 logits.float(), target, pos_weight=positive_weight

Explanation: Continues the implementation of `boundary\_loss`; this line is part of the surrounding multi-line expression or statement.

L082)

Explanation: Closes the multi-line call/list/tuple/array started above.

L083 dice = binary_dice_loss(logits.float().sigmoid(), target)

Explanation: Assigns `dice` as the six-class soft Dice loss.

L084 return 0.6 * bce + 0.4 * dice

Explanation: Returns the computed value(s) to the caller.

L087 def complete_loss(

Explanation: Builds the five-part training objective and returns total loss, scalar components, and six-class probabilities.

L088 model: AccurateTissueNet,

Explanation: Continues the surrounding multi-line call/container with this argument or value.

L089 outputs: dict[str, torch.Tensor],

Explanation: Continues the surrounding multi-line call/container with this argument or value.

L090 mask: torch.Tensor,

Explanation: Continues the surrounding multi-line call/container with this argument or value.

L091 boundary: torch.Tensor,

Explanation: Continues the surrounding multi-line call/container with this argument or value.

L092 muscle_presence: torch.Tensor,

Explanation: Continues the surrounding multi-line call/container with this argument or value.

L093 class_weights: torch.Tensor | None,

Explanation: Continues the surrounding multi-line call/container with this argument or value.

L094) -> tuple[torch.Tensor, dict[str, float], torch.Tensor]:

Explanation: Closes or continues the surrounding multi-line Python expression.

L095 coarse_weight = None

Explanation: Assigns the result of this expression to `coarse\_weight` for use later in `complete\_loss`.

L096 if class_weights is not None:

Explanation: Conditional branch: evaluates this condition and executes the indented block only when it is true.

L097 coarse_weight = torch.stack(

Explanation: Computes tensor quantity `coarse\_weight` using PyTorch operations in the surrounding complete_loss logic.

L098 [

Explanation: Continues the implementation of `complete\_loss`; this line is part of the surrounding multi-line expression or statement.

L099 class_weights[0],

Explanation: Continues the surrounding multi-line call/container with this argument or value.

L100 0.5 * (class_weights[1] + class_weights[3]),

Explanation: Continues the surrounding multi-line call/container with this argument or value.

L101 class_weights[2],

Explanation: Continues the surrounding multi-line call/container with this argument or value.

L102 class_weights[4],

Explanation: Continues the surrounding multi-line call/container with this argument or value.

L103 class_weights[5],

Explanation: Continues the surrounding multi-line call/container with this argument or value.

L104]

Explanation: Closes the multi-line container started above.

L105)

Explanation: Closes the multi-line call/list/tuple/array started above.

L106 coarse = F.cross_entropy(

Explanation: Assigns `coarse` as the coarse five-class loss/probability tensor.

L107 outputs["coarse_logits"],

Explanation: Continues the surrounding multi-line call/container with this argument or value.

L108 coarse_targets(mask),

Explanation: Continues the surrounding multi-line call/container with this argument or value.

L109 weight=coarse_weight,

Explanation: Assigns the result of this expression to `weight` for use later in `complete\_loss`.

L110)

Explanation: Closes the multi-line call/list/tuple/array started above.

L111 muscle = masked_muscle_loss(outputs["muscle_logits"], mask)

Explanation: Assigns `muscle` as the conditional child loss for Muscle versus Fibrocartilage.

L112 # Accumulate Dice terms in float32. A 512x512 mixed-precision batch can

Explanation: Comment states the intent to accumulate Dice in float32 to avoid FP16 reduction overflow.

L113 # exceed the largest finite FP16 reduction value.

Explanation: Developer comment: exceed the largest finite FP16 reduction value.

L114 probabilities = model.probabilities_from_outputs(outputs).float()

Explanation: Converts the already-composed probabilities to float32. Important: this happens after softmax/sigmoid/multiply arithmetic, so any FP16 rounding introduced while constructing the six probabilities is already present.

L115 dice = multiclass_dice_loss(probabilities, mask, class_weights)

Explanation: Assigns `dice` as the six-class soft Dice loss.

L116 boundary_component = boundary_loss(outputs["boundary_logits"], boundary)

Explanation: Assigns `boundary\_component` as the Bone/Fibrocartilage boundary loss.

L117 presence = F.binary_cross_entropy_with_logits(

Explanation: Assigns `presence` as the regional muscle-presence loss.

L118 outputs["muscle_presence_logits"][:, 0],

Explanation: Continues the surrounding multi-line call/container with this argument or value.

L119 muscle_presence.to(outputs["muscle_presence_logits"].dtype),

Explanation: Continues the surrounding multi-line call/container with this argument or value.

L120)

Explanation: Closes the multi-line call/list/tuple/array started above.

L121 total = (

Explanation: Assigns `total` as the weighted sum of all loss components.

L122 0.40 * coarse

Explanation: Continues the implementation of `complete\_loss`; this line is part of the surrounding multi-line expression or statement.

L123 + 0.20 * muscle

Explanation: Continues the implementation of `complete\_loss`; this line is part of the surrounding multi-line expression or statement.

L124 + 0.25 * dice

Explanation: Continues the implementation of `complete\_loss`; this line is part of the surrounding multi-line expression or statement.

L125 + 0.10 * boundary_component

Explanation: Continues the implementation of `complete\_loss`; this line is part of the surrounding multi-line expression or statement.

L126 + 0.05 * presence

Explanation: Continues the implementation of `complete\_loss`; this line is part of the surrounding multi-line expression or statement.

L127)

Explanation: Closes the multi-line call/list/tuple/array started above.

L128 components = {

Explanation: Assigns the result of this expression to `components` for use later in `complete\_loss`.

L129 "coarse": coarse.detach().item(),

Explanation: Adds this string literal to the surrounding tuple/list/message or multi-line string.

L130 "muscle": muscle.detach().item(),

Explanation: Adds this string literal to the surrounding tuple/list/message or multi-line string.

L131 "dice": dice.detach().item(),

Explanation: Adds this string literal to the surrounding tuple/list/message or multi-line string.

L132 "boundary": boundary_component.detach().item(),

Explanation: Adds this string literal to the surrounding tuple/list/message or multi-line string.

L133 "presence": presence.detach().item(),

Explanation: Adds this string literal to the surrounding tuple/list/message or multi-line string.

L134 }

Explanation: Closes the multi-line container started above.

L135 return total, components, probabilities

Explanation: Returns the computed value(s) to the caller.

L138 class Metrics:

Explanation: Accumulates a six-class confusion matrix and boundary true/false positives for epoch metrics.

L139 def __init__(self, classes: int = 6) -> None:

Explanation: Initializes metric state or class-specific module state.

L140 self.classes = classes

Explanation: Stores `classes` on the module/dataset instance for reuse by later methods.

L141 self.confusion = torch.zeros(classes, classes, dtype=torch.float64)

Explanation: Stores `confusion` on the module/dataset instance for reuse by later methods.

L142 self.boundary_tp = 0

Explanation: Stores `boundary\_tp` on the module/dataset instance for reuse by later methods.

L143 self.boundary_fp = 0

Explanation: Stores `boundary\_fp` on the module/dataset instance for reuse by later methods.

L144 self.boundary_fn = 0

Explanation: Stores `boundary\_fn` on the module/dataset instance for reuse by later methods.

L146 def update(

Explanation: Adds one batch of segmentation and boundary predictions to the metric accumulators.

L147 self,

Explanation: Continues the surrounding multi-line argument, tuple, list, or unpacking expression.

L148 probabilities: torch.Tensor,

Explanation: Continues the surrounding multi-line call/container with this argument or value.

L149 target: torch.Tensor,

Explanation: Continues the surrounding multi-line call/container with this argument or value.

L150 boundary_probability: torch.Tensor,
Explanation: Continues the surrounding multi-line call/container with this argument or value.

L151 boundary_target: torch.Tensor,
Explanation: Continues the surrounding multi-line call/container with this argument or value.

L152)-> None:
Explanation: Closes or continues the surrounding multi-line Python expression.

L153 prediction = probabilities.argmax(1)
Explanation: Assigns the result of this expression to `prediction` for use later in `update`.

L154 flat = target.reshape(-1) * self.classes + prediction.reshape(-1)
Explanation: Assigns the result of this expression to `flat` for use later in `update`.

L155 matrix = torch.bincount(
Explanation: Computes tensor quantity `matrix` using PyTorch operations in the surrounding update logic.

L156 flat, minlength=self.classes * self.classes
Explanation: Continues the implementation of `update`; this line is part of the surrounding multi-line expression or statement.

L157).reshape(self.classes, self.classes)
Explanation: Closes or continues the surrounding multi-line Python expression.

L158 self.confusion += matrix.detach().cpu()
Explanation: Continues the implementation of `update`; this line is part of the surrounding multi-line expression or statement.

L160 predicted_boundary = boundary_probability[:, 0] >= 0.5
Explanation: Assigns the result of this expression to `predicted\_boundary` for use later in `update`.

L161 true_boundary = boundary_target >= 0.5
Explanation: Assigns the result of this expression to `true\_boundary` for use later in `update`.

L162 self.boundary_tp += int((predicted_boundary & true_boundary).sum())
Explanation: Continues the implementation of `update`; this line is part of the surrounding multi-line expression or statement.

L163 self.boundary_fp += int((predicted_boundary & ~true_boundary).sum())
Explanation: Continues the implementation of `update`; this line is part of the surrounding multi-line expression or statement.

L164 self.boundary_fn += int((~predicted_boundary & true_boundary).sum())
Explanation: Continues the implementation of `update`; this line is part of the surrounding multi-line expression or statement.

L166 def results(self) -> tuple[float, list[float], float]:
Explanation: Computes class IoU, mean IoU, and boundary F1 from accumulated counts.

L167 intersection = self.confusion.diag()
Explanation: Assigns the result of this expression to `intersection` for use later in `results`.

L168 union = (
Explanation: Assigns the result of this expression to `union` for use later in `results`.

L169 self.confusion.sum(0)
Explanation: Continues the implementation of `results`; this line is part of the surrounding multi-line expression or statement.

L170 + self.confusion.sum(1)
Explanation: Continues the implementation of `results`; this line is part of the surrounding multi-line expression or statement.

L171 - intersection
Explanation: Continues the implementation of `results`; this line is part of the surrounding multi-line expression or statement.

L172)
Explanation: Closes the multi-line call/list/tuple/array started above.

L173 valid = union > 0
Explanation: Assigns the result of this expression to `valid` for use later in `results`.

L174 per_class = torch.full((self.classes,), float("nan"))
Explanation: Computes tensor quantity `per\_class` using PyTorch operations in the surrounding results logic.

L175 per_class[valid] = intersection[valid] / union[valid]
Explanation: Continues the implementation of `results`; this line is part of the surrounding multi-line expression or statement.

L176 mean_iou = float(per_class[valid].mean()) if torch.any(valid) else 0.0
Explanation: Computes tensor quantity `mean\_iou` using PyTorch operations in the surrounding results logic.

L177 boundary_f1 = (
Explanation: Assigns the result of this expression to `boundary\_f1` for use later in `results`.

L178 2 * self.boundary_tp

Explanation: Continues the implementation of `results`; this line is part of the surrounding multi-line expression or statement.

L179 / max(

Explanation: Continues the implementation of `results`; this line is part of the surrounding multi-line expression or statement.

L180 1,

Explanation: Continues the surrounding multi-line call/container with this argument or value.

L181 2 * self.boundary_tp

Explanation: Continues the implementation of `results`; this line is part of the surrounding multi-line expression or statement.

L182 + self.boundary_fp

Explanation: Continues the implementation of `results`; this line is part of the surrounding multi-line expression or statement.

L183 + self.boundary_fn,

Explanation: Continues the surrounding multi-line call/container with this argument or value.

L184)

Explanation: Closes the multi-line call/list/tuple/array started above.

L185)

Explanation: Closes the multi-line call/list/tuple/array started above.

L186 return mean_iou, per_class.tolist(), boundary_f1

Explanation: Returns the computed value(s) to the caller.

L189 def make_loader(

Explanation: Creates a DataLoader with shuffling/pinned-memory/persistent-worker settings appropriate to train or validation.

L190 dataset: SegmentationDataset,

Explanation: Continues the surrounding multi-line call/container with this argument or value.

L191 batch_size: int,

Explanation: Continues the surrounding multi-line call/container with this argument or value.

L192 workers: int,

Explanation: Continues the surrounding multi-line call/container with this argument or value.

L193 training: bool,

Explanation: Continues the surrounding multi-line call/container with this argument or value.

L194 use_cuda: bool,

Explanation: Continues the surrounding multi-line call/container with this argument or value.

L195) -> DataLoader:

Explanation: Closes or continues the surrounding multi-line Python expression.

L196 options = {

Explanation: Assigns the result of this expression to `options` for use later in `make\_loader`.

L197 "dataset": dataset,

Explanation: Adds this string literal to the surrounding tuple/list/message or multi-line string.

L198 "batch_size": batch_size,

Explanation: Adds this string literal to the surrounding tuple/list/message or multi-line string.

L199 "shuffle": training,

Explanation: Adds this string literal to the surrounding tuple/list/message or multi-line string.

L200 "num_workers": workers,

Explanation: Adds this string literal to the surrounding tuple/list/message or multi-line string.

L201 "pin_memory": use_cuda,

Explanation: Adds this string literal to the surrounding tuple/list/message or multi-line string.

L202 "persistent_workers": workers > 0,

Explanation: Adds this string literal to the surrounding tuple/list/message or multi-line string.

L203 "drop_last": training and len(dataset) >= batch_size,

Explanation: Adds this string literal to the surrounding tuple/list/message or multi-line string.

L204 }

Explanation: Closes the multi-line container started above.

L205 if workers > 0:

Explanation: Conditional branch: evaluates this condition and executes the indented block only when it is true.

L206 options["prefetch_factor"] = 2

Explanation: Continues the implementation of `make\_loader`; this line is part of the surrounding multi-line expression or statement.

L207 return DataLoader(**options)

Explanation: Returns the computed value(s) to the caller.

L210 def run_epoch(

Explanation: Runs one full training or validation epoch, including AMP, gradient accumulation, clipping, optimizer steps, loss tracking, and metrics.

L211 model: AccurateTissueNet,

Explanation: Continues the surrounding multi-line call/container with this argument or value.

L212 loader: DataLoader,

Explanation: Continues the surrounding multi-line call/container with this argument or value.

L213 device: torch.device,

Explanation: Continues the surrounding multi-line call/container with this argument or value.

L214 class_weights: torch.Tensor | None,

Explanation: Continues the surrounding multi-line call/container with this argument or value.

L215 amp: bool,

Explanation: Continues the surrounding multi-line call/container with this argument or value.

L216 optimizer: torch.optim.Optimizer | None = None,

Explanation: Assigns the result of this expression to `optimizer` for use later in `run\_epoch`.

L217 scaler: torch.cuda.amp.GradScaler | None = None,

Explanation: Assigns the result of this expression to `scaler` for use later in `run\_epoch`.

L218 accumulation_steps: int = 1,

Explanation: Assigns the result of this expression to `accumulation\_steps` for use later in `run\_epoch`.

L219 gradient_clip: float = 1.0,

Explanation: Assigns the result of this expression to `gradient\_clip` for use later in `run\_epoch`.

L220) -> tuple[float, float, list[float], float, dict[str, float]]:

Explanation: Closes or continues the surrounding multi-line Python expression.

L221 training = optimizer is not None

Explanation: Assigns the result of this expression to `training` for use later in `run\_epoch`.

L222 model.train(training)

Explanation: Continues the implementation of `run\_epoch`; this line is part of the surrounding multi-line expression or statement.

L223 metrics = Metrics()

Explanation: Assigns the result of this expression to `metrics` for use later in `run\_epoch`.

L224 total_loss = 0.0

Explanation: Assigns the result of this expression to `total\_loss` for use later in `run\_epoch`.

L225 components: dict[str, float] = defaultdict(float)

Explanation: Assigns the result of this expression to `components` for use later in `run\_epoch`.

L226 if training:

Explanation: Conditional branch: evaluates this condition and executes the indented block only when it is true.

L227 optimizer.zero_grad(set_to_none=True)

Explanation: Continues the implementation of `run\_epoch`; this line is part of the surrounding multi-line expression or statement.

L229 for batch_index, batch in enumerate(loader):

Explanation: Loop: iterates over the specified sequence/range and executes the following block for each item.

L230 local = batch["local"].to(device, non_blocking=True)

Explanation: Moves `local` data to the selected compute device, allowing non-blocking transfer when possible.

L231 context = batch["context"].to(device, non_blocking=True)

Explanation: Moves `context` data to the selected compute device, allowing non-blocking transfer when possible.

L232 if device.type == "cuda":

Explanation: Conditional branch: evaluates this condition and executes the indented block only when it is true.

L233 local = local.contiguous(memory_format=torch.channels_last)

Explanation: Computes tensor quantity `local` using PyTorch operations in the surrounding run_epoch logic.

L234 context = context.contiguous(memory_format=torch.channels_last)

Explanation: Computes tensor quantity `context` using PyTorch operations in the surrounding run_epoch logic.

L235 mask = batch["mask"].to(device, non_blocking=True)

Explanation: Moves `mask` data to the selected compute device, allowing non-blocking transfer when possible.

L236 boundary = batch["boundary"].to(device, non_blocking=True)

Explanation: Moves `boundary` data to the selected compute device, allowing non-blocking transfer when possible.

L237 presence = batch["muscle_presence"].to(device, non_blocking=True)

Explanation: Assigns `presence` as the regional muscle-presence loss.

L239 with torch.set_grad_enabled(training):

Explanation: Enables gradients for training and disables them for validation according to the `training` flag.

L240 with torch.autocast(

Explanation: Opens an automatic mixed-precision region; on CUDA, selected operations may run in float16 for speed/memory savings.

L241 device_type=device.type,

Explanation: Assigns the result of this expression to `device\_type` for use later in `run\_epoch`.

L242 dtype=torch.float16,

Explanation: Computes tensor quantity `dtype` using PyTorch operations in the surrounding run_epoch logic.

L243 enabled=amp,

Explanation: Assigns the result of this expression to `enabled` for use later in `run\_epoch`.

L244):

Explanation: Closes or continues the surrounding multi-line Python expression.

L245 outputs = model(local, context)

Explanation: Assigns the result of this expression to `outputs` for use later in `run\_epoch`.

L246 loss, batch_components, probabilities = complete_loss(

Explanation: Continues the implementation of `run\_epoch`; this line is part of the surrounding multi-line expression or statement.

L247 model,

Explanation: Continues the surrounding multi-line argument, tuple, list, or unpacking expression.

L248 outputs,

Explanation: Continues the surrounding multi-line argument, tuple, list, or unpacking expression.

L249 mask,

Explanation: Continues the surrounding multi-line argument, tuple, list, or unpacking expression.

L250 boundary,

Explanation: Continues the surrounding multi-line argument, tuple, list, or unpacking expression.

L251 presence,

Explanation: Continues the surrounding multi-line argument, tuple, list, or unpacking expression.

L252 class_weights,

Explanation: Continues the surrounding multi-line argument, tuple, list, or unpacking expression.

L253)

Explanation: Closes the multi-line call/list/tuple/array started above.

L254 if training:

Explanation: Conditional branch: evaluates this condition and executes the indented block only when it is true.

L255 assert scaler is not None

Explanation: Runtime sanity check; raises `AssertionError` when the asserted invariant is false.

L256 scaler.scale(loss / accumulation_steps).backward()

Explanation: Continues the implementation of `run\_epoch`; this line is part of the surrounding multi-line expression or statement.

L257 should_step = (

Explanation: Assigns the result of this expression to `should\_step` for use later in `run\_epoch`.

L258 (batch_index + 1) % accumulation_steps == 0

Explanation: Continues the implementation of `run\_epoch`; this line is part of the surrounding multi-line expression or statement.

L259 or batch_index + 1 == len(loader)

Explanation: Continues the implementation of `run\_epoch`; this line is part of the surrounding multi-line expression or statement.

L260)

Explanation: Closes the multi-line call/list/tuple/array started above.

L261 if should_step:

Explanation: Conditional branch: evaluates this condition and executes the indented block only when it is true.

L262 scaler.unscale_(optimizer)

Explanation: Continues the implementation of `run\_epoch`; this line is part of the surrounding multi-line expression or statement.

L263 torch.nn.utils.clip_grad_norm_(

Explanation: Executes this numerical/layer operation as part of the surrounding expression.

L264 model.parameters(), gradient_clip

Explanation: Continues the implementation of `run\_epoch`; this line is part of the surrounding multi-line expression or statement.

L265)

Explanation: Closes the multi-line call/list/tuple/array started above.

L266 scaler.step(optimizer)

Explanation: Continues the implementation of `run\_epoch`; this line is part of the surrounding multi-line expression or statement.

L267 scaler.update()

Explanation: Continues the implementation of `run\_epoch`; this line is part of the surrounding multi-line expression or statement.

L268 optimizer.zero_grad(set_to_none=True)

Explanation: Continues the implementation of `run\_epoch`; this line is part of the surrounding multi-line expression or statement.

L270 total_loss += loss.detach().item()

Explanation: Continues the implementation of `run\_epoch`; this line is part of the surrounding multi-line expression or statement.

L271 for name, value in batch_components.items():

Explanation: Loop: iterates over the specified sequence/range and executes the following block for each item.

L272 components[name] += value

Explanation: Continues the implementation of `run\_epoch`; this line is part of the surrounding multi-line expression or statement.

L273 metrics.update()

Explanation: Continues the implementation of `run\_epoch`; this line is part of the surrounding multi-line expression or statement.

L274 probabilities.detach(),

Explanation: Continues the surrounding multi-line call/container with this argument or value.

L275 mask,

Explanation: Continues the surrounding multi-line argument, tuple, list, or unpacking expression.

L276 outputs["boundary_logits"].detach().sigmoid(),

Explanation: Continues the surrounding multi-line call/container with this argument or value.

L277 boundary,

Explanation: Continues the surrounding multi-line argument, tuple, list, or unpacking expression.

L278)

Explanation: Closes the multi-line call/list/tuple/array started above.

L280 batches = len(loader)

Explanation: Assigns the result of this expression to `batches` for use later in `run\_epoch`.

L281 mean_iou, per_class, boundary_f1 = metrics.results()

Explanation: Continues the implementation of `run\_epoch`; this line is part of the surrounding multi-line expression or statement.

L282 return (

Explanation: Returns the computed value(s) to the caller.

L283 total_loss / batches,

Explanation: Continues the surrounding multi-line call/container with this argument or value.

L284 mean_iou,

Explanation: Continues the surrounding multi-line argument, tuple, list, or unpacking expression.

L285 per_class,

Explanation: Continues the surrounding multi-line argument, tuple, list, or unpacking expression.

L286 boundary_f1,

Explanation: Continues the surrounding multi-line argument, tuple, list, or unpacking expression.

L287 {name: value / batches for name, value in components.items()},

Explanation: Continues the surrounding multi-line call/container with this argument or value.

L288)

Explanation: Closes the multi-line call/list/tuple/array started above.

L291 def parse_args() -> argparse.Namespace:

Explanation: Defines and validates the training command-line interface.

L292 parser = argparse.ArgumentParser()

Explanation: Assigns the result of this expression to `parser` for use later in `parse\_args`.

L293 parser.add_argument("--data", type=Path, required=True)

Explanation: Defines command-line option `--data` and its parsing/default behavior.

L294 `parser.add_argument(`

Explanation: Adds a command-line argument to the parser.

L295 `--val-data",`

Explanation: Adds this string literal to the surrounding tuple/list/message or multi-line string.

L296 `type=Path,`

Explanation: Assigns the result of this expression to `type` for use later in `parse\_args`.

L297 `help="Separate specimen-level validation dataset. Recommended.",`

Explanation: Assigns the result of this expression to `help` for use later in `parse\_args`.

L298 `)`

Explanation: Closes the multi-line call/list/tuple/array started above.

L299 `parser.add_argument("--epochs", type=int, default=80)`

Explanation: Defines command-line option `--epochs` and its parsing/default behavior.

L300 `parser.add_argument("--batch-size", type=int, default=4)`

Explanation: Defines command-line option `--batch-size` and its parsing/default behavior.

L301 `parser.add_argument("--crop-size", type=int, default=512)`

Explanation: Defines command-line option `--crop-size` and its parsing/default behavior.

L302 `parser.add_argument("--context-scale", type=int, default=4)`

Explanation: Defines command-line option `--context-scale` and its parsing/default behavior.

L303 `parser.add_argument("--train-patches-per-image", type=int, default=2)`

Explanation: Defines command-line option `--train-patches-per-image` and its parsing/default behavior.

L304 `parser.add_argument("--val-patches-per-image", type=int, default=4)`

Explanation: Defines command-line option `--val-patches-per-image` and its parsing/default behavior.

L305 `parser.add_argument("--boundary-focus", type=float, default=0.45)`

Explanation: Defines command-line option `--boundary-focus` and its parsing/default behavior.

L306 `parser.add_argument("--class-focus", type=float, default=0.35)`

Explanation: Defines command-line option `--class-focus` and its parsing/default behavior.

L307 `parser.add_argument("--lr", type=float, default=3e-4)`

Explanation: Defines command-line option `--lr` and its parsing/default behavior.

L308 `parser.add_argument("--encoder-lr", type=float, default=3e-5)`

Explanation: Defines command-line option `--encoder-lr` and its parsing/default behavior.

L309 `parser.add_argument("--weight-decay", type=float, default=1e-4)`

Explanation: Defines command-line option `--weight-decay` and its parsing/default behavior.

L310 `parser.add_argument("--accumulation-steps", type=int, default=1)`

Explanation: Defines command-line option `--accumulation-steps` and its parsing/default behavior.

L311 `parser.add_argument("--gradient-clip", type=float, default=1.0)`

Explanation: Defines command-line option `--gradient-clip` and its parsing/default behavior.

L312 `parser.add_argument("--workers", type=int, default=4)`

Explanation: Defines command-line option `--workers` and its parsing/default behavior.

L313 `parser.add_argument("--device", choices=("auto", "cpu", "cuda"), default="auto")`

Explanation: Defines command-line option `--device` and its parsing/default behavior.

L314 `parser.add_argument("--no-amp", action="store_true")`

Explanation: Defines command-line option `--no-amp` and its parsing/default behavior.

L315 `parser.add_argument("--no-pretrained", action="store_true")`

Explanation: Defines command-line option `--no-pretrained` and its parsing/default behavior.

L316 `parser.add_argument("--class-weights", type=float, nargs=6)`

Explanation: Defines command-line option `--class-weights` and its parsing/default behavior.

L317 `parser.add_argument("--output", type=Path, default=Path("best_model.pt"))`

Explanation: Defines command-line option `--output` and its parsing/default behavior.

L318 `parser.add_argument("--last-output", type=Path, default=Path("last_model.pt"))`

Explanation: Defines command-line option `--last-output` and its parsing/default behavior.

L319 `parser.add_argument("--resume", type=Path)`

Explanation: Defines command-line option `--resume` and its parsing/default behavior.

L320 `parser.add_argument("--seed", type=int, default=42)`

Explanation: Defines command-line option `--seed` and its parsing/default behavior.

L321 args = parser.parse_args()

Explanation: Assigns the result of this expression to `args` for use later in `parse\_args`.

L322 if args.crop_size <= 0 or args.crop_size % 32:

Explanation: Conditional branch: evaluates this condition and executes the indented block only when it is true.

L323 parser.error("--crop-size must be a positive multiple of 32")

Explanation: Terminates argument parsing with a user-facing validation error.

L324 if args.context_scale < 1:

Explanation: Conditional branch: evaluates this condition and executes the indented block only when it is true.

L325 parser.error("--context-scale must be at least 1")

Explanation: Terminates argument parsing with a user-facing validation error.

L326 if args.batch_size < 1 or args.accumulation_steps < 1:

Explanation: Conditional branch: evaluates this condition and executes the indented block only when it is true.

L327 parser.error("Batch and accumulation sizes must be positive")

Explanation: Terminates argument parsing with a user-facing validation error.

L328 if args.boundary_focus + args.class_focus > 1:

Explanation: Conditional branch: evaluates this condition and executes the indented block only when it is true.

L329 parser.error("--boundary-focus + --class-focus cannot exceed 1")

Explanation: Terminates argument parsing with a user-facing validation error.

L330 return args

Explanation: Returns the computed value(s) to the caller.

L333 def main() -> None:

Explanation: Seeds RNGs, selects device, builds data/model/optimizer/scheduler/scaler, resumes if requested, then trains and checkpoints.

L334 args = parse_args()

Explanation: Assigns the result of this expression to `args` for use later in `main`.

L335 torch.manual_seed(args.seed)

Explanation: Executes this numerical/layer operation as part of the surrounding expression.

L336 random.seed(args.seed)

Explanation: Continues the implementation of `main`; this line is part of the surrounding multi-line expression or statement.

L337 np.random.seed(args.seed)

Explanation: Executes this numerical/layer operation as part of the surrounding expression.

L339 if args.device == "cuda" and not torch.cuda.is_available():

Explanation: Conditional branch: evaluates this condition and executes the indented block only when it is true.

L340 raise RuntimeError(

Explanation: Stops execution with an explicit error because an input/state invariant is violated.

L341 "CUDA was requested but is unavailable. Install a CUDA-enabled "

Explanation: Adds this string literal to the surrounding tuple/list/message or multi-line string.

L342 "PyTorch build and an NVIDIA driver."

Explanation: Adds this string literal to the surrounding tuple/list/message or multi-line string.

L343)

Explanation: Closes the multi-line call/list/tuple/array started above.

L344 device = torch.device(

Explanation: Computes tensor quantity `device` using PyTorch operations in the surrounding main logic.

L345 "cuda"

Explanation: Adds this string literal to the surrounding tuple/list/message or multi-line string.

L346 if args.device == "cuda"

Explanation: Conditional branch: evaluates this condition and executes the indented block only when it is true.

L347 or (args.device == "auto" and torch.cuda.is_available())

Explanation: Continues the implementation of `main`; this line is part of the surrounding multi-line expression or statement.

L348 else "cpu"

Explanation: Continues the implementation of `main`; this line is part of the surrounding multi-line expression or statement.

L349)

Explanation: Closes the multi-line call/list/tuple/array started above.

L350 amp = device.type == "cuda" and not args.no_amp

Explanation: Assigns the result of this expression to `amp` for use later in `main`.

L351 if device.type == "cuda":

Explanation: Conditional branch: evaluates this condition and executes the indented block only when it is true.

L352 torch.backends.cudnn.benchmark = True

Explanation: Assigns the result of this expression to `torch.backends.cudnn.benchmark` for use later in `main`.

L353 else:

Explanation: Fallback branch used when the preceding condition is false.

L354 torch.set_num_threads(int(os.environ.get("OMP_NUM_THREADS", "8")))

Explanation: Executes this numerical/layer operation as part of the surrounding expression.

L355 print(f'Using device={device}, AMP={amp}')

Explanation: Prints runtime progress/diagnostic information to standard output.

L356 if device.type == "cuda":

Explanation: Conditional branch: evaluates this condition and executes the indented block only when it is true.

L357 print(f'GPU: {torch.cuda.get_device_name(0)}')

Explanation: Prints runtime progress/diagnostic information to standard output.

L359 resume_checkpoint = None

Explanation: Assigns the result of this expression to `resume\_checkpoint` for use later in `main`.

L360 runtime_context_scale = args.context_scale

Explanation: Assigns the result of this expression to `runtime\_context\_scale` for use later in `main`.

L361 if args.resume:

Explanation: Conditional branch: evaluates this condition and executes the indented block only when it is true.

L362 resume_checkpoint = torch.load(

Explanation: Computes tensor quantity `resume\_checkpoint` using PyTorch operations in the surrounding main logic.

L363 args.resume, map_location="cpu", weights_only=False

Explanation: Continues the implementation of `main`; this line is part of the surrounding multi-line expression or statement.

L364)

Explanation: Closes the multi-line call/list/tuple/array started above.

L365 runtime_context_scale = int(

Explanation: Assigns the result of this expression to `runtime\_context\_scale` for use later in `main`.

L366 resume_checkpoint.get("model_config", {}).get(

Explanation: Continues the implementation of `main`; this line is part of the surrounding multi-line expression or statement.

L367 "context_scale", args.context_scale

Explanation: Adds this string literal to the surrounding tuple/list/message or multi-line string.

L368)

Explanation: Closes the multi-line call/list/tuple/array started above.

L369)

Explanation: Closes the multi-line call/list/tuple/array started above.

L371 if args.val_data:

Explanation: Conditional branch: evaluates this condition and executes the indented block only when it is true.

L372 train_indices = None

Explanation: Assigns the result of this expression to `train\_indices` for use later in `main`.

L373 val_indices = None

Explanation: Assigns the result of this expression to `val\_indices` for use later in `main`.

L374 validation_root = args.val_data

Explanation: Assigns the result of this expression to `validation\_root` for use later in `main`.

L375 else:

Explanation: Fallback branch used when the preceding condition is false.

L376 sample_count = len(discover_samples(args.data))

Explanation: Assigns the result of this expression to `sample\_count` for use later in `main`.

L377 order = torch.randperm(

Explanation: Creates random tensor data for `order` used by the smoke test.

L378 sample_count, generator=torch.Generator().manual_seed(args.seed)
Explanation: Continues the implementation of `main`; this line is part of the surrounding multi-line expression or statement.

L379).tolist()
Explanation: Closes or continues the surrounding multi-line Python expression.

L380 validation_count = max(1, round(0.15 * sample_count))
Explanation: Assigns the result of this expression to `validation\_count` for use later in `main`.

L381 val_indices = order[:validation_count]
Explanation: Assigns the result of this expression to `val\_indices` for use later in `main`.

L382 train_indices = order[validation_count:]
Explanation: Assigns the result of this expression to `train\_indices` for use later in `main`.

L383 validation_root = args.data
Explanation: Assigns the result of this expression to `validation\_root` for use later in `main`.

L384 print(
Explanation: Prints runtime progress/diagnostic information to standard output.

L385 "Warning: using an automatic image-level 85/15 split. For final "
Explanation: Adds this string literal to the surrounding tuple/list/message or multi-line string.

L386 "measurements, pass --val-data split by patient/specimen."
Explanation: Adds this string literal to the surrounding tuple/list/message or multi-line string.

L387)
Explanation: Closes the multi-line call/list/tuple/array started above.

L389 train_dataset = SegmentationDataset(
Explanation: Assigns the result of this expression to `train\_dataset` for use later in `main`.

L390 args.data,
Explanation: Continues the surrounding multi-line call/container with this argument or value.

L391 augment=True,
Explanation: Assigns the result of this expression to `augment` for use later in `main`.

L392 crop_size=args.crop_size,
Explanation: Assigns the result of this expression to `crop\_size` for use later in `main`.

L393 context_scale=runtime_context_scale,
Explanation: Assigns the result of this expression to `context\_scale` for use later in `main`.

L394 boundary_focus_probability=args.boundary_focus,
Explanation: Assigns the result of this expression to `boundary\_focus\_probability` for use later in `main`.

L395 class_focus_probability=args.class_focus,
Explanation: Assigns the result of this expression to `class\_focus\_probability` for use later in `main`.

L396 patches_per_image=args.train_patches_per_image,
Explanation: Assigns the result of this expression to `patches\_per\_image` for use later in `main`.

L397 sample_indices=train_indices,
Explanation: Assigns the result of this expression to `sample\_indices` for use later in `main`.

L398)
Explanation: Closes the multi-line call/list/tuple/array started above.

L399 val_dataset = SegmentationDataset(
Explanation: Assigns the result of this expression to `val\_dataset` for use later in `main`.

L400 validation_root,
Explanation: Continues the surrounding multi-line argument, tuple, list, or unpacking expression.

L401 augment=False,
Explanation: Assigns the result of this expression to `augment` for use later in `main`.

L402 crop_size=args.crop_size,
Explanation: Assigns the result of this expression to `crop\_size` for use later in `main`.

L403 context_scale=runtime_context_scale,
Explanation: Assigns the result of this expression to `context\_scale` for use later in `main`.

L404 boundary_focus_probability=0,
Explanation: Assigns the result of this expression to `boundary\_focus\_probability` for use later in `main`.

L405 class_focus_probability=0,

Explanation: Assigns the result of this expression to `class\_focus\_probability` for use later in `main`.

L406 patches_per_image=args.val_patches_per_image,

Explanation: Assigns the result of this expression to `patches\_per\_image` for use later in `main`.

L407 sample_indices=val_indices,

Explanation: Assigns the result of this expression to `sample\_indices` for use later in `main`.

L408)

Explanation: Closes the multi-line call/list/tuple/array started above.

L409 train_loader = make_loader(

Explanation: Assigns the result of this expression to `train\_loader` for use later in `main`.

L410 train_dataset, args.batch_size, args.workers, True, device.type == "cuda"

Explanation: Continues the implementation of `main`; this line is part of the surrounding multi-line expression or statement.

L411)

Explanation: Closes the multi-line call/list/tuple/array started above.

L412 val_loader = make_loader(

Explanation: Assigns the result of this expression to `val\_loader` for use later in `main`.

L413 val_dataset, args.batch_size, args.workers, False, device.type == "cuda"

Explanation: Continues the implementation of `main`; this line is part of the surrounding multi-line expression or statement.

L414)

Explanation: Closes the multi-line call/list/tuple/array started above.

L416 model_configuration = (

Explanation: Assigns the result of this expression to `model\_configuration` for use later in `main`.

L417 resume_checkpoint["model_config"]

Explanation: Continues the implementation of `main`; this line is part of the surrounding multi-line expression or statement.

L418 if resume_checkpoint is not None

Explanation: Conditional branch: evaluates this condition and executes the indented block only when it is true.

L419 else {

Explanation: Continues the implementation of `main`; this line is part of the surrounding multi-line expression or statement.

L420 "context_scale": runtime_context_scale,

Explanation: Adds this string literal to the surrounding tuple/list/message or multi-line string.

L421 "pretrained": not args.no_pretrained,

Explanation: Adds this string literal to the surrounding tuple/list/message or multi-line string.

L422 }

Explanation: Closes the multi-line container started above.

L423)

Explanation: Closes the multi-line call/list/tuple/array started above.

L424 model = AccurateTissueNet(**model_configuration)

Explanation: Assigns the result of this expression to `model` for use later in `main`.

L425 if device.type == "cuda":

Explanation: Conditional branch: evaluates this condition and executes the indented block only when it is true.

L426 model = model.to(memory_format=torch.channels_last)

Explanation: Computes tensor quantity `model` using PyTorch operations in the surrounding main logic.

L427 model = model.to(device)

Explanation: Moves `model` data to the selected compute device, allowing non-blocking transfer when possible.

L429 encoder_prefixes = (

Explanation: Assigns the result of this expression to `encoder\_prefixes` for use later in `main`.

L430 "rgb_",

Explanation: Adds this string literal to the surrounding tuple/list/message or multi-line string.

L431 "local_",

Explanation: Adds this string literal to the surrounding tuple/list/message or multi-line string.

L432 "context_",

Explanation: Adds this string literal to the surrounding tuple/list/message or multi-line string.

L433)

Explanation: Closes the multi-line call/list/tuple/array started above.

L434 encoder_parameters = []

Explanation: Assigns the result of this expression to `encoder\_parameters` for use later in `main`.

L435 other_parameters = []

Explanation: Assigns the result of this expression to `other\_parameters` for use later in `main`.

L436 for name, parameter in model.named_parameters():

Explanation: Loop: iterates over the specified sequence/range and executes the following block for each item.

L437 target = (

Explanation: Assigns the result of this expression to `target` for use later in `main`.

L438 encoder_parameters

Explanation: Continues the surrounding multi-line argument, tuple, list, or unpacking expression.

L439 if name.startswith(encoder_prefixes)

Explanation: Conditional branch: evaluates this condition and executes the indented block only when it is true.

L440 else other_parameters

Explanation: Continues the implementation of `main`; this line is part of the surrounding multi-line expression or statement.

L441)

Explanation: Closes the multi-line call/list/tuple/array started above.

L442 target.append(parameter)

Explanation: Continues the implementation of `main`; this line is part of the surrounding multi-line expression or statement.

L443 optimizer = torch.optim.AdamW(

Explanation: Computes tensor quantity `optimizer` using PyTorch operations in the surrounding main logic.

L444 [

Explanation: Continues the implementation of `main`; this line is part of the surrounding multi-line expression or statement.

L445 {"params": encoder_parameters, "lr": args.encoder_lr},

Explanation: Continues the surrounding multi-line call/container with this argument or value.

L446 {"params": other_parameters, "lr": args.lr},

Explanation: Continues the surrounding multi-line call/container with this argument or value.

L447],

Explanation: Closes or continues the surrounding multi-line Python expression.

L448 weight_decay=args.weight_decay,

Explanation: Assigns the result of this expression to `weight\_decay` for use later in `main`.

L449)

Explanation: Closes the multi-line call/list/tuple/array started above.

L450 scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(

Explanation: Computes tensor quantity `scheduler` using PyTorch operations in the surrounding main logic.

L451 optimizer, mode="max", patience=6, factor=0.5, min_lr=1e-7

Explanation: Continues the implementation of `main`; this line is part of the surrounding multi-line expression or statement.

L452)

Explanation: Closes the multi-line call/list/tuple/array started above.

L453 scaler = torch.cuda.amp.GradScaler(enabled=amp)

Explanation: Creates the CUDA gradient scaler used with mixed precision; modern PyTorch also provides the device-agnostic torch.amp GradScaler API.

L454 class_weights = (

Explanation: Assigns the result of this expression to `class\_weights` for use later in `main`.

L455 torch.tensor(args.class_weights, device=device)

Explanation: Executes this numerical/layer operation as part of the surrounding expression.

L456 if args.class_weights

Explanation: Conditional branch: evaluates this condition and executes the indented block only when it is true.

L457 else None

Explanation: Continues the implementation of `main`; this line is part of the surrounding multi-line expression or statement.

L458)

Explanation: Closes the multi-line call/list/tuple/array started above.

L460 start_epoch, best_iou = 1, -1.0

Explanation: Continues the implementation of `main`; this line is part of the surrounding multi-line expression or statement.

L461 if resume_checkpoint is not None:

Explanation: Conditional branch: evaluates this condition and executes the indented block only when it is true.

L462 model.load_state_dict(resume_checkpoint["model"])

Explanation: Continues the implementation of `main`; this line is part of the surrounding multi-line expression or statement.

L463 optimizer.load_state_dict(resume_checkpoint["optimizer"])

Explanation: Continues the implementation of `main`; this line is part of the surrounding multi-line expression or statement.

L464 scheduler.load_state_dict(resume_checkpoint["scheduler"])

Explanation: Continues the implementation of `main`; this line is part of the surrounding multi-line expression or statement.

L465 scaler.load_state_dict(resume_checkpoint["scaler"])

Explanation: Continues the implementation of `main`; this line is part of the surrounding multi-line expression or statement.

L466 start_epoch = int(resume_checkpoint["epoch"]) + 1

Explanation: Assigns the result of this expression to `start\_epoch` for use later in `main`.

L467 best_iou = float(resume_checkpoint.get("best_miou", -1))

Explanation: Assigns the result of this expression to `best\_iou` for use later in `main`.

L468 print(f"Resumed {args.resume} at epoch {start_epoch}")

Explanation: Prints runtime progress/diagnostic information to standard output.

L469 del resume_checkpoint

Explanation: Drops references early so large tensors/arrays can be reclaimed before later work.

L471 args.output.parent.mkdir(parents=True, exist_ok=True)

Explanation: Continues the implementation of `main`; this line is part of the surrounding multi-line expression or statement.

L472 args.last_output.parent.mkdir(parents=True, exist_ok=True)

Explanation: Continues the implementation of `main`; this line is part of the surrounding multi-line expression or statement.

L473 for epoch in range(start_epoch, args.epochs + 1):

Explanation: Loop: iterates over the specified sequence/range and executes the following block for each item.

L474 train_result = run_epoch(

Explanation: Assigns the result of this expression to `train\_result` for use later in `main`.

L475 model,

Explanation: Continues the surrounding multi-line argument, tuple, list, or unpacking expression.

L476 train_loader,

Explanation: Continues the surrounding multi-line argument, tuple, list, or unpacking expression.

L477 device,

Explanation: Continues the surrounding multi-line argument, tuple, list, or unpacking expression.

L478 class_weights,

Explanation: Continues the surrounding multi-line argument, tuple, list, or unpacking expression.

L479 amp,

Explanation: Continues the surrounding multi-line argument, tuple, list, or unpacking expression.

L480 optimizer=optimizer,

Explanation: Assigns the result of this expression to `optimizer` for use later in `main`.

L481 scaler=scaler,

Explanation: Assigns the result of this expression to `scaler` for use later in `main`.

L482 accumulation_steps=args.accumulation_steps,

Explanation: Assigns the result of this expression to `accumulation\_steps` for use later in `main`.

L483 gradient_clip=args.gradient_clip,

Explanation: Assigns the result of this expression to `gradient\_clip` for use later in `main`.

L484)

Explanation: Closes the multi-line call/list/tuple/array started above.

L485 with torch.inference_mode():

Explanation: Runs the following block without gradient tracking.

L486 val_result = run_epoch(

Explanation: Assigns the result of this expression to `val\_result` for use later in `main`.

L487 model,

Explanation: Continues the surrounding multi-line argument, tuple, list, or unpacking expression.

L488 val_loader,

Explanation: Continues the surrounding multi-line argument, tuple, list, or unpacking expression.

L489 device,

Explanation: Continues the surrounding multi-line argument, tuple, list, or unpacking expression.

L490 class_weights,

Explanation: Continues the surrounding multi-line argument, tuple, list, or unpacking expression.

L491 amp,

Explanation: Continues the surrounding multi-line argument, tuple, list, or unpacking expression.

L492)

Explanation: Closes the multi-line call/list/tuple/array started above.

L493 train_loss, train_iou = train_result[0], train_result[1]

Explanation: Continues the implementation of `main`; this line is part of the surrounding multi-line expression or statement.

L494 val_loss, val_iou, class_iou, boundary_f1, components = val_result

Explanation: Continues the implementation of `main`; this line is part of the surrounding multi-line expression or statement.

L495 scheduler.step(val_iou)

Explanation: Continues the implementation of `main`; this line is part of the surrounding multi-line expression or statement.

L496 class_summary = ".join(

Explanation: Assigns the result of this expression to `class\_summary` for use later in `main`.

L497 f'{name}={score:.3f}"

Explanation: Continues the implementation of `main`; this line is part of the surrounding multi-line expression or statement.

L498 for name, score in zip(CLASS_NAMES, class_iou)

Explanation: Loop: iterates over the specified sequence/range and executes the following block for each item.

L499)

Explanation: Closes the multi-line call/list/tuple/array started above.

L500 print(

Explanation: Prints runtime progress/diagnostic information to standard output.

L501 f'{epoch:03d}/{args.epochs} train loss={train_loss:.4f} "

Explanation: Continues the implementation of `main`; this line is part of the surrounding multi-line expression or statement.

L502 f'mIoU={train_iou:.3f} | val loss={val_loss:.4f} "

Explanation: Continues the implementation of `main`; this line is part of the surrounding multi-line expression or statement.

L503 f'mIoU={val_iou:.3f} boundaryF1={boundary_f1:.3f}"

Explanation: Continues the implementation of `main`; this line is part of the surrounding multi-line expression or statement.

L504)

Explanation: Closes the multi-line call/list/tuple/array started above.

L505 print(f" IoU: {class_summary}")

Explanation: Prints runtime progress/diagnostic information to standard output.

L506 print(

Explanation: Prints runtime progress/diagnostic information to standard output.

L507 " losses: "

Explanation: Adds this string literal to the surrounding tuple/list/message or multi-line string.

L508 + " ".join(f'{name}={value:.4f}' for name, value in components.items())

Explanation: Continues the implementation of `main`; this line is part of the surrounding multi-line expression or statement.

L509)

Explanation: Closes the multi-line call/list/tuple/array started above.

L511 checkpoint = {

Explanation: Assigns the result of this expression to `checkpoint` for use later in `main`.

L512 "model": model.state_dict(),

Explanation: Adds this string literal to the surrounding tuple/list/message or multi-line string.

L513 "model_config": model.model_config(),

Explanation: Adds this string literal to the surrounding tuple/list/message or multi-line string.

L514 "classes": CLASS_NAMES,

Explanation: Adds this string literal to the surrounding tuple/list/message or multi-line string.

L515 "epoch": epoch,

Explanation: Adds this string literal to the surrounding tuple/list/message or multi-line string.

L516 "best_miou": max(best_iou, val_iou),

Explanation: Adds this string literal to the surrounding tuple/list/message or multi-line string.

L517 "val_miou": val_iou,

Explanation: Adds this string literal to the surrounding tuple/list/message or multi-line string.

L518 "val_class_iou": class_iou,

Explanation: Adds this string literal to the surrounding tuple/list/message or multi-line string.

L519 "val_boundary_f1": boundary_f1,

Explanation: Adds this string literal to the surrounding tuple/list/message or multi-line string.

L520 "optimizer": optimizer.state_dict(),

Explanation: Adds this string literal to the surrounding tuple/list/message or multi-line string.

L521 "scheduler": scheduler.state_dict(),

Explanation: Adds this string literal to the surrounding tuple/list/message or multi-line string.

L522 "scaler": scaler.state_dict(),

Explanation: Adds this string literal to the surrounding tuple/list/message or multi-line string.

L523 "args": vars(args),

Explanation: Adds this string literal to the surrounding tuple/list/message or multi-line string.

L524 }

Explanation: Closes the multi-line container started above.

L525 torch.save(checkpoint, args.last_output)

Explanation: Executes this numerical/layer operation as part of the surrounding expression.

L526 if val_iou > best_iou:

Explanation: Conditional branch: evaluates this condition and executes the indented block only when it is true.

L527 best_iou = val_iou

Explanation: Assigns the result of this expression to `best\_iou` for use later in `main`.

L528 torch.save(checkpoint, args.output)

Explanation: Executes this numerical/layer operation as part of the surrounding expression.

L529 print(f" saved new best checkpoint: {args.output}")

Explanation: Prints runtime progress/diagnostic information to standard output.

L531 print(f"Best validation mIoU={best_iou:.3f}; checkpoint={args.output}")

Explanation: Prints runtime progress/diagnostic information to standard output.

L534 if __name__ == "__main__":

Explanation: Python entry-point guard: the indented call runs only when this file is executed as a script.

L535 main()

Explanation: Continues the implementation of `train.py`; this line is part of the surrounding multi-line expression or statement.

predict.py

Major definitions: tile_starts, blend_window, temporary_path, close_memmap, extract_context, tiled_prediction, save_prediction_images, parse_args, main

L001 from __future__ import annotations

Explanation: Postpones evaluation of type annotations, reducing runtime coupling and allowing forward references.

L003 import argparse

Explanation: Imports dependencies used by this module: command-line argument parsing

L004 from itertools import islice, product

Explanation: Imports dependencies used by this module: batching an iterator without materializing all coordinates; Cartesian product of tile row/column starts

L005 from pathlib import Path

Explanation: Imports dependencies used by this module: filesystem paths

L006 import tempfile

Explanation: Imports dependencies used by this module: safe temporary-file creation

L008 import numpy as np

Explanation: Imports dependencies used by this module: array and numerical operations

L009 import torch

Explanation: Imports dependencies used by this module: tensor computation and automatic differentiation

L010 from PIL import Image

Explanation: Imports dependencies used by this module: image I/O and resizing

L012 from model import AccurateTissueNet, CLASS_NAMES

Explanation: Imports dependencies used by this module.

L015 Image.MAX_IMAGE_PIXELS = None

Explanation: Assigns the result of this expression to 'Image.MAX_IMAGE_PIXELS' for use later in 'predict.py'.

L016 PALETTE = np.array(

Explanation: Assigns 'PALETTE' as the RGB colors used for the saved indexed color mask.

L017 [

Explanation: Continues the implementation of 'predict.py'; this line is part of the surrounding multi-line expression or statement.

L018 [240, 220, 80],

Explanation: Continues the surrounding multi-line call/container with this argument or value.

L019 [30, 180, 210],

Explanation: Continues the surrounding multi-line call/container with this argument or value.

L020 [60, 210, 110],

Explanation: Continues the surrounding multi-line call/container with this argument or value.

L021 [210, 70, 90],

Explanation: Continues the surrounding multi-line call/container with this argument or value.

L022 [150, 80, 190],

Explanation: Continues the surrounding multi-line call/container with this argument or value.

L023 [0, 0, 0],

Explanation: Continues the surrounding multi-line call/container with this argument or value.

L024],

Explanation: Closes or continues the surrounding multi-line Python expression.

L025 dtype=np.uint8,

Explanation: Computes NumPy quantity 'dtype' used by the surrounding module logic.

L026)

Explanation: Closes the multi-line call/list/tuple/array started above.

L029 def tile_starts(length: int, tile_size: int, overlap: int) -> list[int]:

Explanation: Computes tile origins that cover an image and forces the last tile to align with the far edge.

L030 if length <= tile_size:

Explanation: Conditional branch: evaluates this condition and executes the indented block only when it is true.

L031 return [0]

Explanation: Returns the computed value(s) to the caller.

L032 step = tile_size - overlap

Explanation: Assigns the result of this expression to `step` for use later in `tile\_starts`.

L033 starts = list(range(0, length - tile_size + 1, step))

Explanation: Assigns the result of this expression to `starts` for use later in `tile\_starts`.

L034 if starts[-1] != length - tile_size:

Explanation: Conditional branch: evaluates this condition and executes the indented block only when it is true.

L035 starts.append(length - tile_size)

Explanation: Continues the implementation of `tile\_starts`; this line is part of the surrounding multi-line expression or statement.

L036 return starts

Explanation: Returns the computed value(s) to the caller.

L039 def blend_window(tile_size: int, overlap: int) -> np.ndarray:

Explanation: Builds a center-weighted Hann blending window with a nonzero floor at borders.

L040 if overlap == 0:

Explanation: Conditional branch: evaluates this condition and executes the indented block only when it is true.

L041 return np.ones((tile_size, tile_size), dtype=np.float32)

Explanation: Returns the computed value(s) to the caller.

L042 axis = np.hanning(tile_size + 2)[1:-1].astype(np.float32)

Explanation: Computes NumPy quantity `axis` used by the surrounding blend_window logic.

L043 # A small floor retains stable values at the outer image boundary.

Explanation: Developer comment: A small floor retains stable values at the outer image boundary.

L044 axis = np.maximum(axis, 0.05)

Explanation: Computes NumPy quantity `axis` used by the surrounding blend_window logic.

L045 window = np.outer(axis, axis)

Explanation: Computes NumPy quantity `window` used by the surrounding blend_window logic.

L046 return window / window.max()

Explanation: Returns the computed value(s) to the caller.

L049 def temporary_path(directory: Path, prefix: str, suffix: str) -> Path:

Explanation: Reserves a uniquely named temporary path in a target directory.

L050 with tempfile.NamedTemporaryFile(

Explanation: Context manager ensuring the opened resource/state is cleaned up correctly after the block.

L051 dir=directory, prefix=prefix, suffix=suffix, delete=False

Explanation: Assigns the result of this expression to `dir` for use later in `temporary\_path`.

L052) as temporary_file:

Explanation: Closes or continues the surrounding multi-line Python expression.

L053 return Path(temporary_file.name)

Explanation: Returns the computed value(s) to the caller.

L056 def close_memmap(array: np.memmap) -> None:

Explanation: Flushes a NumPy memory map and explicitly closes its underlying mmap handle.

L057 array.flush()

Explanation: Continues the implementation of `close\_memmap`; this line is part of the surrounding multi-line expression or statement.

L058 memory_map = getattr(array, "_mmap", None)

Explanation: Assigns the result of this expression to `memory\_map` for use later in `close\_memmap`.

L059 if memory_map is not None:

Explanation: Conditional branch: evaluates this condition and executes the indented block only when it is true.

L060 memory_map.close()

Explanation: Continues the implementation of `close\_memmap`; this line is part of the surrounding multi-line expression or statement.

L063 def extract_context(

Explanation: Extracts the larger native context field around a tile center, edge-pads it, then downsamples to model input size.

L064 image: np.ndarray,

Explanation: Continues the surrounding multi-line call/container with this argument or value.

L065 center_y: int,

Explanation: Continues the surrounding multi-line call/container with this argument or value.

L066 center_x: int,

Explanation: Continues the surrounding multi-line call/container with this argument or value.

L067 native_size: int,

Explanation: Continues the surrounding multi-line call/container with this argument or value.

L068 output_size: int,

Explanation: Continues the surrounding multi-line call/container with this argument or value.

L069) -> np.ndarray:

Explanation: Closes or continues the surrounding multi-line Python expression.

L070 height, width = image.shape[:2]

Explanation: Continues the implementation of `extract\_context`; this line is part of the surrounding multi-line expression or statement.

L071 top = center_y - native_size // 2

Explanation: Assigns the result of this expression to `top` for use later in `extract\_context`.

L072 left = center_x - native_size // 2

Explanation: Assigns the result of this expression to `left` for use later in `extract\_context`.

L073 bottom, right = top + native_size, left + native_size

Explanation: Continues the implementation of `extract\_context`; this line is part of the surrounding multi-line expression or statement.

L074 pad_top, pad_left = max(0, -top), max(0, -left)

Explanation: Continues the implementation of `extract\_context`; this line is part of the surrounding multi-line expression or statement.

L075 pad_bottom, pad_right = max(0, bottom - height), max(0, right - width)

Explanation: Continues the implementation of `extract\_context`; this line is part of the surrounding multi-line expression or statement.

L076 top, left = max(0, top), max(0, left)

Explanation: Continues the implementation of `extract\_context`; this line is part of the surrounding multi-line expression or statement.

L077 bottom, right = min(height, bottom), min(width, right)

Explanation: Continues the implementation of `extract\_context`; this line is part of the surrounding multi-line expression or statement.

L078 context = image[top:bottom, left:right]

Explanation: Assigns the result of this expression to `context` for use later in `extract\_context`.

L079 if pad_top or pad_bottom or pad_left or pad_right:

Explanation: Conditional branch: evaluates this condition and executes the indented block only when it is true.

L080 context = np.pad(

Explanation: Computes NumPy quantity `context` used by the surrounding `extract\_context` logic.

L081 context,

Explanation: Continues the surrounding multi-line argument, tuple, list, or unpacking expression.

L082 (

Explanation: Continues the implementation of `extract\_context`; this line is part of the surrounding multi-line expression or statement.

L083 (pad_top, pad_bottom),

Explanation: Continues the surrounding multi-line call/container with this argument or value.

L084 (pad_left, pad_right),

Explanation: Continues the surrounding multi-line call/container with this argument or value.

L085 (0, 0),

Explanation: Continues the surrounding multi-line call/container with this argument or value.

L086),

Explanation: Closes or continues the surrounding multi-line Python expression.

L087 mode="edge",

Explanation: Assigns the result of this expression to `mode` for use later in `extract\_context`.

L088)

Explanation: Closes the multi-line call/list/tuple/array started above.

L089 context_image = Image.fromarray(np.ascontiguousarray(context))

Explanation: Computes NumPy quantity `context\_image` used by the surrounding `extract\_context` logic.

L090 resized = np.array(

Explanation: Converts image/data into an independent NumPy array stored as `resized`.

L091 context_image.resize(

Explanation: Continues the implementation of `extract\_context`; this line is part of the surrounding multi-line expression or statement.

L092 (output_size, output_size),

Explanation: Continues the surrounding multi-line call/container with this argument or value.

L093 `Image.Resampling.LANCZOS,`

Explanation: Continues the surrounding multi-line call/container with this argument or value.

L094 `reducing_gap=2.0,`

Explanation: Assigns the result of this expression to `reducing\_gap` for use later in `extract\_context`.

L095 `),`

Explanation: Closes or continues the surrounding multi-line Python expression.

L096 `dtype=np.uint8,`

Explanation: Computes NumPy quantity `dtype` used by the surrounding `extract\_context` logic.

L097 `copy=True,`

Explanation: Assigns the result of this expression to `copy` for use later in `extract\_context`.

L098 `)`

Explanation: Closes the multi-line call/list/tuple/array started above.

L099 `context_image.close()`

Explanation: Continues the implementation of `extract\_context`; this line is part of the surrounding multi-line expression or statement.

L100 `return resized`

Explanation: Returns the computed value(s) to the caller.

L103 `@torch.inference_mode()`

Explanation: Disables autograd for the following inference routine, reducing memory use and overhead.

L104 `def tiled_prediction(`

Explanation: Runs native-resolution overlapping-tile inference, accumulates weighted probabilities/boundary scores in disk-backed arrays, and normalizes by blend weights.

L105 `model: AccurateTissueNet,`

Explanation: Continues the surrounding multi-line call/container with this argument or value.

L106 `image: np.ndarray,`

Explanation: Continues the surrounding multi-line call/container with this argument or value.

L107 `device: torch.device,`

Explanation: Continues the surrounding multi-line call/container with this argument or value.

L108 `tile_size: int,`

Explanation: Continues the surrounding multi-line call/container with this argument or value.

L109 `overlap: int,`

Explanation: Continues the surrounding multi-line call/container with this argument or value.

L110 `batch_size: int,`

Explanation: Continues the surrounding multi-line call/container with this argument or value.

L111 `context_scale: int,`

Explanation: Continues the surrounding multi-line call/container with this argument or value.

L112 `probability_path: Path,`

Explanation: Continues the surrounding multi-line call/container with this argument or value.

L113 `use_amp: bool,`

Explanation: Continues the surrounding multi-line call/container with this argument or value.

L114 `) -> tuple[np.memmap, np.memmap, Path]:`

Explanation: Closes or continues the surrounding multi-line Python expression.

L115 `height, width = image.shape[:2]`

Explanation: Continues the implementation of `tiled\_prediction`; this line is part of the surrounding multi-line expression or statement.

L116 `y_starts = tile_starts(height, tile_size, overlap)`

Explanation: Assigns the result of this expression to `y\_starts` for use later in `tiled\_prediction`.

L117 `x_starts = tile_starts(width, tile_size, overlap)`

Explanation: Assigns the result of this expression to `x\_starts` for use later in `tiled\_prediction`.

L118 `coordinate_iterator = iter(product(y_starts, x_starts))`

Explanation: Assigns the result of this expression to `coordinate\_iterator` for use later in `tiled\_prediction`.

L119 `total_tiles = len(y_starts) * len(x_starts)`

Explanation: Assigns the result of this expression to `total\_tiles` for use later in `tiled\_prediction`.

L120 `window = blend_window(tile_size, overlap)`

Explanation: Assigns the result of this expression to `window` for use later in `tiled\_prediction`.

L122 `probability_sum = np.lib.format.open_memmap(`
Explanation: Assigns `probability\_sum` as the disk-backed accumulator for weighted class probabilities.

L123 `probability_path,`
Explanation: Continues the surrounding multi-line argument, tuple, list, or unpacking expression.

L124 `mode="w+",`
Explanation: Assigns the result of this expression to `mode` for use later in `tiled\_prediction`.

L125 `dtype=np.float32,`
Explanation: Computes NumPy quantity `dtype` used by the surrounding tiled_prediction logic.

L126 `shape=(len(CLASS_NAMES), height, width),`
Explanation: Assigns the result of this expression to `shape` for use later in `tiled\_prediction`.

L127 `)`
Explanation: Closes the multi-line call/list/tuple/array started above.

L128 `probability_sum[:] = 0`
Explanation: Continues the implementation of `tiled\_prediction`; this line is part of the surrounding multi-line expression or statement.

L129 `weight_path = temporary_path(`
Explanation: Assigns the result of this expression to `weight\_path` for use later in `tiled\_prediction`.

L130 `probability_path.parent, "blend_weights_", ".dat"`
Explanation: Continues the implementation of `tiled\_prediction`; this line is part of the surrounding multi-line expression or statement.

L131 `)`
Explanation: Closes the multi-line call/list/tuple/array started above.

L132 `weight_sum = np.memmap(`
Explanation: Assigns `weight\_sum` as the disk-backed accumulator of tile blending weights.

L133 `weight_path, mode="w+", dtype=np.float32, shape=(height, width)`
Explanation: Continues the implementation of `tiled\_prediction`; this line is part of the surrounding multi-line expression or statement.

L134 `)`
Explanation: Closes the multi-line call/list/tuple/array started above.

L135 `weight_sum[:] = 0`
Explanation: Continues the implementation of `tiled\_prediction`; this line is part of the surrounding multi-line expression or statement.

L136 `boundary_path = temporary_path(`
Explanation: Assigns the result of this expression to `boundary\_path` for use later in `tiled\_prediction`.

L137 `probability_path.parent, "boundary_", ".dat"`
Explanation: Continues the implementation of `tiled\_prediction`; this line is part of the surrounding multi-line expression or statement.

L138 `)`
Explanation: Closes the multi-line call/list/tuple/array started above.

L139 `boundary_sum = np.memmap(`
Explanation: Assigns `boundary\_sum` as the disk-backed accumulator for boundary probabilities.

L140 `boundary_path, mode="w+", dtype=np.float32, shape=(height, width)`
Explanation: Continues the implementation of `tiled\_prediction`; this line is part of the surrounding multi-line expression or statement.

L141 `)`
Explanation: Closes the multi-line call/list/tuple/array started above.

L142 `boundary_sum[:] = 0`
Explanation: Continues the implementation of `tiled\_prediction`; this line is part of the surrounding multi-line expression or statement.

L144 `pin_memory = device.type == "cuda"`
Explanation: Assigns the result of this expression to `pin\_memory` for use later in `tiled\_prediction`.

L145 `local_host = torch.empty(`
Explanation: Computes tensor quantity `local\_host` using PyTorch operations in the surrounding tiled_prediction logic.

L146 `(batch_size, 3, tile_size, tile_size),`
Explanation: Continues the surrounding multi-line call/container with this argument or value.

L147 `dtype=torch.float32,`
Explanation: Computes tensor quantity `dtype` using PyTorch operations in the surrounding tiled_prediction logic.

L148 `pin_memory=pin_memory,`
Explanation: Assigns the result of this expression to `pin\_memory` for use later in `tiled\_prediction`.

L149 `)`

Explanation: Closes the multi-line call/list/tuple/array started above.

L150 context_host = torch.empty(

Explanation: Computes tensor quantity `context\_host` using PyTorch operations in the surrounding tiled_prediction logic.

L151 (batch_size, 3, tile_size, tile_size),

Explanation: Continues the surrounding multi-line call/container with this argument or value.

L152 dtype=torch.float32,

Explanation: Computes tensor quantity `dtype` using PyTorch operations in the surrounding tiled_prediction logic.

L153 pin_memory=pin_memory,

Explanation: Assigns the result of this expression to `pin\_memory` for use later in `tiled\_prediction`.

L154)

Explanation: Closes the multi-line call/list/tuple/array started above.

L155 processed_tiles = 0

Explanation: Assigns the result of this expression to `processed\_tiles` for use later in `tiled\_prediction`.

L156 try:

Explanation: Starts exception-safe resource-management block.

L157 while True:

Explanation: Loop: repeats the following block while the condition remains true.

L158 coordinates = list(islice(coordinate_iterator, batch_size))

Explanation: Assigns the result of this expression to `coordinates` for use later in `tiled\_prediction`.

L159 if not coordinates:

Explanation: Conditional branch: evaluates this condition and executes the indented block only when it is true.

L160 break

Explanation: Exits the current loop.

L161 valid_shapes = []

Explanation: Assigns the result of this expression to `valid\_shapes` for use later in `tiled\_prediction`.

L162 for batch_index, (top, left) in enumerate(coordinates):

Explanation: Loop: iterates over the specified sequence/range and executes the following block for each item.

L163 local = image[top : top + tile_size, left : left + tile_size]

Explanation: Assigns the result of this expression to `local` for use later in `tiled\_prediction`.

L164 valid_height, valid_width = local.shape[:2]

Explanation: Continues the implementation of `tiled\_prediction`; this line is part of the surrounding multi-line expression or statement.

L165 valid_shapes.append((valid_height, valid_width))

Explanation: Continues the implementation of `tiled\_prediction`; this line is part of the surrounding multi-line expression or statement.

L166 if valid_height < tile_size or valid_width < tile_size:

Explanation: Conditional branch: evaluates this condition and executes the indented block only when it is true.

L167 local = np.pad(

Explanation: Computes NumPy quantity `local` used by the surrounding tiled_prediction logic.

L168 local,

Explanation: Continues the surrounding multi-line argument, tuple, list, or unpacking expression.

L169 (

Explanation: Continues the implementation of `tiled\_prediction`; this line is part of the surrounding multi-line expression or statement.

L170 (0, tile_size - valid_height),

Explanation: Continues the surrounding multi-line call/container with this argument or value.

L171 (0, tile_size - valid_width),

Explanation: Continues the surrounding multi-line call/container with this argument or value.

L172 (0, 0),

Explanation: Continues the surrounding multi-line call/container with this argument or value.

L173),

Explanation: Closes or continues the surrounding multi-line Python expression.

L174 mode="edge",

Explanation: Assigns the result of this expression to `mode` for use later in `tiled\_prediction`.

L175)

Explanation: Closes the multi-line call/list/tuple/array started above.

L176 center_y = top + tile_size // 2
Explanation: Assigns the result of this expression to `center\_y` for use later in `tiled\_prediction`.

L177 center_x = left + tile_size // 2
Explanation: Assigns the result of this expression to `center\_x` for use later in `tiled\_prediction`.

L178 context = extract_context(
Explanation: Assigns the result of this expression to `context` for use later in `tiled\_prediction`.

L179 image,
Explanation: Continues the surrounding multi-line argument, tuple, list, or unpacking expression.

L180 center_y,
Explanation: Continues the surrounding multi-line argument, tuple, list, or unpacking expression.

L181 center_x,
Explanation: Continues the surrounding multi-line argument, tuple, list, or unpacking expression.

L182 tile_size * context_scale,
Explanation: Continues the surrounding multi-line call/container with this argument or value.

L183 tile_size,
Explanation: Continues the surrounding multi-line argument, tuple, list, or unpacking expression.

L184)
Explanation: Closes the multi-line call/list/tuple/array started above.

L185 local_tensor = torch.from_numpy(
Explanation: Creates a Torch tensor view/copy path from the NumPy array and stores it as `local\_tensor`.

L186 np.ascontiguousarray(local)
Explanation: Executes this numerical/layer operation as part of the surrounding expression.

L187).permute(2, 0, 1)
Explanation: Closes or continues the surrounding multi-line Python expression.

L188 context_tensor = torch.from_numpy(context).permute(2, 0, 1)
Explanation: Creates a Torch tensor view/copy path from the NumPy array and stores it as `context\_tensor`.

L189 local_host[batch_index].copy_(local_tensor)
Explanation: Continues the implementation of `tiled\_prediction`; this line is part of the surrounding multi-line expression or statement.

L190 context_host[batch_index].copy_(context_tensor)
Explanation: Continues the implementation of `tiled\_prediction`; this line is part of the surrounding multi-line expression or statement.

L191 local_host[batch_index].div_(255)
Explanation: Continues the implementation of `tiled\_prediction`; this line is part of the surrounding multi-line expression or statement.

L192 context_host[batch_index].div_(255)
Explanation: Continues the implementation of `tiled\_prediction`; this line is part of the surrounding multi-line expression or statement.

L194 local_device = local_host[: len(coordinates)].to(
Explanation: Assigns the result of this expression to `local\_device` for use later in `tiled\_prediction`.

L195 device, non_blocking=True
Explanation: Continues the implementation of `tiled\_prediction`; this line is part of the surrounding multi-line expression or statement.

L196)
Explanation: Closes the multi-line call/list/tuple/array started above.

L197 context_device = context_host[: len(coordinates)].to(
Explanation: Assigns the result of this expression to `context\_device` for use later in `tiled\_prediction`.

L198 device, non_blocking=True
Explanation: Continues the implementation of `tiled\_prediction`; this line is part of the surrounding multi-line expression or statement.

L199)
Explanation: Closes the multi-line call/list/tuple/array started above.

L200 if device.type == "cuda":
Explanation: Conditional branch: evaluates this condition and executes the indented block only when it is true.

L201 local_device = local_device.contiguous(
Explanation: Assigns the result of this expression to `local\_device` for use later in `tiled\_prediction`.

L202 memory_format=torch.channels_last
Explanation: Computes tensor quantity `memory\_format` using PyTorch operations in the surrounding tiled_prediction logic.

L203)

Explanation: Closes the multi-line call/list/tuple/array started above.

L204 context_device = context_device.contiguous(

Explanation: Assigns the result of this expression to `context\_device` for use later in `tiled\_prediction`.

L205 memory_format=torch.channels_last

Explanation: Computes tensor quantity `memory\_format` using PyTorch operations in the surrounding tiled_prediction logic.

L206)

Explanation: Closes the multi-line call/list/tuple/array started above.

L207 with torch.autocast(

Explanation: Opens an automatic mixed-precision region; on CUDA, selected operations may run in float16 for speed/memory savings.

L208 device_type=device.type,

Explanation: Assigns the result of this expression to `device\_type` for use later in `tiled\_prediction`.

L209 dtype=torch.float16,

Explanation: Computes tensor quantity `dtype` using PyTorch operations in the surrounding tiled_prediction logic.

L210 enabled=device.type == "cuda" and use_amp,

Explanation: Assigns the result of this expression to `enabled` for use later in `tiled\_prediction`.

L211):

Explanation: Closes or continues the surrounding multi-line Python expression.

L212 outputs = model(local_device, context_device)

Explanation: Assigns the result of this expression to `outputs` for use later in `tiled\_prediction`.

L213 probabilities = model.probabilities_from_outputs(outputs)

Explanation: Builds final six-class probabilities inside the CUDA autocast block. Thus inference also performs the hierarchical split in reduced precision before line 215 casts to float32.

L214 boundaries = outputs["boundary_logits"].sigmoid()

Explanation: Assigns the result of this expression to `boundaries` for use later in `tiled\_prediction`.

L215 probabilities_cpu = probabilities.float().cpu().numpy()

Explanation: Casts probabilities to float32 only after the hierarchical probability calculation, preserving any earlier FP16 rounding.

L216 boundaries_cpu = boundaries[:, 0].float().cpu().numpy()

Explanation: Assigns the result of this expression to `boundaries\_cpu` for use later in `tiled\_prediction`.

L218 for (

Explanation: Loop: iterates over the specified sequence/range and executes the following block for each item.

L219 tile_probabilities,

Explanation: Continues the surrounding multi-line argument, tuple, list, or unpacking expression.

L220 tile_boundary,

Explanation: Continues the surrounding multi-line argument, tuple, list, or unpacking expression.

L221 (top, left),

Explanation: Continues the surrounding multi-line call/container with this argument or value.

L222 (valid_height, valid_width),

Explanation: Continues the surrounding multi-line call/container with this argument or value.

L223) in zip(

Explanation: Closes or continues the surrounding multi-line Python expression.

L224 probabilities_cpu,

Explanation: Continues the surrounding multi-line argument, tuple, list, or unpacking expression.

L225 boundaries_cpu,

Explanation: Continues the surrounding multi-line argument, tuple, list, or unpacking expression.

L226 coordinates,

Explanation: Continues the surrounding multi-line argument, tuple, list, or unpacking expression.

L227 valid_shapes,

Explanation: Continues the surrounding multi-line argument, tuple, list, or unpacking expression.

L228):

Explanation: Closes or continues the surrounding multi-line Python expression.

L229 bottom, right = top + valid_height, left + valid_width

Explanation: Continues the implementation of `tiled\_prediction`; this line is part of the surrounding multi-line expression or statement.

L230 tile_weight = window[:, valid_height, :valid_width]

Explanation: Assigns the result of this expression to `tile\_weight` for use later in `tiled\_prediction`.

L231 probability_sum[:, top:bottom, left:right] += (

Explanation: Continues the implementation of `tiled\_prediction`; this line is part of the surrounding multi-line expression or statement.

L232 tile_probabilities[:, :valid_height, :valid_width]

Explanation: Continues the implementation of `tiled\_prediction`; this line is part of the surrounding multi-line expression or statement.

L233 * tile_weight[None]

Explanation: Continues the implementation of `tiled\_prediction`; this line is part of the surrounding multi-line expression or statement.

L234)

Explanation: Closes the multi-line call/list/tuple/array started above.

L235 boundary_sum[top:bottom, left:right] += (

Explanation: Continues the implementation of `tiled\_prediction`; this line is part of the surrounding multi-line expression or statement.

L236 tile_boundary[:, valid_height, :valid_width] * tile_weight

Explanation: Continues the implementation of `tiled\_prediction`; this line is part of the surrounding multi-line expression or statement.

L237)

Explanation: Closes the multi-line call/list/tuple/array started above.

L238 weight_sum[top:bottom, left:right] += tile_weight

Explanation: Continues the implementation of `tiled\_prediction`; this line is part of the surrounding multi-line expression or statement.

L240 processed_tiles += len(coordinates)

Explanation: Continues the implementation of `tiled\_prediction`; this line is part of the surrounding multi-line expression or statement.

L241 if processed_tiles == total_tiles or processed_tiles % 50 == 0:

Explanation: Conditional branch: evaluates this condition and executes the indented block only when it is true.

L242 print(f"Processed {processed_tiles}/{total_tiles} tiles")

Explanation: Prints runtime progress/diagnostic information to standard output.

L243 del (

Explanation: Drops references early so large tensors/arrays can be reclaimed before later work.

L244 local_device,

Explanation: Continues the surrounding multi-line argument, tuple, list, or unpacking expression.

L245 context_device,

Explanation: Continues the surrounding multi-line argument, tuple, list, or unpacking expression.

L246 outputs,

Explanation: Continues the surrounding multi-line argument, tuple, list, or unpacking expression.

L247 probabilities,

Explanation: Continues the surrounding multi-line argument, tuple, list, or unpacking expression.

L248 boundaries,

Explanation: Continues the surrounding multi-line argument, tuple, list, or unpacking expression.

L249 probabilities_cpu,

Explanation: Continues the surrounding multi-line argument, tuple, list, or unpacking expression.

L250 boundaries_cpu,

Explanation: Continues the surrounding multi-line argument, tuple, list, or unpacking expression.

L251)

Explanation: Closes the multi-line call/list/tuple/array started above.

L253 rows_per_chunk = max(1, min(256, height))

Explanation: Assigns the result of this expression to `rows\_per\_chunk` for use later in `tiled\_prediction`.

L254 for top in range(0, height, rows_per_chunk):

Explanation: Loop: iterates over the specified sequence/range and executes the following block for each item.

L255 bottom = min(top + rows_per_chunk, height)

Explanation: Assigns the result of this expression to `bottom` for use later in `tiled\_prediction`.

L256 weights = weight_sum[top:bottom]

Explanation: Assigns the result of this expression to `weights` for use later in `tiled\_prediction`.

L257 for class_id in range(len(CLASS_NAMES)):

Explanation: Loop: iterates over the specified sequence/range and executes the following block for each item.

L258 block = probability_sum[class_id, top:bottom]

Explanation: Assigns the result of this expression to `block` for use later in `tiled\_prediction`.

L259 np.divide(block, weights, out=block)

Explanation: Executes this numerical/layer operation as part of the surrounding expression.

L260 boundary_block = boundary_sum[top:bottom]

Explanation: Assigns the result of this expression to `boundary\_block` for use later in `tiled\_prediction`.

L261 np.divide(boundary_block, weights, out=boundary_block)

Explanation: Executes this numerical/layer operation as part of the surrounding expression.

L262 probability_sum.flush()

Explanation: Continues the implementation of `tiled\_prediction`; this line is part of the surrounding multi-line expression or statement.

L263 boundary_sum.flush()

Explanation: Continues the implementation of `tiled\_prediction`; this line is part of the surrounding multi-line expression or statement.

L264 return probability_sum, boundary_sum, boundary_path

Explanation: Returns the computed value(s) to the caller.

L265 except Exception:

Explanation: Handles an exception from the preceding `try` block, performing cleanup before re-raising or recovery.

L266 close_memmap(probability_sum)

Explanation: Continues the implementation of `tiled\_prediction`; this line is part of the surrounding multi-line expression or statement.

L267 close_memmap(boundary_sum)

Explanation: Continues the implementation of `tiled\_prediction`; this line is part of the surrounding multi-line expression or statement.

L268 boundary_path.unlink(missing_ok=True)

Explanation: Continues the implementation of `tiled\_prediction`; this line is part of the surrounding multi-line expression or statement.

L269 raise

Explanation: Continues the surrounding multi-line argument, tuple, list, or unpacking expression.

L270 finally:

Explanation: Cleanup block that runs whether the preceding operation succeeds or fails.

L271 close_memmap(weight_sum)

Explanation: Continues the implementation of `tiled\_prediction`; this line is part of the surrounding multi-line expression or statement.

L272 weight_path.unlink(missing_ok=True)

Explanation: Continues the implementation of `tiled\_prediction`; this line is part of the surrounding multi-line expression or statement.

L275 def save_prediction_images(

Explanation: Writes hard labels, a palette color mask, per-class probability PNGs, and boundary probability PNG from disk-backed predictions.

L276 probabilities: np.memmap,

Explanation: Continues the surrounding multi-line call/container with this argument or value.

L277 boundary_probability: np.memmap,

Explanation: Continues the surrounding multi-line call/container with this argument or value.

L278 output_dir: Path,

Explanation: Continues the surrounding multi-line call/container with this argument or value.

L279 rows_per_chunk: int = 256,

Explanation: Assigns the result of this expression to `rows\_per\_chunk` for use later in `save\_prediction\_images`.

L280) -> None:

Explanation: Closes or continues the surrounding multi-line Python expression.

L281 _, height, width = probabilities.shape

Explanation: Continues the implementation of `save\_prediction\_images`; this line is part of the surrounding multi-line expression or statement.

L282 image_buffer_path = temporary_path(output_dir, "image_buffer_", ".dat")

Explanation: Assigns the result of this expression to `image\_buffer\_path` for use later in `save\_prediction\_images`.

L283 image_buffer = np.memmap(

Explanation: Computes NumPy quantity `image\_buffer` used by the surrounding save_prediction_images logic.

L284 image_buffer_path, mode="w+", dtype=np.uint8, shape=(height, width)

Explanation: Continues the implementation of `save\_prediction\_images`; this line is part of the surrounding multi-line expression or statement.

L285)

Explanation: Closes the multi-line call/list/tuple/array started above.

L286 try:

Explanation: Starts exception-safe resource-management block.

L287 for top in range(0, height, rows_per_chunk):

Explanation: Loop: iterates over the specified sequence/range and executes the following block for each item.

L288 bottom = min(top + rows_per_chunk, height)

Explanation: Assigns the result of this expression to `bottom` for use later in `save\_prediction\_images`.

L289 image_buffer[top:bottom] = np.argmax(

Explanation: Continues the implementation of `save\_prediction\_images`; this line is part of the surrounding multi-line expression or statement.

L290 probabilities[:, top:bottom], axis=0

Explanation: Continues the implementation of `save\_prediction\_images`; this line is part of the surrounding multi-line expression or statement.

L291).astype(np.uint8)

Explanation: Closes or continues the surrounding multi-line Python expression.

L292 image_buffer.flush()

Explanation: Continues the implementation of `save\_prediction\_images`; this line is part of the surrounding multi-line expression or statement.

L293 label_image = Image.fromarray(image_buffer)

Explanation: Assigns the result of this expression to `label\_image` for use later in `save\_prediction\_images`.

L294 label_image.save(output_dir / "class_ids.png")

Explanation: Continues the implementation of `save\_prediction\_images`; this line is part of the surrounding multi-line expression or statement.

L295 palette = np.zeros((256, 3), dtype=np.uint8)

Explanation: Computes NumPy quantity `palette` used by the surrounding save_prediction_images logic.

L296 palette[:len(PALETTE)] = PALETTE

Explanation: Continues the implementation of `save\_prediction\_images`; this line is part of the surrounding multi-line expression or statement.

L297 label_image.putpalette(palette.ravel().tolist())

Explanation: Continues the implementation of `save\_prediction\_images`; this line is part of the surrounding multi-line expression or statement.

L298 label_image.save(output_dir / "color_mask.png")

Explanation: Continues the implementation of `save\_prediction\_images`; this line is part of the surrounding multi-line expression or statement.

L299 label_image.close()

Explanation: Continues the implementation of `save\_prediction\_images`; this line is part of the surrounding multi-line expression or statement.

L301 for class_id, name in enumerate(CLASS_NAMES):

Explanation: Loop: iterates over the specified sequence/range and executes the following block for each item.

L302 for top in range(0, height, rows_per_chunk):

Explanation: Loop: iterates over the specified sequence/range and executes the following block for each item.

L303 bottom = min(top + rows_per_chunk, height)

Explanation: Assigns the result of this expression to `bottom` for use later in `save\_prediction\_images`.

L304 image_buffer[top:bottom] = np rint(

Explanation: Continues the implementation of `save\_prediction\_images`; this line is part of the surrounding multi-line expression or statement.

L305 probabilities[class_id, top:bottom] * 255

Explanation: Continues the implementation of `save\_prediction\_images`; this line is part of the surrounding multi-line expression or statement.

L306).astype(np.uint8)

Explanation: Closes or continues the surrounding multi-line Python expression.

L307 image_buffer.flush()

Explanation: Continues the implementation of `save\_prediction\_images`; this line is part of the surrounding multi-line expression or statement.

L308 probability_image = Image.fromarray(image_buffer)

Explanation: Assigns the result of this expression to `probability\_image` for use later in `save\_prediction\_images`.

L309 `probability_image.save(output_dir / f'{class_id}_{name}.png')`

Explanation: Continues the implementation of `save\_prediction\_images`; this line is part of the surrounding multi-line expression or statement.

L310 `probability_image.close()`

Explanation: Continues the implementation of `save\_prediction\_images`; this line is part of the surrounding multi-line expression or statement.

L312 `for top in range(0, height, rows_per_chunk):`

Explanation: Loop: iterates over the specified sequence/range and executes the following block for each item.

L313 `bottom = min(top + rows_per_chunk, height)`

Explanation: Assigns the result of this expression to `bottom` for use later in `save\_prediction\_images`.

L314 `image_buffer[top:bottom] = np rint(`

Explanation: Continues the implementation of `save\_prediction\_images`; this line is part of the surrounding multi-line expression or statement.

L315 `boundary_probability[top:bottom] * 255`

Explanation: Continues the implementation of `save\_prediction\_images`; this line is part of the surrounding multi-line expression or statement.

L316 `).astype(np.uint8)`

Explanation: Closes or continues the surrounding multi-line Python expression.

L317 `image_buffer.flush()`

Explanation: Continues the implementation of `save\_prediction\_images`; this line is part of the surrounding multi-line expression or statement.

L318 `boundary_image = Image.fromarray(image_buffer)`

Explanation: Assigns the result of this expression to `boundary\_image` for use later in `save\_prediction\_images`.

L319 `boundary_image.save(output_dir / "bone_fibro_boundary.png")`

Explanation: Continues the implementation of `save\_prediction\_images`; this line is part of the surrounding multi-line expression or statement.

L320 `boundary_image.close()`

Explanation: Continues the implementation of `save\_prediction\_images`; this line is part of the surrounding multi-line expression or statement.

L321 `finally:`

Explanation: Cleanup block that runs whether the preceding operation succeeds or fails.

L322 `close_memmap(image_buffer)`

Explanation: Continues the implementation of `save\_prediction\_images`; this line is part of the surrounding multi-line expression or statement.

L323 `image_buffer_path.unlink(missing_ok=True)`

Explanation: Continues the implementation of `save\_prediction\_images`; this line is part of the surrounding multi-line expression or statement.

L326 `def parse_args() -> argparse.Namespace:`

Explanation: Defines and validates the prediction command-line interface.

L327 `parser = argparse.ArgumentParser()`

Explanation: Assigns the result of this expression to `parser` for use later in `parse\_args`.

L328 `parser.add_argument("--image", type=Path, required=True)`

Explanation: Defines command-line option `--image` and its parsing/default behavior.

L329 `parser.add_argument("--checkpoint", type=Path, required=True)`

Explanation: Defines command-line option `--checkpoint` and its parsing/default behavior.

L330 `parser.add_argument("--output-dir", type=Path, default=Path("prediction"))`

Explanation: Defines command-line option `--output-dir` and its parsing/default behavior.

L331 `parser.add_argument("--device", choices=("auto", "cpu", "cuda"), default="auto")`

Explanation: Defines command-line option `--device` and its parsing/default behavior.

L332 `parser.add_argument("--tile-size", type=int, default=512)`

Explanation: Defines command-line option `--tile-size` and its parsing/default behavior.

L333 `parser.add_argument("--overlap", type=int, default=128)`

Explanation: Defines command-line option `--overlap` and its parsing/default behavior.

L334 `parser.add_argument("--batch-size", type=int, default=2)`

Explanation: Defines command-line option `--batch-size` and its parsing/default behavior.

L335 `parser.add_argument("--no-save-probabilities", action="store_true")`

Explanation: Defines command-line option `--no-save-probabilities` and its parsing/default behavior.

L336 `parser.add_argument("--no-amp", action="store_true")`

Explanation: Defines command-line option `--no-amp` and its parsing/default behavior.

L337 `args = parser.parse_args()`

Explanation: Assigns the result of this expression to `args` for use later in `parse\_args`.

L338 `if args.tile_size <= 0 or args.tile_size % 32:`

Explanation: Conditional branch: evaluates this condition and executes the indented block only when it is true.

L339 `parser.error("--tile-size must be a positive multiple of 32")`

Explanation: Terminates argument parsing with a user-facing validation error.

L340 `if args.overlap < 0 or args.overlap >= args.tile_size:`

Explanation: Conditional branch: evaluates this condition and executes the indented block only when it is true.

L341 `parser.error("--overlap must be at least 0 and smaller than --tile-size")`

Explanation: Terminates argument parsing with a user-facing validation error.

L342 `if args.batch_size <= 0:`

Explanation: Conditional branch: evaluates this condition and executes the indented block only when it is true.

L343 `parser.error("--batch-size must be positive")`

Explanation: Terminates argument parsing with a user-facing validation error.

L344 `return args`

Explanation: Returns the computed value(s) to the caller.

L347 `def main() -> None:`

Explanation: Loads image/checkpoint, reconstructs the model, performs tiled prediction, writes outputs, and cleans temporary files.

L348 `args = parse_args()`

Explanation: Assigns the result of this expression to `args` for use later in `main`.

L349 `if args.device == "cuda" and not torch.cuda.is_available():`

Explanation: Conditional branch: evaluates this condition and executes the indented block only when it is true.

L350 `raise RuntimeError(`

Explanation: Stops execution with an explicit error because an input/state invariant is violated.

L351 `"CUDA was requested but is unavailable. Install a CUDA-enabled "`

Explanation: Adds this string literal to the surrounding tuple/list/message or multi-line string.

L352 `"PyTorch build and an NVIDIA driver."`

Explanation: Adds this string literal to the surrounding tuple/list/message or multi-line string.

L353 `)`

Explanation: Closes the multi-line call/list/tuple/array started above.

L354 `device = torch.device(`

Explanation: Computes tensor quantity `device` using PyTorch operations in the surrounding main logic.

L355 `"cuda"`

Explanation: Adds this string literal to the surrounding tuple/list/message or multi-line string.

L356 `if args.device == "cuda"`

Explanation: Conditional branch: evaluates this condition and executes the indented block only when it is true.

L357 `or (args.device == "auto" and torch.cuda.is_available())`

Explanation: Continues the implementation of `main`; this line is part of the surrounding multi-line expression or statement.

L358 `else "cpu"`

Explanation: Continues the implementation of `main`; this line is part of the surrounding multi-line expression or statement.

L359 `)`

Explanation: Closes the multi-line call/list/tuple/array started above.

L360 `print(f"Using device: {device}")`

Explanation: Prints runtime progress/diagnostic information to standard output.

L361 `if device.type == "cuda":`

Explanation: Conditional branch: evaluates this condition and executes the indented block only when it is true.

L362 `torch.backends.cudnn.benchmark = True`

Explanation: Assigns the result of this expression to `torch.backends.cudnn.benchmark` for use later in `main`.

L364 `args.output_dir.mkdir(parents=True, exist_ok=True)`

Explanation: Continues the implementation of `main`; this line is part of the surrounding multi-line expression or statement.

L365 `with Image.open(args.image) as original:`

Explanation: Context manager ensuring the opened resource/state is cleaned up correctly after the block.

L366 `image = np.array(original.convert("RGB"), dtype=np.uint8, copy=True)`

Explanation: Converts image/data into an independent NumPy array stored as `image`.

L368 `checkpoint = torch.load(`

Explanation: Computes tensor quantity `checkpoint` using PyTorch operations in the surrounding main logic.

L369 `args.checkpoint, map_location="cpu", weights_only=False`

Explanation: Continues the implementation of `main`; this line is part of the surrounding multi-line expression or statement.

L370 `)`

Explanation: Closes the multi-line call/list/tuple/array started above.

L371 `if "model_config" not in checkpoint:`

Explanation: Conditional branch: evaluates this condition and executes the indented block only when it is true.

L372 `raise RuntimeError(`

Explanation: Stops execution with an explicit error because an input/state invariant is violated.

L373 `"This is a legacy TinyFeatureUNet checkpoint. Retrain with the "`

Explanation: Adds this string literal to the surrounding tuple/list/message or multi-line string.

L374 `"updated train.py before using dual-scale prediction."`

Explanation: Adds this string literal to the surrounding tuple/list/message or multi-line string.

L375 `)`

Explanation: Closes the multi-line call/list/tuple/array started above.

L376 `model = AccurateTissueNet(**checkpoint["model_config"])`

Explanation: Assigns the result of this expression to `model` for use later in `main`.

L377 `model.load_state_dict(checkpoint["model"])`

Explanation: Continues the implementation of `main`; this line is part of the surrounding multi-line expression or statement.

L378 `del checkpoint`

Explanation: Drops references early so large tensors/arrays can be reclaimed before later work.

L379 `if device.type == "cuda":`

Explanation: Conditional branch: evaluates this condition and executes the indented block only when it is true.

L380 `model = model.to(memory_format=torch.channels_last)`

Explanation: Computes tensor quantity `model` using PyTorch operations in the surrounding main logic.

L381 `model.to(device).eval()`

Explanation: Continues the implementation of `main`; this line is part of the surrounding multi-line expression or statement.

L382 `context_scale = model.context_scale`

Explanation: Assigns the result of this expression to `context\_scale` for use later in `main`.

L384 `probability_path = (`

Explanation: Assigns the result of this expression to `probability\_path` for use later in `main`.

L385 `temporary_path(args.output_dir, "probabilities_", ".npz")`

Explanation: Continues the implementation of `main`; this line is part of the surrounding multi-line expression or statement.

L386 `if args.no_save_probabilities`

Explanation: Conditional branch: evaluates this condition and executes the indented block only when it is true.

L387 `else args.output_dir / "probabilities.npz"`

Explanation: Continues the implementation of `main`; this line is part of the surrounding multi-line expression or statement.

L388 `)`

Explanation: Closes the multi-line call/list/tuple/array started above.

L389 `probabilities, boundary_probability, boundary_path = tiled_prediction(`

Explanation: Continues the implementation of `main`; this line is part of the surrounding multi-line expression or statement.

L390 `model,`

Explanation: Continues the surrounding multi-line argument, tuple, list, or unpacking expression.

L391 `image,`

Explanation: Continues the surrounding multi-line argument, tuple, list, or unpacking expression.

L392 device,
Explanation: Continues the surrounding multi-line argument, tuple, list, or unpacking expression.

L393 tile_size=args.tile_size,
Explanation: Assigns the result of this expression to `tile\_size` for use later in `main`.

L394 overlap=args.overlap,
Explanation: Assigns the result of this expression to `overlap` for use later in `main`.

L395 batch_size=args.batch_size,
Explanation: Assigns the result of this expression to `batch\_size` for use later in `main`.

L396 context_scale=context_scale,
Explanation: Assigns the result of this expression to `context\_scale` for use later in `main`.

L397 probability_path=probability_path,
Explanation: Assigns the result of this expression to `probability\_path` for use later in `main`.

L398 use_amp=not args.no_amp,
Explanation: Assigns the result of this expression to `use\_amp` for use later in `main`.

L399)
Explanation: Closes the multi-line call/list/tuple/array started above.

L400 del image, model
Explanation: Drops references early so large tensors/arrays can be reclaimed before later work.

L401 if device.type == "cuda":
Explanation: Conditional branch: evaluates this condition and executes the indented block only when it is true.

L402 torch.cuda.empty_cache()
Explanation: Executes this numerical/layer operation as part of the surrounding expression.

L404 try:
Explanation: Starts exception-safe resource-management block.

L405 save_prediction_images(
Explanation: Continues the implementation of `main`; this line is part of the surrounding multi-line expression or statement.

L406 probabilities,
Explanation: Continues the surrounding multi-line argument, tuple, list, or unpacking expression.

L407 boundary_probability,
Explanation: Continues the surrounding multi-line argument, tuple, list, or unpacking expression.

L408 args.output_dir,
Explanation: Continues the surrounding multi-line call/container with this argument or value.

L409)
Explanation: Closes the multi-line call/list/tuple/array started above.

L410 (args.output_dir / "classes.txt").write_text(
Explanation: Continues the implementation of `main`; this line is part of the surrounding multi-line expression or statement.

L411 "\n".join(
Explanation: Adds this string literal to the surrounding tuple/list/message or multi-line string.

L412 f"{class_id}: {name}"
Explanation: Continues the implementation of `main`; this line is part of the surrounding multi-line expression or statement.

L413 for class_id, name in enumerate(CLASS_NAMES)
Explanation: Loop: iterates over the specified sequence/range and executes the following block for each item.

L414),
Explanation: Closes or continues the surrounding multi-line Python expression.

L415 encoding="utf-8",
Explanation: Assigns the result of this expression to `encoding` for use later in `main`.

L416)
Explanation: Closes the multi-line call/list/tuple/array started above.

L417 finally:
Explanation: Cleanup block that runs whether the preceding operation succeeds or fails.

L418 close_memmap(probabilities)
Explanation: Continues the implementation of `main`; this line is part of the surrounding multi-line expression or statement.

L419 close_memmap(boundary_probability)

Explanation: Continues the implementation of `main`; this line is part of the surrounding multi-line expression or statement.

L420 boundary_path.unlink(missing_ok=True)

Explanation: Continues the implementation of `main`; this line is part of the surrounding multi-line expression or statement.

L421 if args.no_save_probabilities:

Explanation: Conditional branch: evaluates this condition and executes the indented block only when it is true.

L422 probability_path.unlink(missing_ok=True)

Explanation: Continues the implementation of `main`; this line is part of the surrounding multi-line expression or statement.

L423 print(f"Saved predictions to {args.output_dir}")

Explanation: Prints runtime progress/diagnostic information to standard output.

L426 if __name__ == "__main__":

Explanation: Python entry-point guard: the indented call runs only when this file is executed as a script.

L427 main()

Explanation: Continues the implementation of `predict.py`; this line is part of the surrounding multi-line expression or statement.

quantify.py

Major definitions: main

L001 from __future__ import annotations

Explanation: Postpones evaluation of type annotations, reducing runtime coupling and allowing forward references.

L003 import argparse

Explanation: Imports dependencies used by this module: command-line argument parsing

L004 import csv

Explanation: Imports dependencies used by this module: CSV output

L005 from pathlib import Path

Explanation: Imports dependencies used by this module: filesystem paths

L007 import numpy as np

Explanation: Imports dependencies used by this module: array and numerical operations

L008 from PIL import Image

Explanation: Imports dependencies used by this module: image I/O and resizing

L010 from model import CLASS_NAMES

Explanation: Imports dependencies used by this module.

L013 Image.MAX_IMAGE_PIXELS = None

Explanation: Assigns the result of this expression to 'Image.MAX_IMAGE_PIXELS' for use later in 'quantify.py'.

L016 def main() -> None:

Explanation: Reads hard labels and optional probability maps, converts pixel counts to calibrated physical area, and writes a CSV summary.

L017 parser = argparse.ArgumentParser(

Explanation: Assigns the result of this expression to 'parser' for use later in 'main'.

L018 description="Calculate hard and probability-weighted tissue areas."

Explanation: Assigns the result of this expression to 'description' for use later in 'main'.

L019)

Explanation: Closes the multi-line call/list/tuple/array started above.

L020 parser.add_argument("--prediction-dir", type=Path, required=True)

Explanation: Defines command-line option '--prediction-dir' and its parsing/default behavior.

L021 parser.add_argument("--pixel-width-um", type=float, required=True)

Explanation: Defines command-line option '--pixel-width-um' and its parsing/default behavior.

L022 parser.add_argument("--pixel-height-um", type=float, required=True)

Explanation: Defines command-line option '--pixel-height-um' and its parsing/default behavior.

L023 parser.add_argument("--output", type=Path)

Explanation: Defines command-line option '--output' and its parsing/default behavior.

L024 args = parser.parse_args()

Explanation: Assigns the result of this expression to 'args' for use later in 'main'.

L025 if args.pixel_width_um <= 0 or args.pixel_height_um <= 0:

Explanation: Conditional branch: evaluates this condition and executes the indented block only when it is true.

L026 parser.error("Pixel dimensions must be positive")

Explanation: Terminates argument parsing with a user-facing validation error.

L028 label_path = args.prediction_dir / "class_ids.png"

Explanation: Assigns the result of this expression to 'label_path' for use later in 'main'.

L029 with Image.open(label_path) as label_image:

Explanation: Context manager ensuring the opened resource/state is cleaned up correctly after the block.

L030 labels = np.array(label_image.convert("L"), dtype=np.uint8, copy=True)

Explanation: Converts image/data into an independent NumPy array stored as 'labels'.

L031 height, width = labels.shape

Explanation: Continues the implementation of 'main'; this line is part of the surrounding multi-line expression or statement.

L032 hard_pixels = np.bincount(

Explanation: Computes NumPy quantity 'hard_pixels' used by the surrounding main logic.

L033 labels.reshape(-1), minlength=len(CLASS_NAMES)

Explanation: Continues the implementation of `main`; this line is part of the surrounding multi-line expression or statement.

L034).astype(np.float64)

Explanation: Closes or continues the surrounding multi-line Python expression.

L036 probability_path = args.prediction_dir / "probabilities.npy"

Explanation: Assigns the result of this expression to `probability\_path` for use later in `main`.

L037 expected_pixels = np.full(len(CLASS_NAMES), np.nan, dtype=np.float64)

Explanation: Computes NumPy quantity `expected\_pixels` used by the surrounding main logic.

L038 if probability_path.exists():

Explanation: Conditional branch: evaluates this condition and executes the indented block only when it is true.

L039 probabilities = np.load(probability_path, mmap_mode="r")

Explanation: Assigns `probabilities` as the six-class per-pixel probability tensor.

L040 if probabilities.shape != (len(CLASS_NAMES), height, width):

Explanation: Conditional branch: evaluates this condition and executes the indented block only when it is true.

L041 raise ValueError(

Explanation: Stops execution with an explicit error because an input/state invariant is violated.

L042 f"Probability shape {probabilities.shape} does not match "

Explanation: Continues the implementation of `main`; this line is part of the surrounding multi-line expression or statement.

L043 f"label dimensions {(height, width)}")

Explanation: Continues the implementation of `main`; this line is part of the surrounding multi-line expression or statement.

L044)

Explanation: Closes the multi-line call/list/tuple/array started above.

L045 expected_pixels[:] = 0

Explanation: Continues the implementation of `main`; this line is part of the surrounding multi-line expression or statement.

L046 for top in range(0, height, 256):

Explanation: Loop: iterates over the specified sequence/range and executes the following block for each item.

L047 expected_pixels += probabilities[

Explanation: Continues the implementation of `main`; this line is part of the surrounding multi-line expression or statement.

L048 :, top : min(top + 256, height)

Explanation: Continues the implementation of `main`; this line is part of the surrounding multi-line expression or statement.

L049].sum(axis=(1, 2), dtype=np.float64)

Explanation: Closes or continues the surrounding multi-line Python expression.

L051 pixel_area_mm2 = (

Explanation: Assigns the result of this expression to `pixel\_area\_mm2` for use later in `main`.

L052 args.pixel_width_um * args.pixel_height_um / 1_000_000.0

Explanation: Continues the implementation of `main`; this line is part of the surrounding multi-line expression or statement.

L053)

Explanation: Closes the multi-line call/list/tuple/array started above.

L054 hard_tissue_pixels = hard_pixels[:-1].sum()

Explanation: Assigns the result of this expression to `hard\_tissue\_pixels` for use later in `main`.

L055 expected_tissue_pixels = np.nansum(expected_pixels[:-1])

Explanation: Computes NumPy quantity `expected\_tissue\_pixels` used by the surrounding main logic.

L056 output = args.output or args.prediction_dir / "area_summary.csv"

Explanation: Assigns the result of this expression to `output` for use later in `main`.

L057 output.parent.mkdir(parents=True, exist_ok=True)

Explanation: Continues the implementation of `main`; this line is part of the surrounding multi-line expression or statement.

L058 with output.open("w", newline="", encoding="utf-8") as output_file:

Explanation: Context manager ensuring the opened resource/state is cleaned up correctly after the block.

L059 writer = csv.writer(output_file)

Explanation: Assigns the result of this expression to `writer` for use later in `main`.

L060 writer.writerow(

Explanation: Continues the implementation of `main`; this line is part of the surrounding multi-line expression or statement.

L061 [

Explanation: Continues the implementation of `main`; this line is part of the surrounding multi-line expression or statement.

L062 "class_id",
Explanation: Adds this string literal to the surrounding tuple/list/message or multi-line string.

L063 "class_name",
Explanation: Adds this string literal to the surrounding tuple/list/message or multi-line string.

L064 "hard_pixels",
Explanation: Adds this string literal to the surrounding tuple/list/message or multi-line string.

L065 "hard_area_mm2",
Explanation: Adds this string literal to the surrounding tuple/list/message or multi-line string.

L066 "hard_percent_non_background",
Explanation: Adds this string literal to the surrounding tuple/list/message or multi-line string.

L067 "expected_pixels",
Explanation: Adds this string literal to the surrounding tuple/list/message or multi-line string.

L068 "expected_area_mm2",
Explanation: Adds this string literal to the surrounding tuple/list/message or multi-line string.

L069 "expected_percent_non_background",
Explanation: Adds this string literal to the surrounding tuple/list/message or multi-line string.

L070]
Explanation: Closes the multi-line container started above.

L071)
Explanation: Closes the multi-line call/list/tuple/array started above.

L072 for class_id, name in enumerate(CLASS_NAMES):
Explanation: Loop: iterates over the specified sequence/range and executes the following block for each item.

L073 hard_percent = (
Explanation: Assigns the result of this expression to `hard\_percent` for use later in `main`.

L074 100 * hard_pixels[class_id] / hard_tissue_pixels
Explanation: Continues the implementation of `main`; this line is part of the surrounding multi-line expression or statement.

L075 if class_id != len(CLASS_NAMES) - 1 and hard_tissue_pixels
Explanation: Conditional branch: evaluates this condition and executes the indented block only when it is true.

L076 else np.nan
Explanation: Continues the implementation of `main`; this line is part of the surrounding multi-line expression or statement.

L077)
Explanation: Closes the multi-line call/list/tuple/array started above.

L078 expected_percent = (
Explanation: Assigns the result of this expression to `expected\_percent` for use later in `main`.

L079 100 * expected_pixels[class_id] / expected_tissue_pixels
Explanation: Continues the implementation of `main`; this line is part of the surrounding multi-line expression or statement.

L080 if class_id != len(CLASS_NAMES) - 1
Explanation: Conditional branch: evaluates this condition and executes the indented block only when it is true.

L081 and expected_tissue_pixels
Explanation: Continues the implementation of `main`; this line is part of the surrounding multi-line expression or statement.

L082 and not np.isnan(expected_pixels[class_id])
Explanation: Continues the implementation of `main`; this line is part of the surrounding multi-line expression or statement.

L083 else np.nan
Explanation: Continues the implementation of `main`; this line is part of the surrounding multi-line expression or statement.

L084)
Explanation: Closes the multi-line call/list/tuple/array started above.

L085 writer.writerow(
Explanation: Continues the implementation of `main`; this line is part of the surrounding multi-line expression or statement.

L086 [
Explanation: Continues the implementation of `main`; this line is part of the surrounding multi-line expression or statement.

L087 class_id,
Explanation: Continues the surrounding multi-line argument, tuple, list, or unpacking expression.

L088 name,

Explanation: Continues the surrounding multi-line argument, tuple, list, or unpacking expression.

L089 `f'{hard_pixels[class_id]:.0f}',`

Explanation: Continues the surrounding multi-line call/container with this argument or value.

L090 `f'{hard_pixels[class_id] * pixel_area_mm2:.9f}',`

Explanation: Continues the surrounding multi-line call/container with this argument or value.

L091 `f'{hard_percent:.6f}',`

Explanation: Continues the surrounding multi-line call/container with this argument or value.

L092 `(`

Explanation: Continues the implementation of `main`; this line is part of the surrounding multi-line expression or statement.

L093 `f'{expected_pixels[class_id]:.6f}'`

Explanation: Continues the implementation of `main`; this line is part of the surrounding multi-line expression or statement.

L094 `if not np.isnan(expected_pixels[class_id])`

Explanation: Conditional branch: evaluates this condition and executes the indented block only when it is true.

L095 `else ""`

Explanation: Continues the implementation of `main`; this line is part of the surrounding multi-line expression or statement.

L096 `),`

Explanation: Closes or continues the surrounding multi-line Python expression.

L097 `(`

Explanation: Continues the implementation of `main`; this line is part of the surrounding multi-line expression or statement.

L098 `f'{expected_pixels[class_id] * pixel_area_mm2:.9f}'`

Explanation: Continues the implementation of `main`; this line is part of the surrounding multi-line expression or statement.

L099 `if not np.isnan(expected_pixels[class_id])`

Explanation: Conditional branch: evaluates this condition and executes the indented block only when it is true.

L100 `else ""`

Explanation: Continues the implementation of `main`; this line is part of the surrounding multi-line expression or statement.

L101 `),`

Explanation: Closes or continues the surrounding multi-line Python expression.

L102 `(`

Explanation: Continues the implementation of `main`; this line is part of the surrounding multi-line expression or statement.

L103 `f'{expected_percent:.6f}'`

Explanation: Continues the implementation of `main`; this line is part of the surrounding multi-line expression or statement.

L104 `if not np.isnan(expected_percent)`

Explanation: Conditional branch: evaluates this condition and executes the indented block only when it is true.

L105 `else ""`

Explanation: Continues the implementation of `main`; this line is part of the surrounding multi-line expression or statement.

L106 `),`

Explanation: Closes or continues the surrounding multi-line Python expression.

L107 `]`

Explanation: Closes the multi-line container started above.

L108 `)`

Explanation: Closes the multi-line call/list/tuple/array started above.

L109 `print(`

Explanation: Prints runtime progress/diagnostic information to standard output.

L110 `f'Saved {output}; image={width}x{height}, "`

Explanation: Continues the implementation of `main`; this line is part of the surrounding multi-line expression or statement.

L111 `f'pixel_area={pixel_area_mm2:.9g} mm^2"`

Explanation: Continues the implementation of `main`; this line is part of the surrounding multi-line expression or statement.

L112 `)`

Explanation: Closes the multi-line call/list/tuple/array started above.

L115 `if __name__ == "__main__":`

Explanation: Python entry-point guard: the indented call runs only when this file is executed as a script.

L116 `main()`

Explanation: Continues the implementation of `quantify.py`; this line is part of the surrounding multi-line expression or statement.

smoke_test.py

Major definitions: main

L001 from __future__ import annotations

Explanation: Postpones evaluation of type annotations, reducing runtime coupling and allowing forward references.

L003 import argparse

Explanation: Imports dependencies used by this module: command-line argument parsing

L005 import torch

Explanation: Imports dependencies used by this module: tensor computation and automatic differentiation

L007 from model import AccurateTissueNet

Explanation: Imports dependencies used by this module.

L008 from train import complete_loss

Explanation: Imports dependencies used by this module.

L011 def main() -> None:

Explanation: Constructs a small random batch and verifies forward loss, six-class probability normalization, and backward gradient finiteness.

L012 parser = argparse.ArgumentParser()

Explanation: Assigns the result of this expression to `parser` for use later in `main`.

L013 parser.add_argument("--device", choices=("auto", "cpu", "cuda"), default="auto")

Explanation: Defines command-line option `--device` and its parsing/default behavior.

L014 parser.add_argument("--size", type=int, default=64)

Explanation: Defines command-line option `--size` and its parsing/default behavior.

L015 parser.add_argument("--forward-only", action="store_true")

Explanation: Defines command-line option `--forward-only` and its parsing/default behavior.

L016 args = parser.parse_args()

Explanation: Assigns the result of this expression to `args` for use later in `main`.

L017 if args.size < 64 or args.size % 32:

Explanation: Conditional branch: evaluates this condition and executes the indented block only when it is true.

L018 parser.error("--size must be a multiple of 32 and at least 64")

Explanation: Terminates argument parsing with a user-facing validation error.

L019 if args.device == "cuda" and not torch.cuda.is_available():

Explanation: Conditional branch: evaluates this condition and executes the indented block only when it is true.

L020 raise RuntimeError("CUDA was requested but torch.cuda.is_available() is false")

Explanation: Stops execution with an explicit error because an input/state invariant is violated.

L021 device = torch.device(

Explanation: Computes tensor quantity `device` using PyTorch operations in the surrounding main logic.

L022 "cuda"

Explanation: Adds this string literal to the surrounding tuple/list/message or multi-line string.

L023 if args.device == "cuda"

Explanation: Conditional branch: evaluates this condition and executes the indented block only when it is true.

L024 or (args.device == "auto" and torch.cuda.is_available())

Explanation: Continues the implementation of `main`; this line is part of the surrounding multi-line expression or statement.

L025 else "cpu"

Explanation: Continues the implementation of `main`; this line is part of the surrounding multi-line expression or statement.

L026)

Explanation: Closes the multi-line call/list/tuple/array started above.

L027 amp = device.type == "cuda"

Explanation: Enables autocast whenever the selected device is CUDA. There is no smoke-test command-line option to disable AMP.

L028 torch.manual_seed(7)

Explanation: Executes this numerical/layer operation as part of the surrounding expression.

L030 model = AccurateTissueNet(pretrained=False)

Explanation: Assigns the result of this expression to `model` for use later in `main`.

L031 if device.type == "cuda":

Explanation: Conditional branch: evaluates this condition and executes the indented block only when it is true.

L032 model = model.to(memory_format=torch.channels_last)

Explanation: Computes tensor quantity `model` using PyTorch operations in the surrounding main logic.

L033 model.to(device).train()

Explanation: Continues the implementation of `main`; this line is part of the surrounding multi-line expression or statement.

L035 local = torch.rand(1, 3, args.size, args.size, device=device)

Explanation: Creates random tensor data for `local` used by the smoke test.

L036 context = torch.rand_like(local)

Explanation: Creates random tensor data for `context` used by the smoke test.

L037 if device.type == "cuda":

Explanation: Conditional branch: evaluates this condition and executes the indented block only when it is true.

L038 local = local.contiguous(memory_format=torch.channels_last)

Explanation: Computes tensor quantity `local` using PyTorch operations in the surrounding main logic.

L039 context = context.contiguous(memory_format=torch.channels_last)

Explanation: Computes tensor quantity `context` using PyTorch operations in the surrounding main logic.

L040 mask = torch.randint(0, 6, (1, args.size, args.size), device=device)

Explanation: Creates random tensor data for `mask` used by the smoke test.

L041 boundary = torch.zeros(1, args.size, args.size, device=device)

Explanation: Computes tensor quantity `boundary` using PyTorch operations in the surrounding main logic.

L042 boundary[:, :, args.size // 2 - 1 : args.size // 2 + 2] = 1

Explanation: Continues the implementation of `main`; this line is part of the surrounding multi-line expression or statement.

L043 muscle_presence = torch.ones(1, device=device)

Explanation: Computes tensor quantity `muscle\_presence` using PyTorch operations in the surrounding main logic.

L045 with torch.autocast(

Explanation: Starts an autocast region; on CUDA, eligible operations in the model may execute in float16.

L046 device_type=device.type,

Explanation: Assigns the result of this expression to `device\_type` for use later in `main`.

L047 dtype=torch.float16,

Explanation: Computes tensor quantity `dtype` using PyTorch operations in the surrounding main logic.

L048 enabled=amp,

Explanation: Assigns the result of this expression to `enabled` for use later in `main`.

L049):

Explanation: Closes or continues the surrounding multi-line Python expression.

L050 outputs = model(local, context)

Explanation: Assigns the result of this expression to `outputs` for use later in `main`.

L051 loss, components, probabilities = complete_loss(

Explanation: Continues the implementation of `main`; this line is part of the surrounding multi-line expression or statement.

L052 model,

Explanation: Continues the surrounding multi-line argument, tuple, list, or unpacking expression.

L053 outputs,

Explanation: Continues the surrounding multi-line argument, tuple, list, or unpacking expression.

L054 mask,

Explanation: Continues the surrounding multi-line argument, tuple, list, or unpacking expression.

L055 boundary,

Explanation: Continues the surrounding multi-line argument, tuple, list, or unpacking expression.

L056 muscle_presence,

Explanation: Continues the surrounding multi-line argument, tuple, list, or unpacking expression.

L057 class_weights=None,

Explanation: Assigns the result of this expression to `class\_weights` for use later in `main`.

L058)

Explanation: Closes the multi-line call/list/tuple/array started above.

L060 assert probabilities.shape == (1, 6, args.size, args.size)

Explanation: Runtime sanity check; raises `AssertionError` when the asserted invariant is false.

L061 `assert torch.isfinite(probabilities).all()`

Explanation: Runtime sanity check; raises `AssertionError` when the asserted invariant is false.

L062 `assert torch.isfinite(loss)`

Explanation: Runtime sanity check; raises `AssertionError` when the asserted invariant is false.

L063 `maximum_sum_error = (`

Explanation: Begins the normalization-invariant check by summing the six probabilities at each pixel.

L064 `probabilities.sum(1).sub(1).abs().max().detach().item()`

Explanation: Subtracts the ideal sum of 1, takes absolute error, then the worst pixel. Because probabilities were built under autocast, this can reflect float16 rounding even though `complete_loss` later casts the tensor to float32.

L065 `)`

Explanation: Closes the multi-line call/list/tuple/array started above.

L066 `assert maximum_sum_error < 1e-4`

Explanation: Requires the maximum sum error to be below 1e-4. This is appropriate for float32 but stricter than ordinary float16 resolution near 1 (about 9.77e-4), which explains the CUDA/AMP failure.

L068 `if not args.forward_only:`

Explanation: Conditional branch: evaluates this condition and executes the indented block only when it is true.

L069 `loss.backward()`

Explanation: Continues the implementation of `main`; this line is part of the surrounding multi-line expression or statement.

L070 `finite_gradient = any(`

Explanation: Assigns the result of this expression to `finite\_gradient` for use later in `main`.

L071 `parameter.grad is not None`

Explanation: Continues the implementation of `main`; this line is part of the surrounding multi-line expression or statement.

L072 `and torch.isfinite(parameter.grad).all()`

Explanation: Continues the implementation of `main`; this line is part of the surrounding multi-line expression or statement.

L073 `for parameter in model.parameters()`

Explanation: Loop: iterates over the specified sequence/range and executes the following block for each item.

L074 `)`

Explanation: Closes the multi-line call/list/tuple/array started above.

L075 `assert finite_gradient`

Explanation: Runtime sanity check; raises `AssertionError` when the asserted invariant is false.

L077 `parameters = sum(parameter.numel() for parameter in model.parameters())`

Explanation: Assigns the result of this expression to `parameters` for use later in `main`.

L078 `print(`

Explanation: Prints runtime progress/diagnostic information to standard output.

L079 `f"PASS device={device} torch={torch.__version__}"`

Explanation: Continues the implementation of `main`; this line is part of the surrounding multi-line expression or statement.

L080 `f"parameters={parameters:,} loss={loss.item():.5f}"`

Explanation: Continues the implementation of `main`; this line is part of the surrounding multi-line expression or statement.

L081 `f"probability_sum_max_error={maximum_sum_error:.3g}"`

Explanation: Continues the implementation of `main`; this line is part of the surrounding multi-line expression or statement.

L082 `)`

Explanation: Closes the multi-line call/list/tuple/array started above.

L083 `print(`

Explanation: Prints runtime progress/diagnostic information to standard output.

L084 `"loss_components "`

Explanation: Adds this string literal to the surrounding tuple/list/message or multi-line string.

L085 `+ " ".join(f"{name}={value:.5f}" for name, value in components.items())`

Explanation: Continues the implementation of `main`; this line is part of the surrounding multi-line expression or statement.

L086 `)`

Explanation: Closes the multi-line call/list/tuple/array started above.

L087 `if device.type == "cuda":`

Explanation: Conditional branch: evaluates this condition and executes the indented block only when it is true.

L088 `print(`

Explanation: Prints runtime progress/diagnostic information to standard output.

L089 f"gpu={torch.cuda.get_device_name(0)} "

Explanation: Continues the implementation of `main`; this line is part of the surrounding multi-line expression or statement.

L090 f"peak_allocated_mb={torch.cuda.max_memory_allocated() / 2**20:.1f}"

Explanation: Continues the implementation of `main`; this line is part of the surrounding multi-line expression or statement.

L091)

Explanation: Closes the multi-line call/list/tuple/array started above.

L094 if __name__ == "__main__":

Explanation: Python entry-point guard: the indented call runs only when this file is executed as a script.

L095 main()

Explanation: Continues the implementation of `smoke\_test.py`; this line is part of the surrounding multi-line expression or statement.

slurm_train.sbatch

L001 #!/bin/bash

Explanation: Shebang selecting Bash when this SLURM script is executed directly.

L002 #SBATCH --job-name=tissue-train

Explanation: SLURM scheduler directive: requests/configures `--job-name=tissue-train` for the submitted job.

L003 #SBATCH --partition=gpu

Explanation: SLURM scheduler directive: requests/configures `--partition=gpu` for the submitted job.

L004 #SBATCH --gres=gpu:1

Explanation: SLURM scheduler directive: requests/configures `--gres=gpu:1` for the submitted job.

L005 #SBATCH --cpus-per-task=8

Explanation: SLURM scheduler directive: requests/configures `--cpus-per-task=8` for the submitted job.

L006 #SBATCH --mem=20G

Explanation: SLURM scheduler directive: requests/configures `--mem=20G` for the submitted job.

L007 #SBATCH --time=2-00:00:00

Explanation: SLURM scheduler directive: requests/configures `--time=2-00:00:00` for the submitted job.

L008 #SBATCH --requeue

Explanation: SLURM scheduler directive: requests/configures `--requeue` for the submitted job.

L009 #SBATCH --output=slurm-%x-%j.out

Explanation: SLURM scheduler directive: requests/configures `--output=slurm-%x-%j.out` for the submitted job.

L011 set -euo pipefail

Explanation: Continues the implementation of `slurm\_train.sbatch`; this line is part of the surrounding multi-line expression or statement.

L013 : "\${DATA:?Set DATA to the dataset directory containing images/ and masks/}"

Explanation: Bash parameter check: aborts with the embedded message if this required environment variable is unset/empty.

L015 VENV="\${VENV:-.venv}"

Explanation: Assigns the result of this expression to `VENV` for use later in `slurm\_train.sbatch`.

L016 OUTPUT="\${OUTPUT:-best_model.pt}"

Explanation: Assigns the result of this expression to `OUTPUT` for use later in `slurm\_train.sbatch`.

L017 LAST_OUTPUT="\${LAST_OUTPUT:-last_model.pt}"

Explanation: Assigns the result of this expression to `LAST\_OUTPUT` for use later in `slurm\_train.sbatch`.

L018 EPOCHS="\${EPOCHS:-80}"

Explanation: Assigns the result of this expression to `EPOCHS` for use later in `slurm\_train.sbatch`.

L019 BATCH_SIZE="\${BATCH_SIZE:-4}"

Explanation: Assigns the result of this expression to `BATCH\_SIZE` for use later in `slurm\_train.sbatch`.

L020 CROP_SIZE="\${CROP_SIZE:-512}"

Explanation: Assigns the result of this expression to `CROP\_SIZE` for use later in `slurm\_train.sbatch`.

L021 WORKERS="\${WORKERS:-4}"

Explanation: Assigns the result of this expression to `WORKERS` for use later in `slurm\_train.sbatch`.

L023 if [[-n "\${MODULES:-}"]]; then

Explanation: Conditional branch: evaluates this condition and executes the indented block only when it is true.

L024 module load \${MODULES}

Explanation: Loads the requested cluster environment module(s), typically CUDA/toolchain modules.

L025 fi

Explanation: Closes the preceding Bash control-flow block.

L027 source "\${VENV}/bin/activate"

Explanation: Activates the selected Python virtual environment for the SLURM job.

L028 export OMP_NUM_THREADS="\${SLURM_CPUS_PER_TASK-8}"

Explanation: Exports an environment variable inherited by the launched Python process.

L029 export PYTORCH_CUDA_ALLOC_CONF="\${PYTORCH_CUDA_ALLOC_CONF:-expandable_segments:True}"

Explanation: Exports an environment variable inherited by the launched Python process.

L031 ARGS=(

Explanation: Begins a Bash array of command-line arguments that will be passed safely to Python.

L032 --data "\${DATA}"

Explanation: Continues the implementation of `slurm\_train.sbatch`; this line is part of the surrounding multi-line expression or statement.

L033 --device cuda

Explanation: Continues the implementation of `slurm\_train.sbatch`; this line is part of the surrounding multi-line expression or statement.

L034 --epochs "\${EPOCHS}"

Explanation: Continues the implementation of `slurm\_train.sbatch`; this line is part of the surrounding multi-line expression or statement.

L035 --batch-size "\${BATCH_SIZE}"

Explanation: Continues the implementation of `slurm\_train.sbatch`; this line is part of the surrounding multi-line expression or statement.

L036 --crop-size "\${CROP_SIZE}"

Explanation: Continues the implementation of `slurm\_train.sbatch`; this line is part of the surrounding multi-line expression or statement.

L037 --workers "\${WORKERS}"

Explanation: Continues the implementation of `slurm\_train.sbatch`; this line is part of the surrounding multi-line expression or statement.

L038 --output "\${OUTPUT}"

Explanation: Continues the implementation of `slurm\_train.sbatch`; this line is part of the surrounding multi-line expression or statement.

L039 --last-output "\${LAST_OUTPUT}"

Explanation: Continues the implementation of `slurm\_train.sbatch`; this line is part of the surrounding multi-line expression or statement.

L040)

Explanation: Closes the multi-line call/list/tuple/array started above.

L042 if [[-n "\${VAL_DATA:-}"]]; then

Explanation: Conditional branch: evaluates this condition and executes the indented block only when it is true.

L043 ARGS+=(--val-data "\${VAL_DATA}")

Explanation: Appends optional user/job settings to the Bash argument array.

L044 fi

Explanation: Closes the preceding Bash control-flow block.

L045 if [[-n "\${RESUME:-}"]]; then

Explanation: Conditional branch: evaluates this condition and executes the indented block only when it is true.

L046 ARGS+=(--resume "\${RESUME}")

Explanation: Appends optional user/job settings to the Bash argument array.

L047 fi

Explanation: Closes the preceding Bash control-flow block.

L048 if [[-n "\${EXTRA_ARGS:-}"]]; then

Explanation: Conditional branch: evaluates this condition and executes the indented block only when it is true.

L049 read -r -a USER_ARGS <<< "\${EXTRA_ARGS}"

Explanation: Continues the implementation of `slurm\_train.sbatch`; this line is part of the surrounding multi-line expression or statement.

L050 ARGS+=("\${USER_ARGS[@]}")

Explanation: Appends optional user/job settings to the Bash argument array.

L051 fi

Explanation: Closes the preceding Bash control-flow block.

L053 srn python -u train.py "\${ARGS[@]}"

Explanation: Launches the Python command as the SLURM job step using the allocated resources.

slurm_predict.sbatch

L001 #!/bin/bash

Explanation: Shebang selecting Bash when this SLURM script is executed directly.

L002 #SBATCH --job-name=tissue-predict

Explanation: SLURM scheduler directive: requests/configures `--job-name=tissue-predict` for the submitted job.

L003 #SBATCH --partition=gpu

Explanation: SLURM scheduler directive: requests/configures `--partition=gpu` for the submitted job.

L004 #SBATCH --gres=gpu:1

Explanation: SLURM scheduler directive: requests/configures `--gres=gpu:1` for the submitted job.

L005 #SBATCH --cpus-per-task=8

Explanation: SLURM scheduler directive: requests/configures `--cpus-per-task=8` for the submitted job.

L006 #SBATCH --mem=20G

Explanation: SLURM scheduler directive: requests/configures `--mem=20G` for the submitted job.

L007 #SBATCH --time=08:00:00

Explanation: SLURM scheduler directive: requests/configures `--time=08:00:00` for the submitted job.

L008 #SBATCH --output=slurm-%x-%j.out

Explanation: SLURM scheduler directive: requests/configures `--output=slurm-%x-%j.out` for the submitted job.

L010 set -euo pipefail

Explanation: Continues the implementation of `slurm\_predict.sbatch`; this line is part of the surrounding multi-line expression or statement.

L012 : "\${IMAGE:?Set IMAGE to the source image}"

Explanation: Bash parameter check: aborts with the embedded message if this required environment variable is unset/empty.

L013 : "\${CHECKPOINT:?Set CHECKPOINT to the trained checkpoint}"

Explanation: Bash parameter check: aborts with the embedded message if this required environment variable is unset/empty.

L015 VENV="\${VENV:-.venv}"

Explanation: Assigns the result of this expression to `VENV` for use later in `slurm\_predict.sbatch`.

L016 OUTPUT_DIR="\${OUTPUT_DIR:-prediction}"

Explanation: Assigns the result of this expression to `OUTPUT\_DIR` for use later in `slurm\_predict.sbatch`.

L017 TILE_SIZE="\${TILE_SIZE:-512}"

Explanation: Assigns the result of this expression to `TILE\_SIZE` for use later in `slurm\_predict.sbatch`.

L018 OVERLAP="\${OVERLAP:-128}"

Explanation: Assigns the result of this expression to `OVERLAP` for use later in `slurm\_predict.sbatch`.

L019 BATCH_SIZE="\${BATCH_SIZE:-2}"

Explanation: Assigns the result of this expression to `BATCH\_SIZE` for use later in `slurm\_predict.sbatch`.

L021 if [[-n "\${MODULES:-}"]]; then

Explanation: Conditional branch: evaluates this condition and executes the indented block only when it is true.

L022 module load \${MODULES}

Explanation: Loads the requested cluster environment module(s), typically CUDA/toolchain modules.

L023 fi

Explanation: Closes the preceding Bash control-flow block.

L025 source "\${VENV}/bin/activate"

Explanation: Activates the selected Python virtual environment for the SLURM job.

L026 export OMP_NUM_THREADS="\${SLURM_CPUS_PER_TASK:-8}"

Explanation: Exports an environment variable inherited by the launched Python process.

L027 export PYTORCH_CUDA_ALLOC_CONF="\${PYTORCH_CUDA_ALLOC_CONF:-expandable_segments:True}"

Explanation: Exports an environment variable inherited by the launched Python process.

L029 ARGS=(

Explanation: Begins a Bash array of command-line arguments that will be passed safely to Python.

L030 --image "\${IMAGE}"

Explanation: Continues the implementation of `slurm\_predict.sbatch`; this line is part of the surrounding multi-line expression or statement.

L031 --checkpoint "\${CHECKPOINT}"

Explanation: Continues the implementation of `slurm\_predict.sbatch`; this line is part of the surrounding multi-line expression or statement.

L032 --output-dir "\${OUTPUT_DIR}"

Explanation: Continues the implementation of `slurm\_predict.sbatch`; this line is part of the surrounding multi-line expression or statement.

L033 --device cuda

Explanation: Continues the implementation of `slurm\_predict.sbatch`; this line is part of the surrounding multi-line expression or statement.

L034 --tile-size "\${TILE_SIZE}"

Explanation: Continues the implementation of `slurm\_predict.sbatch`; this line is part of the surrounding multi-line expression or statement.

L035 --overlap "\${OVERLAP}"

Explanation: Continues the implementation of `slurm\_predict.sbatch`; this line is part of the surrounding multi-line expression or statement.

L036 --batch-size "\${BATCH_SIZE}"

Explanation: Continues the implementation of `slurm\_predict.sbatch`; this line is part of the surrounding multi-line expression or statement.

L037)

Explanation: Closes the multi-line call/list/tuple/array started above.

L039 if [["\${NO_SAVE_PROBABILITIES:-0}" == "1"]]; then

Explanation: Conditional branch: evaluates this condition and executes the indented block only when it is true.

L040 ARGS+=(--no-save-probabilities)

Explanation: Appends optional user/job settings to the Bash argument array.

L041 fi

Explanation: Closes the preceding Bash control-flow block.

L042 if [[-n "\${EXTRA_ARGS:-}"]]; then

Explanation: Conditional branch: evaluates this condition and executes the indented block only when it is true.

L043 read -r -a USER_ARGS <<< "\${EXTRA_ARGS}"

Explanation: Continues the implementation of `slurm\_predict.sbatch`; this line is part of the surrounding multi-line expression or statement.

L044 ARGS+=("\${USER_ARGS[@]}")

Explanation: Appends optional user/job settings to the Bash argument array.

L045 fi

Explanation: Closes the preceding Bash control-flow block.

L047 srun python -u predict.py "\${ARGS[@]}"

Explanation: Launches the Python command as the SLURM job step using the allocated resources.

requirements.txt

L001 torch>=2.2

Explanation: Continues the implementation of `requirements.txt`; this line is part of the surrounding multi-line expression or statement.

L002 torchvision>=0.17

Explanation: Continues the implementation of `requirements.txt`; this line is part of the surrounding multi-line expression or statement.

L003 numpy>=1.24

Explanation: Continues the implementation of `requirements.txt`; this line is part of the surrounding multi-line expression or statement.

L004 Pillow>=10.0

Explanation: Continues the implementation of `requirements.txt`; this line is part of the surrounding multi-line expression or statement.

Appendix B. Uploaded checkpoint inspection

The binary `best_model.pt` cannot be meaningfully annotated line-by-line. Structural inspection found:

- Top-level keys: ['model', 'classes', 'val_miou'].
- Classes: Bone, Fibrocartilage, Cartilage, Muscle, Marrow, Background.
- Stored `val_miou`: approximately 0.71782.
- No `model_config` key, so current `predict.py` treats it as a legacy checkpoint.
- State-dict names include `features.sobel_x`, `enc1.*`, `enc2.*`, `bottleneck.*`, `up2.*`, `dec2.*`, `up1.*`, `dec1.*`, `classifier.*`.
- State tensors contain about 118,805 elements, far smaller than the current `AccurateTissueNet` (~70.97 million parameters).

Final recommendation

Before training the rewritten project, first make the hierarchical probability conversion `float32`, strengthen smoke tests, pin a reproducible software environment, and regenerate a checkpoint with the current architecture. Then compare your independent implementation against the original only at the level of tensor shapes, probability normalization, losses on toy inputs, and final validation metrics—not by line-for-line code similarity.